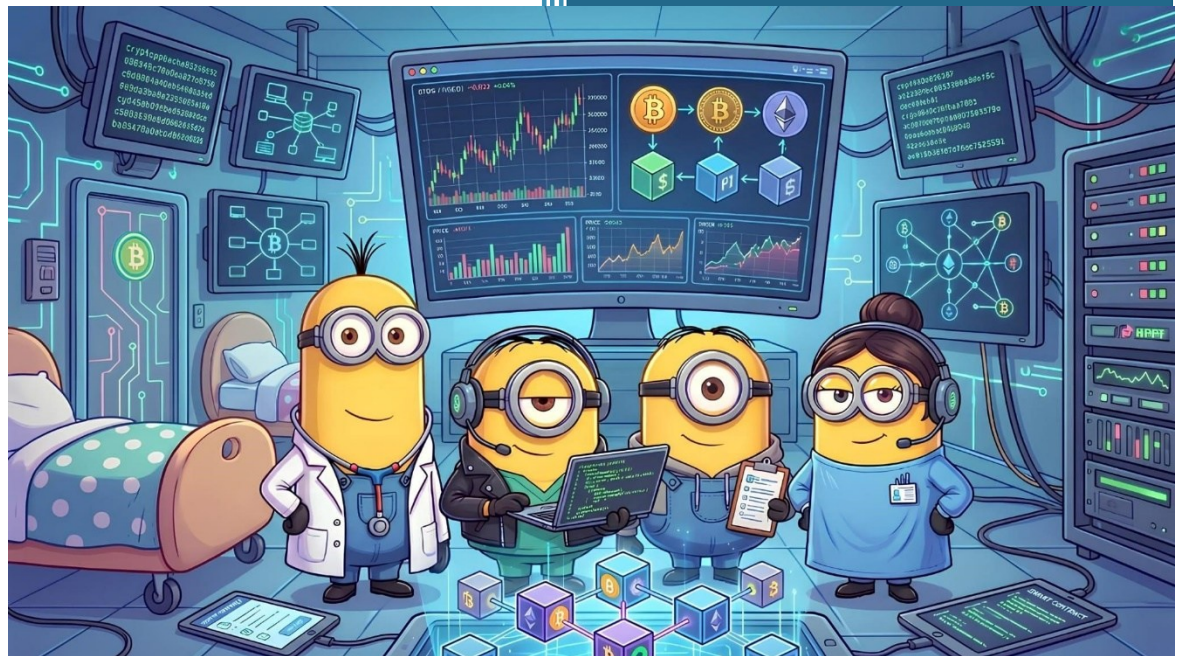


2026

BLOCKCHAINS PROJECT



POSTIGLIONE GIORGIA 2512

MARNA CARLO 2505

LEMBO SERGIO 2520

VITOLO ANDREA 2515

Gruppo 10

02/06/2026

Sommario

WP1 – System and Threat Model	4
Inquadramento del sistema	4
Scenario d’interesse.....	4
Entità coinvolte, obiettivi, capacità e limiti operativi	5
Organizzazioni indipendenti	5
Utenti operativi	6
Soggetti dei documenti	6
Auditor	7
Gli asset.....	9
Trust Assumptions sul sistema	9
Meccanismi per l’autenticazione	9
Sicurezza delle primitive e dei meccanismi di verifica	9
Indipendenza e collusione delle autorità.....	9
Freschezza e stato corrente.....	10
Threat model.....	11
Attaccanti interni	11
Attaccanti esterni	17
Minacce su coerenza, freschezza e integrità documentale	21
Proprietà di sicurezza da preservare	24
WP2 – System Design.....	25
Inquadramento iniziale	25
DLT Permissioned	25
Architettura complessiva del sistema.....	26
Livello di Governance e Coordinamento	26
Tipi di policy.....	27
Il quorum di governance.....	27
Verifica delle policy: RBAC E ABAC	28
Policy Engine	28
Registri logici del sistema	29
Informazioni registrate nei ledger	32
Livello identitario e autorizzativo	38
Identity Registry	39
DID Document.....	40
Verifiable Credential	41
Livello di Storage	46

Storage distribuito: IPFS	46
Storage delle policy	47
Storage dei documenti	47
Storage degli Audit	47
Storage dei DID Document	49
Flussi operativi	50
Accesso a documento cifrato da parte di un utente operativo del sistema	50
Accesso ad un documento cifrato o verifica selettiva da parte di un requester esterno	55
Creazione di un nuovo documento e registrazione nel Document Lifecycle Registry	56
Creazione di una nuova versione di un documento esistente	60
Creazione di un record di delega	63
Revoca di una delega	68
Revoca di una Verifiable Credential	70
Creazione delle policy di governance e di accesso globale	72
Creazione di una policy patient-driven	75
Emissione di una Verifiable Credential	77
Registrazione di un DID e del DID Document associato	79
Auditabilità e ricostruzione ex post	82
Revoca di un documento	85
WP3 – Security Analysis	88
Analisi rispetto alle proprietà di sicurezza definite in WP1	88
Integrità documentale e integrità dello stato	88
Autenticità delle operazioni	88
Non ripudio	89
Autorizzazione per operazioni sui documenti	89
Correttezza della delega	89
Correttezza della revoca e propagazione degli effetti	90
Coerenza del lifecycle documentale	90
Accountability e auditabilità	91
Disponibilità e tempestività operativa	91
Confidenzialità e minimizzazione	91
Analisi degli attacchi principali	92
Collusione tra organizzazioni	92
Privilege escalation da parte di utenti operativi	92
Abuso di diritti delegati	92
Replay	92

Tampering documentale e compromissione dello storage off-chain	93
Man-in-the-Middle	93
Compromissione di chiavi e wallet	93
Requester esterno malevolo.....	94
Auditor curious o malevolo.....	94
Debolezze e limiti del design	94
Dipendenza dal wallet del paziente.....	94
Revoca non retroattiva dopo rilascio della DEK.....	94
Correlabilità dei metadati on-chain	94
Gestione quorum applicativo	95
WP4 – Implementation and Evaluation.....	96
Obiettivo del prototipo.....	96
Scelte implementative e assunzioni	96
Architettura implementata.....	96
<i>DataTypes.sol</i>	96
<i>AuditRegistry.sol</i>	98
<i>OrganizationRegistry.sol</i>	99
<i>PolicyGovernance.sol</i>	101
<i>PolicyRegistry.sol</i>	104
<i>DocumentLifecycleRegistry.sol</i>	107
<i>PatientDrivenPolicyRegistry.sol</i>	109
<i>DelegationRegistry.sol</i>	111
<i>AccessController.sol</i>	116
<i>IdentityRegistry.sol</i>	118
<i>TrustRegistry.sol</i>	119
<i>CredentialStatusRegistry.sol</i>	121
Script Deploy.ts	124
Analisi delle performance	128
Metodologia delle misurazioni	128
Analisi della latenza	128
Throughput.....	131
Storage Overhead	132
Gas consumption	132
Conclusioni	133
Indice delle figure	134
Indice delle tabelle.....	134

WP1 – System and Threat Model

Inquadramento del sistema

Si considera un sistema decentralizzato per la governance del ciclo di vita di documenti sensibili e per il controllo degli accessi in un contesto multi-organizzazione: più organizzazioni indipendenti devono cooperare su documenti che evolvono nel tempo, senza potersi affidare a una singola autorità centrale pienamente fidata. Il sistema deve, quindi, permettere di stabilire chi possa creare, certificare, aggiornare, archiviare o consultare un documento, come tali diritti possano essere delegati e revocati, e come ogni azione rilevante resti verificabile nel tempo tramite uno stato del sistema auditabile e resistente alla manomissione.

Scenario d'interesse

Lo scenario considerato per istanziare il modello è quello sanitario. In tale contesto, più autorità indipendenti — ad esempio ospedali, laboratori, strutture territoriali, enti assicurativi o organismi di controllo — cooperano nella gestione del fascicolo documentale di un paziente.

I documenti possono includere referti, prescrizioni, consensi informati, lettere di dimissione e altri documenti clinico-amministrativi. Tali documenti possono essere creati, certificati, consultati, aggiornati, versionati, archiviati o revocati nel tempo, secondo regole condivise tra le organizzazioni coinvolte. Nel modello considerato, l'accesso ai documenti non deve dipendere da un'unica autorità centrale pienamente fidata, ma da regole condivise formalizzate da una governance congiunta tra più autorità indipendenti.

Il sistema deve inoltre considerare il ruolo del paziente come soggetto a cui i dati si riferiscono, gli utenti operativi che agiscono per conto delle organizzazioni, i verificatori esterni interessati a controllare specifiche proprietà e gli auditor incaricati di ricostruire ex post la correttezza delle operazioni.

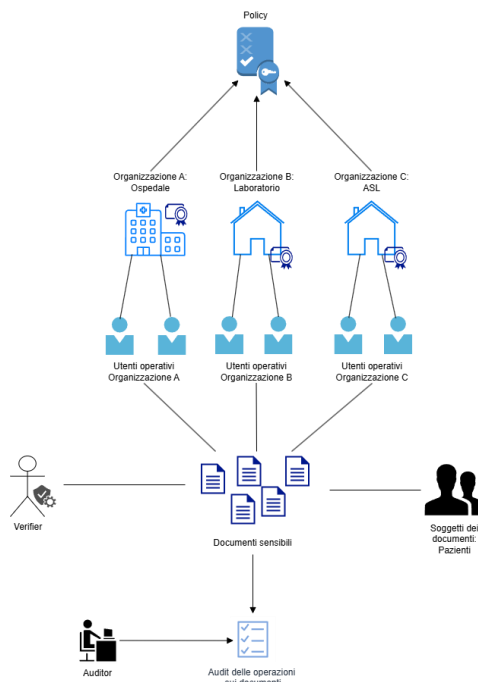


Figura 1, Schema ad alto livello scenario d'interesse

Entità coinvolte, obiettivi, capacità e limiti operativi

Le entità principali del modello sono: organizzazioni indipendenti, utenti operativi, soggetti dei documenti, verificatori esterni e auditor. Nel caso sanitario considerato, le organizzazioni indipendenti possono essere ospedali, laboratori, ASL, strutture territoriali o altri enti coinvolti nella gestione documentale; gli utenti operativi possono essere medici, specialisti, infermieri, tecnici di laboratorio o altre figure autorizzate; i soggetti dei documenti corrispondono ai pazienti; i verificatori esterni possono essere enti assicurativi o altri soggetti legittimati a verificare specifiche proprietà; gli auditor sono incaricati di controllare ex post la correttezza delle operazioni.

La distinzione tra queste entità è rilevante perché separa tre piani diversi: chi definisce o approva le regole di governance, chi esegue operativamente azioni sui documenti e chi è il soggetto a cui i dati si riferiscono. Questa separazione evita di confondere il titolare degli interessi sul dato con l'utente che opera sul sistema o con l'autorità che partecipa alla definizione delle policy.

Organizzazioni indipendenti

Le organizzazioni indipendenti sono le autorità che partecipano alla governance del sistema. Il loro obiettivo è cooperare nella gestione di documenti sensibili e nella definizione, approvazione e applicazione delle regole relative agli accessi, alle deleghe, alle revoche e al ciclo di vita documentale.

Le loro capacità includono l'emissione di attestazioni relative a identità, ruoli o attributi degli utenti appartenenti al proprio dominio; la partecipazione a decisioni di policy; l'approvazione o il rifiuto di richieste rilevanti; la certificazione di eventi documentali; e la revoca di autorizzazioni o attestazioni di propria competenza.

Un limite fondamentale è che nessuna organizzazione, agendo individualmente, deve poter controllare l'intero sistema, alterare unilateralmente lo stato condiviso o imporre transizioni documentali non conformi alle regole comuni. Si assume inoltre che le organizzazioni non si fidino pienamente l'una dell'altra: alcune possono comportarsi in modo scorretto, colludere o abusare del proprio ruolo. Il

sistema deve quindi essere modellato in modo che le decisioni critiche richiedano forme di controllo congiunto.

Utenti operativi

Gli utenti operativi sono i soggetti che compiono azioni sui documenti per conto di una o più organizzazioni. Nel dominio sanitario possono essere medici, specialisti, infermieri, tecnici di laboratorio o altre figure abilitate. Essi agiscono a livello operativo e sono soggetti alle policy definite dalle autorità competenti.

Il loro obiettivo è creare, consultare, aggiornare, certificare o gestire documenti solo nei limiti delle autorizzazioni ricevute. Le loro capacità dipendono dal ruolo, dagli attributi, dal contesto operativo e da eventuali deleghe valide, ma principalmente possono firmare operazioni, delegare permessi, richiedere revoche. Non devono poter auto-attribuirsi privilegi, estendere autonomamente la durata o l'ambito delle deleghe, ignorare revoche o compiere azioni non consentite.

Un caso particolare è rappresentato dagli utenti delegati, cioè utenti che ricevono temporaneamente uno o più diritti derivati da un'autorizzazione preesistente. Tali diritti non devono eccedere il perimetro del diritto originario e non devono rimanere validi dopo la revoca della delega o della sua sorgente.

Si assume che gli utenti operativi non siano pienamente fidati: possono tentare di accedere a documenti non autorizzati, abusare di privilegi legittimi, utilizzare deleghe oltre il loro scopo o continuare ad agire dopo una revoca.

Soggetti dei documenti

I soggetti dei documenti sono le persone o entità a cui i documenti si riferiscono. Nel caso sanitario corrispondono ai pazienti. Essi non coincidono necessariamente con chi genera, aggiorna o certifica il documento, ma sono portatori di interessi rilevanti rispetto alla riservatezza, correttezza e liceità del trattamento dei dati.

Il loro obiettivo è che i documenti che li riguardano siano trattati solo da soggetti autorizzati, secondo finalità lecite, verificabili e coerenti con le regole del sistema e del dominio applicativo. A seconda delle policy adottate, possono esprimere consensi, revocare consensi precedentemente concessi o consultare lo storico delle operazioni che li riguardano.

I soggetti dei documenti non partecipano direttamente alla governance globale del sistema e non dispongono, solo in quanto soggetti del dato, di un potere generale di modifica dei documenti o dello stato del sistema. Si assume che vogliano proteggere i propri dati, ma che i loro dispositivi, credenziali o canali di accesso possano essere compromessi.

Requester esterno

I Requester esterni sono soggetti che non partecipano alla governance interna, ma possono avere interesse o titolo a verificare specifiche proprietà relative a documenti, certificazioni o stati di validità. Nel caso sanitario, un esempio è un ente assicurativo che debba verificare l'esistenza o la validità di un determinato documento senza accedere necessariamente all'intero contenuto informativo.

Il loro obiettivo è ottenere evidenze sufficienti a verificare proprietà limitate, come esistenza, provenienza, validità, stato di revoca o coerenza di una certificazione. Le loro capacità devono essere limitate allo scopo della verifica. Non devono poter ottenere informazioni eccedenti né trasformare il processo di verifica in un accesso generalizzato ai documenti.

Si assume che i verificatori esterni non siano pienamente fidati: possono tentare di raccogliere più informazioni del necessario, correlare verifiche distinte o inferire dati sensibili non richiesti dallo scopo della verifica.

Auditor

Gli auditor sono entità interne incaricate di controllare ex post la correttezza delle operazioni e la coerenza del ciclo di vita documentale. Ogni organizzazione ha un gruppo di auditor. Il loro obiettivo è ricostruire chi ha effettuato una determinata azione, quando, con quale autorizzazione, rispetto a quale documento, versione o stato delle policy.

Le loro capacità includono l'accesso a registrazioni, metadati ed evidenze necessarie all'audit al fine di ricostruire la storia dei documenti ed individuare abusi e accessi impropri. Tuttavia, tale accesso deve essere limitato allo scopo di controllo: gli auditor non devono avere poteri impliciti di modifica dello stato del sistema né devono poter accedere a dati sensibili oltre quanto necessario.

Si assume che gli auditor possano essere honest but curious, oppure potenzialmente interessati a correlare informazioni oltre il proprio mandato. Per questo motivo, il modello deve considerare non solo la correttezza dell'audit, ma anche la minimizzazione delle informazioni esposte durante l'attività di verifica.

Entità	Ruolo	Obiettivi	Capacità	Trust assumptions
<i>Organizzazioni indipendenti</i>	Autorità di governance	Definire e applicare regole su accessi, deleghe, revoche e lifecycle documentale	Emettere attestazioni, partecipare a decisioni di policy, certificare eventi, revocare autorizzazioni di propria competenza	Non pienamente fidate; alcune possono essere malevole o colludere; nessuna deve poter controllare unilateralmente il sistema
<i>Utenti operativi</i>	Attori che agiscono sui documenti	Operare sui documenti secondo autorizzazioni, ruoli, attributi e deleghe valide	Creare, consultare, aggiornare o certificare documenti se autorizzati; usare deleghe ricevute	Non pienamente fidati; possono abusare di privilegi, tentare escalation o usare diritti revocati
<i>Utenti delegati</i>	Sottoclasse degli utenti operativi	Esercitare diritti derivati entro limiti di scopo, durata e sorgente	Usare privilegi delegati, se validi e non revocati	Non devono poter ampliare o mantenere privilegi oltre la delega ricevuta
<i>Soggetti dei documenti</i>	Soggetti a cui i dati si riferiscono	Proteggere riservatezza, correttezza e trattamento lecito dei dati	Esprimere o revocare consensi ove previsto; consultare informazioni di audit pertinenti	Generalmente interessati a proteggere i dati, ma dispositivi o credenziali possono essere compromessi
<i>Verificatori esterni</i>	Entità di verifica selettiva	Verificare proprietà limitate di documenti o certificazioni	Richiedere evidenze su esistenza, validità, provenienza o revoca	Non pienamente fidati; possono tentare over-disclosure o correlazione indebita
<i>Auditor</i>	Controllo ex post	Verificare correttezza, accountability e coerenza del lifecycle	Accedere a registrazioni, metadati ed evidenze necessarie all'audit	Possono essere honest-but-curious; non devono avere poteri di modifica né accesso eccedente

Tabella 1, Entità sistema

Gli asset

Gli asset di sicurezza rilevanti sono i seguenti:

1. **Documenti e contenuti documentali:** comprendono il contenuto dei documenti sensibili, le loro versioni e il relativo stato di validità. Nel caso sanitario, tali documenti possono avere valore operativo, clinico, amministrativo o giuridico.
2. **Policy e decisioni di governance:** includono le regole che stabiliscono chi può accedere, modificare, certificare, delegare o revocare diritti sui documenti, nonché le decisioni assunte dalle autorità nell'applicazione di tali regole.
3. **Materiale identitario e autorizzativo:** comprende le informazioni necessarie ad associare soggetti, ruoli, attributi, deleghe e autorizzazioni alle entità del sistema. La compromissione o falsificazione di questo asset può consentire impersonificazione, privilege escalation o uso illecito di permessi.
4. **Evidenze e registrazioni degli eventi rilevanti:** comprendono le informazioni necessarie a ricostruire accessi, aggiornamenti, certificazioni, deleghe, revoche e altre operazioni significative. Sono essenziali per auditability, accountability e non ripudio.
5. **Riferimenti e repository documentali:** comprendono le risorse in cui i documenti o i loro riferimenti sono conservati. Un attaccante può tentare di ottenere accesso non autorizzato, alterare contenuti o presentare versioni non coerenti con lo stato valido del sistema.

Trust Assumptions sul sistema

Le assunzioni di fiducia definiscono quali condizioni il modello considera necessarie affinché le proprietà di sicurezza siano significative. Tali assunzioni non descrivono ancora i meccanismi concreti con cui il sistema sarà realizzato, ma delimitano il perimetro entro cui il threat model viene valutato.

Meccanismi per l'autenticazione

Si assume che l'ammissione iniziale delle entità al sistema e l'associazione tra identità, ruoli e attributi siano effettuate da autorità competenti secondo procedure corrette. Se identità false, duplicate o non riconducibili a soggetti reali venissero accettate come valide già in fase di enrollment, le proprietà di autenticità, accountability e autorizzazione risulterebbero compromesse alla base.

Questa assunzione non implica che tutte le identità restino sempre sicure dopo l'enrollment: il threat model considera infatti la possibilità che utenti, credenziali, dispositivi o canali operativi vengano successivamente compromessi o abusati.

Sicurezza delle primitive e dei meccanismi di verifica

Si assume l'esistenza di meccanismi astratti sicuri per autenticare gli attori, attribuire operazioni, proteggere la riservatezza dei contenuti e verificare l'integrità di documenti, stati e decisioni. Il threat model non considera la rottura diretta di tali meccanismi, ma considera invece abuso di credenziali valide, compromissione di endpoint, riuso di evidenze obsolete, collusione tra autorità e manipolazione dello storage o dei canali operativi.

Indipendenza e collusione delle autorità

Si assume che non tutte le autorità indipendenti siano simultaneamente corrotte o colluse. Il sistema deve tollerare comportamenti malevoli, errori o abusi da parte di una o più autorità entro i limiti previsti dal modello di fiducia; oltre tali limiti, alcune proprietà, come correttezza delle decisioni, disponibilità o integrità della governance, potrebbero non essere più garantite.

Nessuna singola autorità deve essere pienamente fidata o capace di controllare unilateralmente il sistema, ma il modello non può garantire sicurezza se una frazione sufficiente delle autorità collude contro le regole comuni.

Freschezza e stato corrente

Si assume che il sistema possa distinguere, almeno a livello di modello, tra informazioni correnti e informazioni obsolete. Questa assunzione è necessaria perché accessi, deleghe, revoche, versioni documentali e stati di validità dipendono dal tempo e dall'evoluzione dello stato del sistema.

Il threat model considera comunque attacchi in cui un avversario tenta di sfruttare ritardi, asincronie, stati non aggiornati o evidenze riutilizzate fuori contesto. Il design dovrà quindi specificare come viene verificata la freschezza delle informazioni e come vengono gestiti stati obsoleti o revoche non ancora propagate.

Threat model

Il modello avversariale considera sia attaccanti esterni sia attori interni legittimi che possono abusare delle proprie capacità. Questa distinzione è essenziale perché, in un sistema multi-organizzazione, il rischio principale non deriva solo da soggetti esterni che tentano di entrare nel sistema, ma anche da autorità, utenti o verificatori che possiedono già un ruolo riconosciuto e tentano di usarlo in modo scorretto.

Per ogni classe di avversario vengono descritti: le capacità disponibili, la conoscenza o posizione dell'attaccante, gli obiettivi perseguiti e l'impatto sulle proprietà di sicurezza desiderate. Il modello non assume che tutti gli attori ammessi siano onesti; assume invece che le garanzie del sistema debbano reggere rispetto ad abusi, compromissioni e collusioni entro i limiti del modello di fiducia definito in precedenza.

Attaccanti interni

Organizzazione malevola

Descrizione dell'attaccante

Una o più organizzazioni indipendenti possono comportarsi in modo malevolo o opportunistico pur avendo un ruolo legittimo nel sistema. Poiché tali organizzazioni partecipano alla governance, alla definizione delle policy, alla validazione di eventi documentali e alla gestione delle autorizzazioni di propria competenza, il loro abuso rappresenta una minaccia particolarmente critica.

Capacità tecniche

Un'autorità malevola può partecipare ai processi di approvazione, proporre o sostenere decisioni non conformi, ritardare o ostacolare decisioni lecite, emettere attestazioni non corrette, omettere informazioni rilevanti o tentare di influenzare altre autorità.

Obiettivi

Gli obiettivi principali sono: approvare accessi o transizioni documentali non conformi; impedire o ritardare revoche dovute; produrre privilegi apparentemente regolari ma sostanzialmente illeciti; alterare la governance a proprio vantaggio; ridurre l'efficacia del controllo multi-authority; o compromettere disponibilità e correttezza del processo decisionale.

L'obiettivo sostanziale non è necessariamente "falsificare" il sistema, ma produrre decisioni formalmente plausibili che risultino però materialmente scorrette rispetto alla policy condivisa. Questo è esattamente il tipo di rischio che emerge quando la fiducia nelle autorità è solo parziale e contestuale, non assoluta.

Impatto

L'impatto è elevato perché questa minaccia colpisce la radice della fiducia tra organizzazioni. Una decisione malevola presa da un'autorità legittima può risultare formalmente plausibile e quindi più difficile da distinguere da un errore o da una decisione lecita. In caso di collusione, il rischio diventa sistemico e può compromettere la governance congiunta.

Obiettivo di sicurezza

Il sistema deve impedire che una singola autorità, o un insieme di autorità entro i limiti di corruzione tollerati dal modello, possa imporre decisioni critiche non conformi alle regole comuni o impedire arbitrariamente decisioni dovute.

Formalizzazione dell'obiettivo

Sia $Decide(a, op, d, t)$ il predicato che vale 1 se l'autorità a approva l'operazione op sul documento d al tempo t , e sia $Policy(op, d, t) = 1$ la policy corretta del sistema. L'obiettivo dell'autorità malevola è realizzare un evento del tipo:

$$G_{\text{mal-auth}}: \neg Decide(a, op, d, t) = 1 \wedge Policy(op, d, t) = 0$$

oppure, dualmente, impedire o ritardare una decisione dovuta, cioè:

$$G_{\text{deny-auth}}: \neg Decide(a, op, d, t) = 0 \wedge Policy(op, d, t) = 1$$

In termini di sicurezza, il sistema per essere robusto è necessario che una singola autorità non possa rendere vera da sola una di queste condizioni per operazioni critiche. Questa esigenza deriva direttamente dal requisito di multi-authority governance senza piena fiducia reciproca. Inoltre, come suddetto è possibile che un gruppo di autorità possano colludere per falsificare il meccanismo di governance del sistema.

Il sistema garantisce sicurezza contro coalizioni di autorità inferiori alla soglia richiesta per la specifica classe di decisione. Le soglie possono variare in base alla criticità dell'operazione.

Sia A l'insieme delle autorità del sistema e sia $C \subseteq A$ un sottoinsieme di autorità colluse.

Per ogni classe di decisione critica op , il sistema definisce una soglia di approvazione k_{op} , dipendente dalla natura e dalla criticità dell'operazione.

Sia $Approved(op, d, t)$ il predicato che vale 1 se l'operazione op sul documento d è approvata dal sistema, e sia $Policy(op, d, t) = 1$ la policy corretta applicabile al tempo t .

La collusione ha successo se un insieme C di autorità, pur essendo inferiore alla soglia richiesta per quella specifica operazione, riesce a determinare una decisione non conforme:

$$G_{\text{collusion}}: Approved(op, d, t) = 1 \wedge Policy(op, d, t) = 0 \wedge |C| < k_{op}$$

Utente operativo compromesso

Gli utenti operativi agiscono sui documenti per conto delle organizzazioni e dispongono di autorizzazioni limitate dal proprio ruolo, dagli attributi, dal contesto e dalle eventuali deleghe ricevute. Un utente operativo può però comportarsi in modo malevolo, abusare di privilegi legittimi oppure essere compromesso da un attaccante che ne usa credenziali o canali di accesso tentando di effettuare operazioni eccedenti i limiti previsti dalle policy o di aggirare i meccanismi di controllo del sistema.

Capacità tecniche

Questo avversario ha accesso autenticato come utente legittimo e, quindi, ha la possibilità di compiere operazioni sui documenti secondo le policy definite. Le sue capacità comprendono la firma di operazioni, l'interazione ordinaria con i servizi di accesso e verifica e la possibilità di sfruttare la conoscenza del contesto operativo.

Obiettivi attesi

Gli obiettivi possono essere:

- effettuare operazioni su documenti sensibili senza essere autorizzato;
- usare credenziali autentiche, ma scadute o revocate, per firmare le operazioni o accedere ai documenti;
- impersonare un altro utente tramite credenziali rubate per compiere operazioni sui documenti con privilegi superiori.

In questo caso il problema è l'uso fraudolento di materiale autentico, sottratto o riutato fuori contesto, per effettuare operazioni privilegiate o aggiuntive rispetto a quelle previste per l'utente operativo considerato.

Impatto

L'impatto riguarda soprattutto **riservatezza, integrità, autorizzazione** e accountability. Un'operazione abusiva eseguita con credenziali apparentemente valide può produrre accessi o modifiche non lecite e, allo stesso tempo, rendere più complessa la distinzione tra azione intenzionale dell'utente, compromissione delle credenziali ed errore operativo.

Obiettivo di sicurezza

Il sistema deve accettare operazioni solo quando l'utente risulta autorizzato rispetto allo specifico documento, operazione, contesto e stato corrente delle policy. Deve inoltre rendere **tracciabili** le operazioni rilevanti, pur riconoscendo che una compromissione completa dell'endpoint può limitare la capacità di distinguere l'intenzione reale dell'utente dall'operazione prodotta.

Formalizzazione di accesso illecito

Sia $Auth(u, op, d, t)$ il predicato che vale 1 se l'utente u è autorizzato a eseguire op sul documento d al tempo t , e sia $Exec(u, op, d, t)$ l'esecuzione osservata. L'obiettivo dell'avversario è:

$$G_{unauth}: Exec(u, op, d, t) = 1 \wedge Auth(u, op, d, t) = 0$$

Il sistema è sicuro rispetto a questa classe se $\Pr[G_{unauth}] \leq \text{negl}(\lambda)$ salvo i casi in cui si assuma esplicitamente compromissione completa dell'endpoint.

Utente operativo delegato compromesso

Descrizione dell'attaccante

Un caso specifico di utente operativo è il delegato, cioè un soggetto che riceve diritti derivati da un'autorizzazione preesistente. La delega può essere limitata per scopo, durata, documento, operazione o contesto. Se sono ammesse catene di delega, ogni diritto esercitato dal delegato deve dipendere da una catena valida e non revocata.

Capacità

Un delegato malevolo può sfruttare una delega valida ma troppo ampia, riutilizzare una delega dopo la scadenza o la revoca, sfruttare ritardi nella propagazione dello stato di revoca, oppure tentare di ri-delegare diritti che non possiede o che non è autorizzato a trasferire.

Obiettivi attesi

Gli obiettivi sono:

- esercitare diritti derivati anche dopo che l'autorizzazione originaria è stata revocata;
- compiere operazioni che non è autorizzato a fare;
- trasformare una delega limitata in un diritto più ampio o ampliarla per un periodo di tempo maggiore;
- mantenere attivo un privilegio derivato dopo la revoca della sorgente;
- sfruttare la mancata sincronizzazione dello stato di revoca per continuare ad agire.

Quindi, lo scopo primario del delegato malevolo è **ampliare i propri privilegi** oltre il perimetro originario; in questo caso il problema principale non è la falsificazione, ma la privilege amplification lungo una catena apparentemente regolare.

Impatto

L'impatto è particolarmente rilevante perché le operazioni del delegato possono apparire **formalmente giustificate** da una delega esistente, pur essendo scorrette rispetto allo scopo, alla durata o alla sorgente del diritto originario. Questo compromette autorizzazione, revoca, accountability e correttezza delle catene di delega.

Obiettivo di sicurezza

Il sistema deve garantire che un diritto delegato sia esercitabile solo se esiste, al momento dell'uso, una catena di delega valida, non scaduta, non revocata e coerente con i vincoli del diritto originario. La revoca di un diritto sorgente deve impedire l'uso dei diritti derivati che dipendono da esso.

Formalizzazione dell'obiettivo

Sia $Del(u, v, p, \tau)$ una delega dal soggetto u al soggetto v relativa al privilegio p con vincoli temporali e contestuali τ . Sia $ValidDelegation(v, p, t)$ l'insieme delle deleghe valide che, al tempo t , giustificano p per v . L'avversario ha successo se realizza una delle due condizioni:

$$G_{amp}: Exec(v, p, t) = 1 \wedge p \notin ValidDelegation(v, p, t)$$

Oppure:

$$G_{stale-del}: Exec(v, p, t) = 1 \wedge Revoked(source(p), t) = 1$$

dove $source(p)$ è l'anello sorgente da cui il privilegio deriva. Un sistema corretto deve garantire che l'esercizio di un privilegio delegato sia possibile solo se esiste, al momento dell'uso, una catena valida, non revocata e coerente con i vincoli di scopo e durata.

Auditor malevolo

Descrizione dell'attaccante

Gli auditor sono incaricati di controllare ex post la correttezza delle operazioni e la coerenza del ciclo di vita documentale. Pur non avendo poteri di modifica sullo stato del sistema, possono abusare della propria capacità di osservazione per apprendere o correlare informazioni oltre il perimetro necessario all'audit.

Capacità

Un auditor malevolo o curioso può accedere a registrazioni, metadati ed evidenze relative a operazioni, versioni documentali, policy e autorizzazioni. Può analizzare tali informazioni nel tempo, correlarle tra eventi distinti e combinarle con conoscenza ausiliaria proveniente da altri contesti.

Obiettivi attesi

Gli obiettivi sono:

- accedere a evidenze o porzioni di log non pertinenti allo specifico scopo di audit, aggirando il principio di minimizzazione;
- correlare operazioni distinte per ricostruire il profilo operativo di utenti, organizzazioni o documenti, trasformando l'attività di audit in una forma di sorveglianza sistematica o di profilazione ex post;
- inferire informazioni sensibili a partire da metadati, versioni, stati delle policy e prove crittografiche.

Quindi, lo scopo primario dell'auditor malevolo non è alterare il sistema, ma abusare della visibilità sistemica per apprendere, correlare o dedurre informazioni oltre il perimetro legittimo dell'audit.

Impatto

L'impatto riguarda soprattutto **riservatezza**, minimizzazione delle informazioni e fiducia nel processo di audit. Evidenze create per garantire accountability potrebbero diventare, se esposte senza limiti, uno strumento di profilazione o ricostruzione indebita delle attività.

Obiettivo di sicurezza

Il sistema deve consentire agli auditor di verificare la correttezza delle operazioni entro il perimetro autorizzato, senza concedere poteri di modifica e senza esporre informazioni eccedenti rispetto allo scopo dell'audit.

Formalizzazione dell'obiettivo

Sia $Scope(a, c)$ l'insieme delle evidenze che l'auditor a è autorizzato a consultare nel caso di audit c . Sia $Read(a, x, c) = 1$ se l'auditor accede all'evidenza x nel contesto c . L'avversario ha successo se accede a informazioni fuori perimetro:

$$G_{over-read}: Read(a, x, c) = 1 \wedge x \notin Scope(a, c)$$

Sia inoltre $Infer(a, s, c) = 1$ se l'auditor a riesce a inferire l'informazione sensibile s nel contesto c e $Necess(c)$ l'insieme delle sole informazioni strettamente necessarie per l'audit. L'avversario ha successo anche se ottiene conoscenza eccedente rispetto allo scopo:

$$G_{over-infer}: Infer(a, s, c) = 1 \wedge s \notin Necess(c)$$

Un caso particolare è la correlazione indebita di eventi distinti. Siano e_1 ed e_2 due eventi di audit; l'avversario a ha successo se riesce a collegarli senza che tale collegamento sia giustificato dallo scopo dell'audit:

$$G_{link}: Link(a, e_1, e_2, c) = 1 \wedge \neg JustifiedLink(e_1, e_2, c)$$

Un sistema corretto deve garantire che l'auditor possa verificare la correttezza delle operazioni solo entro il perimetro autorizzato, senza acquisire informazioni ulteriori rispetto a quelle necessarie, e che le evidenze di audit siano verificabili in modo indipendente senza esporre dati sensibili non richiesti.

Requester esterno malevolo

Descrizione dell'attaccante

I verificatori esterni sono entità che non partecipano alla governance interna, ma possono avere titolo a verificare specifiche proprietà relative a un documento, a una certificazione o a uno stato di validità. Un verificatore malevolo può tentare di ottenere più informazioni di quelle necessarie allo scopo della verifica.

Capacità tecniche

Il verificatore può richiedere verifiche, raccogliere metadati, conservare risultati ottenuti in momenti diversi, correlare più interazioni e combinare le informazioni ricevute con fonti esterne.

Obiettivi attesi

Gli obiettivi principali sono: ottenere dati sensibili non necessari; collegare verifiche distinte riferite allo stesso soggetto o documento; ricostruire profili individuali o organizzativi; trasformare una verifica selettiva in un accesso informativo più ampio.

Impatto

L'impatto riguarda la riservatezza dei soggetti dei documenti e la minimizzazione delle informazioni rivelate. Nel caso sanitario, una verifica eccessivamente informativa può esporre dati clinici, amministrativi o relazionali non necessari rispetto allo scopo dichiarato.

Obiettivo di sicurezza

Il sistema deve limitare le informazioni rivelate ai verificatori esterni a ciò che è necessario per la specifica verifica autorizzata, evitando accessi generalizzati o correlazioni indebite quando non giustificate dal contesto.

Formalizzazione dell'obiettivo

Sia $Disc(P)$ l'insieme delle informazioni effettivamente rivelate in una presentazione P , e $Need(svc)$ l'insieme minimo di attributi necessari al servizio. Una violazione di minimizzazione si ha quando:

$$G_{\text{over-disclose}}: DG_{\text{over-disclose}} : \exists x \in Disc(P) \text{ such that } x \notin Need(svc)$$

Una violazione di unlinkability si ha invece quando due presentazioni P_1, P_2 provenienti dallo stesso holder risultano correlabili da parte del verificatore oltre quanto strettamente necessario alla funzione di servizio. In termini generali, il sistema dovrebbe rendere trascurabile la probabilità di apprendere informazioni eccedenti o di correlare presentazioni che dovrebbero restare logicamente separate.

Compromissione del materiale autentificativo e autorizzativo

Descrizione

Pur avendo fatto come assunzione di sicurezza che le primitive crittografiche sono sicure, è necessario considerare il caso in cui un attaccante interno può rubare chiavi o credenziali valide di altri attori del sistema. In tale scenario, il sistema potrebbe osservare una firma corretta, una presentazione verificabile valida o un accesso apparentemente legittimo, pur essendo l'operazione il risultato di furto, abuso, compromissione o uso improprio delle chiavi.

Capacità

L'avversario può tentare di:

- usare una chiave privata sottratta per firmare richieste o autorizzazioni;
- utilizzare attestazioni o prove autorizzative precedentemente ottenute per produrre richieste apparentemente valide;
- accedere a documenti cifrati se ottiene la relativa chiave di decifratura;
- riutilizzare chiavi non più valide.

Obiettivi

Gli obiettivi principali dell'avversario sono:

- impersonare un soggetto legittimo producendo firme valide;
- firmare operazioni che l'utente non avrebbe autorizzato consapevolmente;
- generare materiale autentificativo usando credenziali autentiche sottratte;
- accedere a documenti cifrati mediante chiavi di decifratura compromesse;
- emettere, approvare o validare operazioni in nome di un'autorità compromessa.

Lo scopo sostanziale dell'attaccante è trasformare il possesso o l'uso indebito di materiale crittografico autentico in una violazione dell'autorizzazione e della riservatezza.

Impatto

L'impatto è elevato perché una chiave compromessa consente di produrre evidenze formalmente corrette. Una firma generata con una chiave autentica può superare i controlli crittografici ordinari, rendendo difficile distinguere tra un'azione realmente voluta dal titolare e un'operazione prodotta da un attaccante che controlla il wallet, il dispositivo o la chiave.

Obiettivo di sicurezza

Il sistema deve garantire che il possesso o l'uso di materiale crittografico sia il più possibile vincolato al soggetto legittimo, al contesto corretto e allo stato corrente del sistema. L'uso di una chiave o di materiale autenticativo deve essere verificato rispetto alla sua validità, al suo scopo, al suo stato di revoca, e all'operazione che viene effettivamente firmata.

È importante però riconoscere un limite importante: se l'endpoint dell'utente è completamente compromesso, non sempre è possibile distinguere con certezza l'intenzione reale del soggetto dall'operazione firmata. Per questo motivo, le garanzie del sistema devono essere rafforzate tramite controlli di conferma esplicita, auditabilità, revoca tempestiva, e limitazione dell'ambito delle chiavi compromesse.

Attaccanti esterni

Gli attaccanti esterni non possiedono necessariamente un ruolo legittimo nel sistema, ma possono osservare, interferire o compromettere canali e componenti operativi. Possono agire passivamente, raccogliendo informazioni, oppure attivamente, tentando di alterare messaggi, riutilizzare evidenze, degradare la disponibilità o indurre attori legittimi a compiere operazioni non intenzionali.

Le principali minacce considerate sono: riuso di informazioni o evidenze valide fuori contesto, indisponibilità o ritardo nell'applicazione di operazioni rilevanti, manipolazione delle comunicazioni e compromissione dell'interfaccia o dell'endpoint operativo.

Replay e uso di stato obsoleto

Descrizione dell'attaccante

Questa minaccia riguarda il caso in cui un avversario riesca a far accettare al sistema informazioni, prove, autorizzazioni, deleghe, credenziali, policy o versioni documentali che non sono più valide, che erano valide solo in un determinato contesto oppure che risultano corrette soltanto rispetto a una vista obsoleta dello stato del sistema.

La minaccia comprende due modalità strettamente correlate. Nel primo caso, l'attaccante riutilizza attivamente materiale precedentemente, presentandolo in un contesto diverso, dopo la scadenza o dopo una revoca. Nel secondo caso, l'attaccante sfrutta il fatto che un verificatore, un decisore o un componente del sistema operi su uno stato non aggiornato, ad esempio una policy non sincronizzata, una revoca non ancora propagata o una versione documentale superata.

In entrambi i casi, il problema principale non è la falsificazione diretta del materiale utilizzato, che può anche risultare crittograficamente valido, ma la sua accettazione fuori dal corretto contesto temporale, operativo o di stato.

Capacità

L'avversario può osservare, ottenere, memorizzare e ripresentare informazioni o evidenze precedentemente valide. Può disporre di credenziali autentiche ma scadute, deleghe formalmente esistenti ma già revocate, prove prodotte per uno specifico contesto, copie locali di registri non aggiornate, cache di policy obsolete o riferimenti a versioni documentali non più corretti.

Può inoltre sfruttare ritardi nella propagazione dello stato, asincronie tra organizzazioni, indisponibilità temporanea dei servizi di verifica, mancata sincronizzazione tra nodi o verificatori che non controllano correttamente freschezza, revoche, versioni e contesto dell'operazione.

Non è necessario che l'attaccante rompa i meccanismi crittografici del sistema. E' sufficiente che riesca a far valutare un'operazione rispetto a informazioni non correnti oppure a riutilizzare evidenze valide in un contesto diverso da quello per cui erano state prodotte.

Obiettivi attesi

Gli obiettivi principali dell'avversario sono:

- far accettare una credenziale, una prova o una delega scaduta o revocata;
- riutilizzare una prova valida fuori dal contesto originario;
- esercitare un privilegio già revocato;
- ottenere accesso tramite una policy localmente non aggiornata;
- far valere una versione documentale non più corrente come se fosse ancora efficace;
- aggirare l'effettività della revoca sfruttando ritardi di propagazione o viste obsolete dello stato;
- ottenere approvazioni o accessi sulla base di informazioni temporalmente superate.

In sostanza, l'obiettivo dell'attaccante è trasformare la latenza, la mancata sincronizzazione o l'assenza di controlli di freschezza in una violazione effettiva dell'autorizzazione, della revoca o della coerenza del ciclo di vita documentale.

Impatto

L'impatto è significativo perché questa minaccia colpisce direttamente la correttezza temporale delle decisioni del sistema. Un documento, una delega o una credenziale possono risultare formalmente validi dal punto di vista della struttura o della verifica crittografica, ma non essere più validi rispetto allo stato corrente del sistema.

Nel contesto considerato, ciò può consentire l'accesso a documenti sensibili dopo una revoca, l'uso di deleghe non più valide, l'accettazione di versioni documentali superate o l'applicazione di policy non aggiornate. Questo compromette autorizzazione, revoca, integrità dello stato, lifecycle documentale e accountability.

La minaccia è particolarmente critica in un sistema multi-organizzazione, perché le decisioni possono dipendere da stati distribuiti, aggiornamenti asincroni e propagazione di revoche tra più autorità. Una revoca corretta ma non ancora recepita da tutti i componenti può generare una finestra temporale in cui privilegi non più leciti continuano a produrre effetti.

Obiettivo di sicurezza

Il sistema deve garantire che ogni decisione di accesso, verifica, delega, aggiornamento o transizione documentale venga presa rispetto allo stato corrente e al contesto corretto. Ogni informazione rilevante deve essere verificata rispetto a scadenze, revoche, versioni, policy applicabili, vincoli temporali e contesto operativo.

In particolare, il sistema deve impedire che prove, credenziali, deleghe o riferimenti documentali precedentemente validi vengano riutilizzati fuori contesto, dopo la scadenza o dopo una revoca. Deve inoltre evitare che componenti del sistema accettino operazioni sulla base di viste obsolete dello stato, soprattutto quando sono coinvolte revoche, aggiornamenti di policy, versioni documentali o diritti delegati.

La sicurezza rispetto a questa minaccia richiede quindi controlli espliciti di freschezza, validità temporale, stato di revoca, contesto di utilizzo e coerenza con il lifecycle corrente del documento.

Formalizzazione dell'obiettivo

Sia x un oggetto rilevante per una decisione del sistema, dove x può rappresentare, una credenziale, una delega, una policy, un'autorizzazione.

Sia $Valid(x, ctx, t)$ il predicato che vale 1 se x è valido nel contesto ctx al tempo t , tenendo conto di scadenze, revoche, stato corrente, policy applicabili e versione documentale.

Sia $Accept(v, x, ctx, t) = 1$ il fatto che il verificatore o decisore v accetti x come valido nel contesto ctx al tempo t .

L'avversario ha successo se il sistema accetta un oggetto che non è valido nel contesto o nel tempo in cui viene usato:

$$G_{stale/replay}: Accept(v, x, ctx, t) = 1 \wedge Valid(x, ctx, t) = 0$$

Un sistema corretto deve rendere trascurabile la probabilità che una prova, credenziale, delega, policy o versione documentale venga accettata quando non è valida rispetto allo stato corrente, al tempo di utilizzo o al contesto operativo. In particolare, ogni verifica deve incorporare controlli su freschezza, revoca, scadenza, contesto e coerenza con lo stato corrente del sistema.

Attacco DoS

Descrizione dell'attaccante

Un attaccante può tentare di degradare la disponibilità del sistema o ritardare l'elaborazione di operazioni rilevanti. Nel contesto del controllo accessi, questo non produce solo un disservizio: può creare finestre temporali in cui revoche, aggiornamenti, certificazioni o controlli non risultano effettivi in tempo utile.

Capacità

L'avversario può generare traffico anomalo, ostacolare la comunicazione tra componenti, rallentare la propagazione di informazioni rilevanti o impedire temporaneamente l'accesso a servizi necessari per verificare autorizzazioni e stati documentali.

Obiettivi attesi

Gli obiettivi principali sono: impedire la consultazione o l'aggiornamento di documenti; ritardare l'applicazione di revoche; ostacolare audit o verifiche; creare incoerenze temporanee tra ciò che è stato deciso e ciò che viene effettivamente applicato.

Impatto

L'impatto riguarda disponibilità, tempestività della revoca, coerenza dello stato e affidabilità operativa. Una revoca corretta ma applicata troppo tardi può consentire l'uso indebito di privilegi che avrebbero dovuto cessare.

Obiettivo di sicurezza

Il sistema deve considerare la disponibilità e la tempestività come proprietà rilevanti, soprattutto per operazioni critiche come revoche, aggiornamenti di policy, certificazioni e accessi a documenti sensibili.

Formalizzazione dell'obiettivo

Sia $Revoke(p, t)$ l'evento di revoca del privilegio p al tempo t , e sia $Enforced(p, t + Delta)$ il fatto che la revoca sia divenuta effettiva entro una soglia operativa Δ . L'avversario ha successo se:

$$G_{\text{delay}}: \text{Revoke}(p, t) = 1 \wedge \neg \text{Enforced}(p, t + \Delta)$$

oppure, più in generale, se riesce a provocare una violazione del livello minimo di disponibilità richiesto dal sistema. Questo tipo di minaccia è importante perché una revoca tardiva può consentire accessi abusivi pur senza alterare la correttezza logica della policy.

Man-in-the-Middle

Descrizione dell'attaccante

Un attaccante esterno può osservare, intercettare, modificare o iniettare comunicazioni tra le entità del sistema. Questa minaccia riguarda sia la riservatezza delle informazioni trasmesse sia l'integrità e l'autenticità delle operazioni scambiate.

Risorse e capacità tecniche

L'avversario può osservare traffico, raccogliere metadati, modificare messaggi, ritardare comunicazioni, iniettare richieste o tentare di impersonare una delle parti coinvolte.

Obiettivi attesi

L'obiettivo dell'attaccante è raccogliere informazioni strategiche sul funzionamento del sistema decentralizzato, al fine di progettare e affinare attacchi successivi, nonché acquisire dati sensibili degli utenti contenuti nei documenti scambiati. Tali informazioni possono essere successivamente sfruttate per finalità illecite, come la vendita a terze parti o l'utilizzo a fini estorsivi, sempre minacciandone la divulgazione.

Ulteriori obiettivi possono essere la modifica dei dati in transito presenti nei documenti, per iniettare informazioni o manipolare identità e attributi, e l'esecuzione di operazioni fraudolente tramite impersonificazione, simulando uno dei partecipanti alla comunicazione.

Impatto

L'impatto di un attacco *Man-in-the-Middle* è particolarmente critico, in quanto compromette simultaneamente la riservatezza, l'integrità e l'autenticità delle comunicazioni tra le parti. Infatti, l'attaccante mina l'autenticità delle comunicazioni, inducendo le parti a fidarsi di informazioni manipolate o provenienti da soggetti non legittimi, e compromette i processi di validazione e coordinamento, soprattutto nel sistema multi-autorità in esame, dove l'affidabilità dello scambio informativo è fondamentale.

Obiettivo di sicurezza

Il sistema deve impedire che comunicazioni alterate, non autentiche o osservate fuori contesto compromettano access control, governance, auditability o lifecycle documentale.

Formalizzazione dell'obiettivo

Sia:

- $\text{Intercept}(m, t)$ l'evento in cui un messaggio/documento m viene intercettato al tempo t ;
- $\text{Modify}(m, t)$ l'evento in cui il messaggio m viene modificato dall'avversario al tempo t ;
- $\text{Inject}(m, t)$ l'evento in cui un messaggio m viene iniettato nel canale di comunicazione al tempo t ;
- $\text{Accept}(m, t + \Delta)$ il fatto che il messaggio m venga accettato come valido entro una soglia operativa Δ ;
- $\text{Auth}(m)$ la proprietà per cui il messaggio è autentico;
- $\text{Int}(m)$ la proprietà di integrità del messaggio.

- $Learn(attackers, m, t)$ il fatto che l'attaccante apprenda il contenuto informativo di m al tempo t .

L'avversario ha successo se riesce a far accettare un messaggio intercettato e alterato come legittimo:

$$G_{MitM-Modify}: Intercept(m, t) = 1 \wedge Modify(m, t) = 1 \wedge Accept(m, t + \Delta) = 1 \wedge \neg Int(m)$$

oppure

se riesce a fingersi uno dei partecipanti:

$$G_{MitM-Impersonation}: Inject(m, t) = 1 \wedge Accept(m, t + \Delta) = 1 \wedge \neg Auth(m)$$

oppure

se riesce ad accedere a informazioni riservate:

$$G_{MitM-Confidentiality}: Learn(attackers, m, t) = 1$$

Minacce su coerenza, freschezza e integrità documentale

Tampering documentale e metadocumentale

Risorse e capacità tecniche

L'avversario può accedere allo storage applicativo o a repository esterni, modificare file, sostituire metadati. Questo scenario è particolarmente realistico quando il contenuto integrale del documento risiede in uno storage esterno, mentre sul registro condiviso restano hash, riferimenti, eventi e metadati di lifecycle.

Obiettivi attesi

L'obiettivo è far accettare un contenuto alterato come se fosse autentico, valido e coerente con lo stato certificato del sistema. In particolare, l'avversario può mirare a:

- sostituire il contenuto di un documento tentando di aggirare i controlli di integrità;
- modificare metadati rilevanti (versione, timestamp, stato di approvazione, riferimenti di policy) per alterare il significato operativo del documento;
- indurre verificatori o sistemi a prendere decisioni basate su informazioni falsate, ma apparentemente legittime.

Quindi, lo scopo non è solo la modifica del dato in sé, ma la manipolazione della fiducia nel legame tra contenuto, metadati e stato certificato.

Impatto

L'impatto è critico perché colpisce direttamente il principio di integrità e affidabilità del documento che rappresenta una base fondamentale per autorizzazioni, audit e governance. Se un contenuto alterato viene accettato come autentico, le decisioni operative risultano basate su informazioni non corrette, viene compromessa la coerenza tra stato registrato e contenuto effettivo del documento e l'audit perde efficacia, poiché le evidenze risultano formalmente valide ma semanticamente false.

Quindi, l'intero sistema perde credibilità, poiché viene meno la fiducia nel fatto che un documento verificato corrisponda effettivamente al contenuto originale.

Formalizzazione dell'obiettivo

Sia $h = H(doc)$ l'hash del documento registrato nel sistema, sia $state_t$ la rappresentazione certificata e corrente del documento al tempo t , comprensiva del contenuto attestato tramite hash, dei metadati rilevanti e del suo stato operativo nel lifecycle del sistema. L'obiettivo dell'avversario è produrre $doc' \neq doc$ tale che il verificatore lo tratti come consistente con lo stato certificato, cioè:

$$G_{\text{tamper-doc}}: \text{Accept}(doc', meta') = 1 \wedge \neg \text{Consistent}(doc', meta', state_t)$$

dove *Consistent* richiede almeno coerenza tra contenuto, hash registrato, versione, owner, timestamp e stato corrente di lifecycle. In termini più stretti, il sistema deve rendere trascurabile la probabilità che un documento alterato venga accettato come valido senza che l'incoerenza emerga in verifica.

Compromissione dello storage off-chain o del componente di gestione documentale

Descrizione

Questa minaccia riguarda il caso in cui l'avversario comprometta, manipoli o renda indisponibili i componenti incaricati della conservazione, pubblicazione, recupero o gestione operativa dei documenti. L'attaccante può essere sia interno al sistema che esterno.

Risorse e capacità tecniche

L'avversario può avere accesso diretto o indiretto allo storage documentale, a repository esterni, a servizi applicativi di caricamento e recupero o a componenti intermedi che mediano l'interazione tra utenti, autorità e documenti.

Le sue capacità possono includere:

- rendere temporaneamente o permanentemente indisponibile un documento;
- impedire il recupero di una determinata versione documentale;
- sostituire il contenuto recuperato con una copia alterata;
- presentare una versione obsoleta come se fosse quella corrente;
- eliminare o rendere irraggiungibili riferimenti necessari alla verifica;
- servire contenuti differenti a soggetti diversi, creando viste incoerenti dello stesso documento;
- generare disallineamento tra documento conservato, documento referenziato e documento effettivamente visualizzato.

Obiettivi attesi

Gli obiettivi principali dell'avversario sono:

- rendere indisponibili documenti;
- far recuperare una versione documentale precedente, incompleta o non più valida;
- far apparire come valido un documento non coerente con lo stato corrente;
- ostacolare la ricostruzione ex post del ciclo di vita documentale;
- ridurre la fiducia nel legame tra documento, metadati, riferimenti e stato certificato;
- generare incertezza operativa su quale sia la versione valida del documento.

Impatto

L'impatto è elevato perché:

- l'attacco può impedire a utenti legittimi di recuperare documenti necessari per attività operative o decisionali;
- è presente il rischio che un contenuto alterato, incompleto o non coerente venga presentato come se fosse valido;
- l'attaccante può tentare di far circolare versioni superate, documenti revocati, documenti non ancora certificati oppure copie non allineate con lo stato corrente;
- la compromissione dello storage o del componente di gestione documentale può rendere difficile ricostruire quale contenuto sia stato effettivamente mostrato, recuperato o utilizzato in una certa operazione.

Obiettivo di sicurezza

Il sistema deve garantire che un documento recuperato, visualizzato o utilizzato in una decisione sia coerente con lo stato certificato e corrente del sistema. La verifica non deve limitarsi alla disponibilità del contenuto, ma deve includere controlli su integrità, versione, stato di lifecycle, riferimenti, metadati rilevanti e validità temporale.

Inoltre, l'indisponibilità di un documento o di una sua versione non deve essere silenziosa: deve produrre un evento rilevabile e auditabile.

Proprietà di sicurezza da preservare

Le proprietà di sicurezza desiderate sono definite rispetto alle entità, agli asset e al modello avversariale descritti nelle sezioni precedenti.

- **Integrità documentale e integrità dello stato:** nessun attore non autorizzato deve poter alterare contenuto, metadati rilevanti, versioni, revoche o policy senza che la modifica risulti rilevabile. È, quindi, necessario mantenere la coerenza dello stato globale, cioè la capacità del sistema di dare a tutti gli attori autorizzati una rappresentazione non contraddittoria dei diritti e delle versioni documentali valide.
- **Autenticità delle operazioni:** deve essere possibile verificare che le operazioni rilevanti su documenti, policy, deleghe e revoche provengano da entità autorizzate e riconoscibili dal sistema.
- **Non ripudio:** ogni operazione rilevante deve essere attribuibile all'entità che l'ha prodotta o approvata, in modo che non possa essere negata arbitrariamente e possa essere ricostruita in fase di audit.
- **Autorizzazione per operazioni sui documenti:** Il sistema deve accettare un'operazione solo se il soggetto che la invoca possiede, al momento dell'uso, i diritti richiesti rispetto a documento, operazione, scopo, contesto, policy e stato corrente.
- **Correttezza della delega:** Un diritto dato in delega è valido solo se deriva da un diritto originario valido. Inoltre, non può essere più ampio, durare più a lungo o permettere più cose rispetto al diritto da cui nasce. Se il diritto viene delegato più volte, ogni passaggio deve poter essere controllato e deve essere coerente con i precedenti.
- **Correttezza della revoca e propagazione degli effetti:** la revoca di un diritto deve impedire che tale diritto continui a produrre effetti indebiti. Se il diritto revocato è sorgente di diritti delegati, la revoca deve essere considerata anche rispetto ai diritti derivati.
- **Coerenza del lifecycle documentale:** creazione, certificazione, aggiornamento, versioning, archiviazione e revoca dei documenti devono produrre uno stato coerente e ricostruibile nel tempo.
- **Accountability e auditabilità:** accessi, aggiornamenti, certificazioni, deleghe, revoche e decisioni rilevanti devono lasciare evidenze verificabili, resistenti a manomissione e interpretabili ex post dagli auditor autorizzati.
- **Disponibilità e tempestività operativa:** il sistema deve garantire la disponibilità dei documenti e la tempestività operativa di revoche, aggiornamenti, verifiche e audit come condizioni necessarie per evitare finestre di abuso o incoerenze operative.
- **Confidenzialità e minimizzazione:** soggetti non autorizzati non devono accedere al contenuto dei documenti né a metadati sensibili oltre quanto necessario. Nel dominio sanitario, la verifica di proprietà specifiche dovrebbe rivelare solo le informazioni necessarie allo scopo dichiarato.
- **Resilienza rispetto ad attori malevoli o compromessi:** le proprietà precedenti devono essere preservate, entro i limiti del modello di fiducia, anche in presenza di utenti compromessi, autorità malevole, abuso di deleghe, replay, tampering e interferenze operative.

WP2 – System Design

Inquadramento iniziale

Il sistema proposto è progettato come una **infrastruttura decentralizzata** per la governance del lifecycle di cartelle cliniche, referti e prescrizioni e per il controllo degli accessi in un contesto multi-organizzazione, in cui più autorità indipendenti devono cooperare senza poter riporre piena fiducia reciproca. La soluzione proposta di seguito evidenzia chiaramente dove risiedono la fiducia, la decisione autorizzativa, la prova crittografica e la responsabilità delle transizioni di stato. La **fiducia non viene delegata a una singola organizzazione**, ma viene costruita mediante tre ingredienti principali: **meccanismi crittografici**, **governance condivisa** e **stato auditabile**; il sistema deve essere progettato in modo tale che nessun singolo attore possa controllare unilateralmente le policy, il ciclo di vita dei documenti o le decisioni di accesso.

DLT Permissioned

Il sistema proposto viene costruito attorno a una **DLT permissioned**; questo tipo di blockchain è adatta a **use case enterprise** in cui i **partecipanti** sono **identificati**, le **transazioni** devono avere **bassa latenza**, **privacy** e **confidenzialità** sono requisiti importanti.

Come evidenziato nel threat model, il sistema non deve fronteggiare solo guasti accidentali o indisponibilità temporanee, ma anche attori malevoli interni al sistema stesso; per questo motivo, il livello di consenso della DLT permissioned viene progettato assumendo un modello **Byzantine Fault Tolerant**.

Dunque, la rete deve essere dimensionata per tollerare fino a f organizzazioni bizantine su un totale di almeno $n \geq 3f + 1$ peer partecipanti al consenso, richiedendo una soglia di *commit* pari ad almeno $2f + 1$ validazioni indipendenti per rendere definitiva una transazione.

Oltre al consenso della DLT, il sistema prevede **controlli applicativi di validazione sulle transazioni**. Questo significa che una transazione non viene accettata solo perché è stata inviata alla rete, ma deve **dimostrare di essere autorizzata e coerente** con le regole del sistema. Per questo motivo, ogni transazione deve contenere le informazioni necessarie alla verifica e, prima del commit, i peer verificano che il proponente sia autorizzato, che le firme siano valide, che gli approvatori siano organizzazioni distinte e abilitate, che il quorum applicativo richiesto sia stato raggiunto e che non vi siano voti duplicati. Inoltre, controllano che la **transazione sia basata sullo stato corrente** del sistema e che non utilizzi policy, credenziali, deleghe o DID revocati, scaduti o non più validi.

Infine, verificano che l'operazione richiesta sia compatibile con il ciclo di vita della risorsa coinvolta. In questo modo, il ledger non registra passivamente qualunque richiesta ricevuta, ma accetta solo transazioni verificabili, autorizzate, aggiornate e coerenti con la governance multi-authority.

Dunque, la soluzione è un modello identitario a due livelli in cui si separa la **fiducia infrastrutturale**, relativa alla partecipazione delle organizzazioni alla rete permissioned, dalla **verifica applicativa dei diritti**, basata su credenziali verificabili, issuer autorizzati, policy versionate e registri di revoca.

Architettura complessiva del sistema

L'architettura è organizzata su tre livelli logici distinti, ma coordinati.

Il primo livello è il **livello di governance e coordinamento**, costituito dal **ledger permissioned condiviso tra le organizzazioni**; questo livello ha il compito di mantenere lo stato condiviso e verificabile relativo a policy, deleghe, revoche, stati di lifecycle dei documenti ed eventi di audit. Qui non vengono conservati i documenti sensibili in chiaro, ma solo le **informazioni che devono essere condivise**. La DLT viene, quindi, utilizzata non come contenitore di tutti i dati, ma come registro condiviso e sincronizzato, adatto a costruire fiducia senza un'autorità centrale unica.

Il secondo livello è il **livello identitario e autorizzativo**, che integra i servizi di rilascio e verifica delle credenziali, i wallet degli utenti, i registri di revoca e i servizi di risoluzione dei DID o, più in generale, di chiavi e metadati necessari alla verifica. Questo livello rappresenta il punto in cui ruoli, attributi, qualifiche professionali, deleghe e consensi vengono trasformati in evidenze verificabili e spendibili nelle decisioni di accesso: le **Verifiable Credentials (VC)**.

Il terzo livello è il **livello di storage**, in cui vengono conservati off-chain i contenuti che non devono essere inseriti integralmente sul ledger, come documenti cifrati, policy complete, DID Document e batch di audit. Le Verifiable Credentials, invece, sono normalmente conservate dagli holder nei rispettivi wallet degli attori; il ledger mantiene solo informazioni di supporto alla verifica, come stato della credenziale, revoche e trust metadata. Nel design proposto, il ledger conserva hash, riferimenti, metadati di lifecycle e prove di stato, mentre i payload sensibili restano off-chain e, quando necessario, cifrati.

L'insieme dei tre livelli produce una separazione netta tra contenuto documentale, stato autorizzativo e stato di governance; questa separazione è fondamentale per evitare due errori frequenti:

- il primo è trattare la blockchain come deposito universale di dati sensibili;
- il secondo è lasciare policy, revoche e deleghe in database locali non verificabili dal resto del consorzio.

Si sottolinea che il wallet dei vari attori è protetto da autenticazione multi-fattore per l'accesso. Le chiavi private associate al wallet devono comunque essere conservate mediante meccanismi di protezione del wallet, in modo da ridurre il rischio di estrazione non autorizzata.

Livello di Governance e Coordinamento

Per la gestione di policy, revoche, deleghe e stati documentali viene utilizzata una **DLT permissioned**. Ogni organizzazione gestisce i **peer della rete permissioned** e tutte insieme prendono parte al processo di governance; quindi, ognuna mantiene il proprio perimetro amministrativo, ma contribuisce a **uno stato condiviso comune**. I peer mantengono una copia del ledger e dello stato condiviso, validano le transazioni secondo le regole del sistema e partecipano, direttamente o tramite componenti dedicate, al consenso e all'ordinamento delle transazioni.

Ogni policy descrive almeno i seguenti aspetti: il dominio documentale a cui si applica, i ruoli e attributi rilevanti, le azioni consentite, le finalità ammissibili e il tipo di quorum richiesto per la modifica della policy stessa. In questo modo la governance non è solo "chi può decidere", ma anche "quali aspetti del comportamento del sistema risultano normati in forma condivisa e verificabile".

Dal punto di vista operativo, la definizione o l'aggiornamento di una policy seguono il pattern:

proposal → governance voting → finalization → consensus & commit,

Ovvero:

- *proposal* → un attore autorizzato (ad esempio, un'organizzazione partecipante alla governance) propone una nuova policy o l'aggiornamento di una esistente;
- *governance voting* → le autorità partecipanti alla governance valutano la proposta e votano secondo le regole definite nelle policy di gestione, con un quorum richiesto differente rispetto al tipo di policy;
- *finalization* → se il quorum richiesto viene raggiunto, la policy è considerata approvata, l'esito viene quindi consolidato in una forma verificabile e preparato per essere registrato nel ledger (si passa da una proposta approvata a una decisione formalizzata);
- *consensus & commit* → la transazione che rappresenta la policy approvata viene sottoposta al protocollo di consenso della DLT, i peer della rete verificano la validità tecnica della transazione, la ordinano e la inseriscono nel ledger. Lo stato viene, quindi, aggiornato e la policy diventa parte dello stato condiviso e replicato su tutti i nodi.

Tipi di policy

In particolare, nel nostro sistema sono stati individuati tre tipi di policy:

- **Policy di gestione della governance:** definiscono le regole necessarie al funzionamento della governance multi-authority, stabilendo, ad esempio, quali organizzazioni possono partecipare ai processi decisionali, chi è autorizzato a proporre o votare modifiche alle policy di accesso, quali soglie di approvazione sono richieste e in quali condizioni un'organizzazione può essere sospesa o rimossa dal sistema. Le policy di gestione possono inoltre definire procedure di sospensione, revisione o rimozione di attori che violano ripetutamente le regole del sistema. Le soglie e le conseguenze non sono hard-coded nel design, ma configurate tramite policy di governance approvate dalle autorità. Tali policy sono necessaria a ridurre la probabilità che comportamenti illeciti si verifichino nel sistema.
- **Policy di accesso globali:** sono le policy necessarie a definire le regole di accesso ai documenti, che saranno poi comuni a tutte le organizzazioni. Non attribuiscono accessi illimitati, ma fissano il perimetro massimo entro cui gli utenti operativi possono agire; eventuali policy individuali o di consenso granulare possono solo restringere tali permessi, non ampliarli.
- **Policy di accesso patient-driven:** sono quelle più specifiche che i pazienti possono definire per restringere i permessi concessi dalle policy di accesso globali definite dalle organizzazioni; questo tipo di policy più restrittive, definite dai pazienti stessi, sono necessarie essendo che nei documenti sono gestiti dati sensibili del paziente stesso.

Il quorum di governance

Il quorum da raggiungere varia in base al tipo di policy da inserire nel ledger; ciò porta a una necessaria distinzione tra i quorum da raggiungere nei diversi casi, per bilanciare sicurezza e usabilità. Infatti, se usassimo, ad esempio, la supermajority per entrambi i casi, le operazioni quotidiane inerenti alla creazione e alla modifica delle policy diventerebbero gravose.

Dunque, il **quorum** è stato definito come segue:

- Per l'aggiunta o la modifica delle **policy di gestione della governance del sistema**, si utilizza la **supermajority**, per evitare che una maggioranza troppo piccola vada a prendere decisioni rilevanti → $q = \text{ceil}\left(\frac{2n}{3}\right)$, con n numero di organizzazioni.
- Per la creazione e la modifica delle policy di accesso globale, viene usato il quorum di **maggioranza semplice** (50%+1) → $q = \text{floor}\left(\frac{n}{2}\right) + 1$, con n numero di organizzazioni.

- **Per le policy di accesso patient-driven** non è richiesto un quorum, perché operano solo in senso restrittivo sui documenti del paziente; tuttavia, la loro registrazione è accettata dal sistema solo se la transazione è firmata dal paziente titolare, se la policy non estende i permessi definiti dalle policy globali e se rispetta i vincoli minimi stabiliti dalla governance.

Verifica delle policy: RBAC E ABAC

Essendo che il sistema dovrà definire in base a quali attributi, ruoli, certificazioni o deleghe un soggetto può accedere a un documento o compiere una certa azione, il modello di controllo degli accessi adottato è di tipo ibrido tra **Role-Based Access Control (RBAC)** e **Attribute-Based Access Control (ABAC)**.

Nel design proposto saranno presenti sia **ruoli strutturali** che **attributi contestuali**; quindi, per stabilire se un soggetto sia autorizzato a compiere una determinata azione, il sistema confronterà la richiesta con le policy di accesso, verificando contemporaneamente:

- il **ruolo** attestato dalla Verifiable Credential del soggetto, ad esempio, medico specialista, auditor o verifier;
- gli **attributi contestuali** associati al profilo operativo del soggetto, quali specializzazione professionale, reparto di appartenenza, ruolo funzionale ricoperto o altri vincoli previsti dalla policy.

Tali informazioni saranno incluse nella **Verifiable Credential** associata all'utente, così da poter essere presentate e verificate durante il processo di autorizzazione. L'autorizzazione viene, quindi, valutata non semplicemente come *“chi sei”*, ma come *“chi sei, con quali titoli, su quale documento, in quale stato, per quale finalità e in quale momento”*.

Policy Engine

Il Policy Engine è il componente logico incaricato di valutare le richieste di accesso, modifica, creazione, delega, revoca. Esso non conserva direttamente le policy, le credenziali o i documenti, ma recupera e valida gli elementi necessari alla decisione interrogando i registri condivisi e i servizi del livello identitario e autorizzativo.

Il Policy Engine non è un'autorità centrale unica: è un componente logico che viene istanziato presso ciascuna organizzazione; le diverse istanze hanno il compito di valutare le richieste rispetto allo stesso stato condiviso registrato nei registry, così da produrre decisioni coerenti e verificabili. Nessuna istanza locale del Policy Engine può modificare unilateralmente policy, deleghe, revoche o stati documentali.

Il suo ruolo è trasformare una richiesta operativa in una decisione autorizzativa verificabile. Una richiesta viene valutata considerando congiuntamente l'identità del richiedente, gli attributi attestati nelle Verifiable Credentials, lo stato corrente delle credenziali, la legittimità dell'issuer, le policy globali applicabili, le eventuali policy patient-driven, le deleghe attive, lo stato di revoca, il lifecycle del documento, la finalità dichiarata e il momento in cui l'operazione viene richiesta.

Il Policy Engine rappresenta, quindi, il punto in cui il modello RBAC/ABAC viene effettivamente applicato; infatti, la componente RBAC consente di valutare il ruolo del soggetto, ad esempio medico, infermiere, auditor o requester esterno, mentre la componente ABAC consente di valutare attributi più granulari e contestuali, come specializzazione, reparto, organizzazione di appartenenza, relazione con il paziente, finalità dell'accesso, intervallo temporale, stato del documento e presenza di una delega valida.

La decisione del Policy Engine non deriva automaticamente dal possesso di una Verifiable Credential, dato che una credenziale valida fornisce soltanto attributi attendibili; l'autorizzazione viene concessa solo se tali attributi soddisfano le policy applicabili e se non esistono restrizioni, revoche o condizioni ostative. In particolare, una richiesta è autorizzabile solo se la policy globale consente l'operazione, se eventuali policy patient-driven non la vietano o la restringono, se la delega eventualmente presentata è valida e non revocata, e se il documento si trova in uno stato compatibile con l'operazione richiesta.

Il Policy Engine produce come output una decisione strutturata, *allow*, *deny*, accompagnata dai riferimenti alle policy, alle versioni documentali, alle credenziali, alle deleghe e agli stati di revoca utilizzati nella valutazione. Tale decisione viene registrata nell'Audit Registry, così da rendere ricostruibile ex post il motivo per cui una richiesta è stata autorizzata o rifiutata.

Il Policy Engine non deve avere accesso indiscriminato al contenuto in chiaro dei documenti; la sua funzione è valutare la legittimità dell'operazione sulla base di attributi, metadati, stati e prove verificabili.

Registri logici del sistema

La DLT permissioned funge da registro autorevole dello stato condiviso e da livello di coordinamento tra le organizzazioni. Essa garantisce ordinamento, tracciabilità e verificabilità delle transazioni senza memorizzare dati sanitari sensibili in chiaro. Il ledger conserva quindi riferimenti, hash, stati, metadati minimizzati e ancoraggi verificabili agli eventi rilevanti; i contenuti completi, come documenti, policy estese o batch di audit, restano off-chain e, quando necessario, cifrati.

I registri logici mantenuti sul ledger sono:

1. **Global Policy Registry**, per le policy di governance e le policy di accesso globali;
2. **Patient-Driven Policy Registry**, per consensi, restrizioni e policy individuali definite dai pazienti;
3. **Delegation Registry**, per deleghe operative, revoche e catene di deleghe;
4. **Document Lifecycle Registry**, per hash, riferimenti, versioni e stati dei documenti;
5. **Audit Registry**, per l'ancoraggio verificabile degli eventi rilevanti del sistema;
6. **Identity Registry**, per la risoluzione dei DID e la verifica dell'integrità dei DID Document;
7. **Credential Status Registry**, per lo stato corrente delle Verifiable Credentials, incluse revoche e sospensioni;
8. **Trust Registry**, per indicare quali issuer, verifier o autorità siano riconosciuti dalla governance e per quali tipologie di credenziali o operazioni.

I primi cinque registri supportano direttamente governance, lifecycle documentale, deleghe e audit; gli ultimi tre appartengono funzionalmente al livello identitario e autorizzativo, ma sono comunque ancorati al ledger perché il loro stato deve essere condiviso, verificabile e non manipolabile unilateralmente.

Policy Registries

I **Policy Registries** sono i registri incaricati di conservare, versionare e rendere verificabili le policy del sistema. Non contengono solo le policy attualmente attive, ma mantengono la storia completa delle policy approvate, aggiornate, sostituite o revocate nel tempo. Questa scelta è fondamentale perché ogni decisione autorizzativa rilevante deve poter essere ricondotta non solo al soggetto o all'insieme di

soggetti che l'hanno presa, ma anche alla **specifica versione della policy vigente** nel momento in cui la decisione è stata eseguita.

A tale scopo, le policy non vengono sovrascritte eliminando le versioni precedenti, bensì ogni aggiornamento produce una nuova versione, collegata alla precedente tramite un riferimento esplicito. In questo modo, il sistema mantiene una storia append-only delle regole di governance e di accesso.

Per ogni policy, il registro conserva almeno un identificativo univoco, il tipo di policy, il numero di versione, lo stato della policy, il periodo di validità, il riferimento alla versione precedente e le informazioni relative alla sua approvazione. Il contenuto completo della policy viene mantenuto off-chain, in uno storage distribuito, mentre on-chain viene registrato l'hash crittografico del contenuto, insieme a un riferimento alla sua posizione. In questo modo si evita di appesantire inutilmente la blockchain, ma si conserva comunque la possibilità di verificare che il testo della policy non sia stato alterato.

I Policy Registries supportano anche la tracciabilità del processo decisionale. Ogni nuova policy o aggiornamento deve essere associato alle autorità che l'hanno approvato e alla soglia di governance raggiunta. Questo permette di verificare che nessuna organizzazione abbia modificato unilateralmente regole critiche del sistema.

Differenze tra le policy

Nel caso delle policy patient-driven, il contenuto completo della scelta del paziente, ad esempio il consenso, la restrizione o la revoca, non viene pubblicato in chiaro sul ledger, poiché potrebbe rivelare informazioni sensibili sul paziente, sul documento o sul rapporto con un determinato medico. Sul ledger viene invece registrata una rappresentazione minimizzata e verificabile della policy, contenente hash, stato, periodo di validità, riferimento pseudonimo al soggetto autorizzato, ambito della risorsa, finalità e riferimento alla policy globale applicabile.

Sempre per le policy patient-driven, invece, il processo decisionale non richiede un quorum tra organizzazioni, poiché tali policy derivano dalla volontà del soggetto del documento. La loro validità dipende dalla firma del paziente, dalla corretta identificazione del soggetto tramite credenziale verificabile e dalla compatibilità con le policy globali attive. Il paziente può restringere il perimetro di accesso previsto dalle policy globali, ma non può ampliarlo.

Due Registri Logici

Sulla base di quanto descritto in precedenza, nel sistema vengono definiti due registri logici distinti:

1. **Global Policy Registry**, incaricato di conservare le **policy di gestione della governance** e le **policy di accesso globale** definite dalle organizzazioni indipendenti. Questo registro contiene le **regole generali del sistema** come i **quorum di approvazione**, i ruoli ammessi, gli attributi richiesti, le finalità consentite, le condizioni di accesso ai diversi domini documentali e le regole generali di delegabilità.
2. **Patient-Driven Policy Registry**, incaricato di conservare le **policy individuali definite dal paziente** quali consensi, restrizioni, revoche e vincoli specifici di accesso ai propri documenti. Questo registro non contiene policy globali valide per tutti gli utenti, ma regole individuali riferite a uno specifico soggetto del documento, a una classe documentale o a uno specifico documento. Tali policy possono autorizzare un determinato medico o ente ad accedere a una risorsa per una finalità e un intervallo temporale definiti, oppure possono **imporre restrizioni aggiuntive**, come la limitazione dell'accesso a una sola categoria di documenti. Le policy patient-driven sono quindi subordinate alle policy globali: una policy del paziente è **valida solo se non concede diritti più ampi** di quelli ammessi dal Global Policy Registry. Ad esempio, se la

policy globale stabilisce che solo un medico cardiologo possa accedere a referti cardiaci, il paziente può autorizzare uno specifico cardiologo per un tempo limitato, ma non può autorizzare un soggetto privo degli attributi richiesti dalla policy globale.

Delegation Registry

Il **Delegation Registry** è il registro logico incaricato di conservare e rendere verificabili le deleghe operative e il relativo stato di validità. A differenza dei **Policy Registries**, che contengono regole generali e versionate, questo registro conserva i fatti concreti relativi alle deleghe: chi ha delegato chi, su quale documento o classe documentale, per quale finalità, entro quali limiti temporali e con quale stato corrente. La sua funzione principale è permettere al sistema di stabilire, nel momento in cui un soggetto richiede un'operazione, se il privilegio che intende esercitare sia effettivamente valido, spendibile e coerente con le policy applicabili.

Una delega non viene considerata come un permesso permanente, ma come un'autorizzazione derivata, limitata e revocabile. Per questo motivo il registro conserva non solo le deleghe attive, ma anche quelle scadute, revocate o sostituite. Anche in questo caso, i record non vengono eliminati dalla storia del sistema: una delega revocata non scompare, ma viene marcata come non più valida.

In questo modo, il sistema può distinguere tra una delega mai esistita, una delega esistita ma ormai scaduta, una delega ancora attiva e una delega esplicitamente revocata. Questa scelta è essenziale per l'audit, perché consente di ricostruire se, in un certo momento, una specifica operazione fosse coperta da una delega valida oppure se sia stata effettuata dopo la scadenza o dopo la revoca.

Document Lifecycle Registry

Il **Document Lifecycle Registry** è il registro logico incaricato di conservare lo stato verificabile dei documenti lungo tutto il loro ciclo di vita. A differenza dello storage documentale, che contiene il documento vero e proprio, questo registro non memorizza dati sensibili in chiaro, ma conserva metadati, hash, riferimenti e stati necessari a dimostrare l'esistenza, l'integrità, la versione e la validità operativa di ciascun documento.

La funzione principale del Document Lifecycle Registry è associare a ogni documento una sequenza ordinata di versioni. Il documento, infatti, non viene considerato come un oggetto statico: può essere creato, aggiornato, sostituito da una nuova versione, archiviato o revocato. Ogni passaggio rilevante produce un evento registrato, in modo che la storia del documento sia ricostruibile e verificabile nel tempo.

Il registro mantiene anche la **successione delle versioni**; infatti, quando un documento viene aggiornato, la versione precedente non viene cancellata, bensì viene aggiunta una nuova versione, collegata alla precedente. In questo modo è possibile ricostruire l'intera evoluzione del documento.

Questa struttura consente di distinguere lo stato corrente del documento dalla sua **storia completa**, in quanto la versione corrente è quella utilizzabile per le operazioni ordinarie, mentre le versioni precedenti restano disponibili per audit, verifica storica, tracciabilità e ricostruzione delle responsabilità.

Il pattern di base riprende la **notarizzazione** documentale: registrare un hash, associare un owner e un timestamp, verificare l'esistenza del documento ed emettere eventi per registrazione, aggiornamento e rimozione.

Per ragioni di privacy, il Document Lifecycle Registry deve minimizzare i dati memorizzati on-chain, per cui non deve contenere il contenuto dei documenti sensibili o dati personali non necessari, ma solo identificativi pseudonimi, hash, riferimenti cifrati, stati e metadati minimi. Il contenuto documentale

resta off-chain e viene protetto tramite cifratura, controllo d'accesso e gestione sicura delle chiavi. È importante evidenziare che l'utilizzo di identificativi pseudonimi non elimina completamente il rischio di correlazione tra documenti riconducibili allo stesso soggetto; questo viene però accettato come trade-off per ragioni di scalabilità.

Audit Registry

L'**Audit Registry** è il registro logico che ancora sul ledger i riferimenti verificabili agli eventi rilevanti del sistema. Per ragioni di riservatezza ed efficienza, il ledger non conserva il contenuto completo degli audit log, ma solo metadati minimizzati, identificativi di batch, hash, CID e riferimenti alle policy, ai documenti o alle deleghe coinvolte. Il contenuto completo dei batch di audit viene conservato off-chain in forma cifrata. La sua funzione non è semplicemente quella di "registrare tutto", ma di costruire una traccia storica affidabile delle operazioni significative, così da supportare accountability, ricostruzione degli eventi, attribuzione delle responsabilità e verifica indipendente da parte di auditor o autorità autorizzate.

A differenza degli altri registri, che conservano stati specifici del sistema, l'Audit Registry conserva la **storia delle azioni**. Il Policy Registry mantiene le versioni delle policy, il Document Lifecycle Registry mantiene lo stato dei documenti, il Delegation and Revocation Registry mantiene deleghe e revoche; l'Audit Registry, invece, registra il fatto che una certa azione sia avvenuta, quando sia avvenuta, da quale soggetto sia stata richiesta, da quale componente sia stata eseguita, con quale esito e rispetto a quale contesto autorizzativo.

L'Audit Registry è fondamentale anche per distinguere tra operazioni autorizzate, operazioni rifiutate e tentativi sospetti. Registrare solo le operazioni riuscite non sarebbe sufficiente, perché un pattern di accessi negati può rivelare tentativi di abuso, privilege escalation o probing del sistema. Per questo il registro include anche eventi di fallimento, come richieste respinte per credenziale scaduta, delega revocata, policy non soddisfatta.

Il registro supporta, quindi, il principio di **accountability**, per cui ogni azione rilevante deve poter essere attribuita a un soggetto, a un ruolo o a un'autorità. Questo non significa necessariamente rendere pubblica l'identità reale di tutti gli attori, ma garantire che, secondo le regole del sistema, sia possibile risalire al responsabile dell'azione quando un auditor autorizzato deve svolgere una verifica.

Informazioni registrate nei ledger

Policy Registries

Le policy non sono considerate regole statiche e immutabili, ma elementi governati lungo tutto il loro ciclo di vita: proposta, validazione, aggiornamento, revoca e applicazione. Le istanze attive delle diverse tipologie di policy sono quindi rappresentate come oggetti di stato registrati e versionati nel ledger, così da garantire tracciabilità, coerenza e verificabilità nel tempo.

Il ledger conserva una rappresentazione verificabile e minimizzata dei consensi, costituita da DID, stato, periodo di validità e puntatori al contenuto off-chain. Il contenuto integrale del consenso o della policy rimane off-chain e cifrato.

Policy

Di seguito vengono definiti i campi comuni a tutti e tre i tipi di policy specificati precedentemente e quelli specifici di ognuna.

Campi presenti in tutte le policy

- *PolicyID* → Identificativo univoco della policy;
- *PolicyType* → Tipo di policy: governance, accesso globale, patient-driven;

- *PolicyVersion* → Numero versione della policy;
- *PolicyCID* → CID necessario per la richiesta del documento integrale della policy al meccanismo di storage utilizzato;
- *CreationTimestamp* → Data e ora di creazione;
- *ProposedByDID* → DID del soggetto autorizzato che ha proposto la policy;
- *PreviousPolicyVersion* → ID versione precedente, essendo le policy oggetti di stato versionati;
- *RevocationReason* → Motivazione (se presente, allora la policy è stata revocata);
- *RevocationTimestamp* → Data e ora della revoca.

Campi delle policy di gestione della governance

Sono le policy che regolano **chi può partecipare alla governance**, chi può votare, quali quorum sono richiesti e quando un'organizzazione può essere sospesa o rimossa.

- *ApprovedByDIDs* → Organizzazioni che hanno approvato la policy;
- *RejectedByDIDs* → Organizzazioni che hanno votato contro;
- *GovernanceDomain* → ambito: ammissione, sospensione, modifica quorum, rimozione organizzazione.

Campi delle policy di accesso globali

Sono le policy che definiscono il **perimetro massimo** dei permessi concessi agli utenti operativi.

- *Role* → identifica il ruolo funzionale o professionale del soggetto a cui la policy si applica (ad esempio medico, infermiere, tecnico di laboratorio, auditor, organizzazione, ecc...) e rappresenta una base autorizzativa di alto livello;
- *Attributes* → l'insieme degli attributi verificabili associati al soggetto che consentono di applicare controlli più granulari rispetto al solo ruolo, ad esempio specializzazione (cardiologo, psichiatra), reparto di appartenenza, paziente in carico o altre condizioni contestuali;
- *DocumentCategory* → Categoria documentale: referto, prescrizione, ecc... ;
- *AllowedActions* → Azioni consentite: read, create, update, certify, archive;
- *PurposeOfUse* → Finalità consentita: cura, diagnosi, continuità assistenziale, ecc... ;
- *ApprovedByDIDs* → Organizzazioni che hanno approvato la policy;
- *RejectedByDIDs* → Organizzazioni che hanno votato contro.

Campi delle policy di accesso patient-driven

Sono le policy definite dal paziente per **restringere** i permessi globali; non possono mai ampliare i diritti concessi dalle policy globali.

- *PatientDID* → DID del paziente che impone la restrizione;
- *ValidFrom* → data e ora di inizio validità della policy;
- *ValidUntil* → data e ora di fine validità della policy.

Ordine di verifica delle policy

Per gestire eventuali conflitti nella verifica dei permessi, le policy verranno valutate secondo un ordine gerarchico di priorità. In primo luogo, saranno analizzate le policy patient-driven, al fine di verificare la presenza di **restrizioni o consensi specifici** espressi dal paziente rispetto a una determinata operazione. In assenza di una policy patient-driven applicabile, verranno considerati eventuali record di delega validi, qualora l'utente agisca sulla base di una **delega temporanea** conferita da un soggetto autorizzato. Solo in ultima istanza saranno applicate le policy di accesso globale, che definiscono **regole generali valide per il sistema** o per specifiche categorie di utenti e risorse.

Document lifecycle registry

Sul Document lifecycle registry vengono salvati due tipi di informazioni relative ai documenti:

- **Versioning e Lifecycle Documentale:** Quando un documento viene aggiornato (es. l'aggiunta di una nuova visita da parte del cardiologo delegato), il dato precedente non viene sovrascritto per garantire auditabilità e conservazione della cronologia, bensì il sistema genera un nuovo documento cifrato e sul ledger viene registrata una transazione che definisce un legame gerarchico (puntatore di versione) tra il nuovo documento e la versione precedente.
- **Identificatore del contenuto off-chain (CID):** Per consentire il recupero del documento integrale dallo storage off-chain, sul ledger viene memorizzato un identificatore del contenuto (il CID, come descritto successivamente nel livello di storage) che consente di richiedere il documento al meccanismo di storage usato per i documenti sensibili. Il CID viene, quindi, associato alla specifica versione documentale, all'hash del payload cifrato, al DID del proprietario e agli ulteriori metadati necessari alla verifica e alla governance associando. Quando il contenuto del documento cambia, automaticamente cambierà anche l'identificatore.

Record documentali

Il record documentale inserito nella blockchain ha lo scopo di tenere traccia sia del cambiamento di stato del documento nel tempo (ad esempio, un update che ne genera una nuova versione), ma anche di consentire a coloro che lo consultano di verificarne l'integrità. Ogni record documentale sarà composto da:

- **Campi identificativi**, necessari a identificare univocamente il documento:
 - *DocumentID* → identificativo del documento. Identifica il documento logico nel tempo, indipendentemente dalle sue versioni; quindi, quando il documento viene aggiornato, l'ID resta lo stesso;
 - *PatientDID* → DID del paziente a cui fa riferimento il documento;
 - *DocumentType* → tipo di documento, ad esempio referto, prescrizione;
 - *CreatorDID* → DID dell'utente che ha creato il documento;
 - *CID* → identificatore del contenuto off-chain, necessario alla richiesta del documento integrale al meccanismo di storage.
- **Campi dello stato del lifecycle:**
 - *DocumentState* → lo stato in cui si trova il documento, tra cui *Certified*, *Archived* o *Revoked*;
 - *VersionNumber* → il numero della versione del documento;
 - *PreviousVersionPointer* → il numero della versione precedente per mantenere la catena del versioning;
 - *CreationTimestamp* → timestamp di creazione del documento.

Lifecycle dei documenti

Nel design proposto, il documento non è trattato come un file statico presente o assente, ma come una **entità dinamica e versionata** che attraversa stati successivi. Ne consegue che il sistema deve sapere quale versione sia corrente, quali versioni precedenti esistano ancora come evidenza storica e quale stato operativo sia associato a ciascuna di esse.

Ogni documento viene gestito tramite due componenti logicamente distinte: la prima è il **payload documentale**, cioè il contenuto effettivo del documento, conservato off-chain in forma cifrata; la seconda è il **record di lifecycle**, conservato sul ledger, che contiene metadati, riferimento allo storage, stato operativo, legame con eventuali versioni precedenti e attestazioni di certificazione.

La sequenza degli stati di lifecycle definita comprende:

- **Certified** → è lo stato dedicato alla versione del documento attualmente in uso, indica che una nuova versione documentale è stata generata, cifrata, associata al relativo *CID* e registrata sul ledger. Lo stato certifica che il documento ha superato i controlli previsti sull'identità del soggetto emittente, sulle autorizzazioni e sulle policy di governance, risultando quindi valido e utilizzabile all'interno del sistema.
- **Archived** → indica che la versione resta disponibile come evidenza storica, ma non è più parte del flusso operativo ordinario, ovvero è presente una nuova versione del documento.
- **Revoked** → indica che il documento o la sua certificazione non possono più essere fatti essere considerate valide per operazioni ordinarie.

Inoltre, un **evento** significativo da analizzare è quello di **New Version** del documento; infatti, quando un documento viene aggiornato, la modifica che porta alla nuova versione non avviene mai come modifica *in-place* invisibile. Quindi, ogni update genera una nuova versione del documento, con un nuovo *CID*, un nuovo *timestamp*, un riferimento esplicito alla versione precedente e un nuovo record di stato, e la versione precedente non viene distrutta, ma resta visibile ai fini di audit e ricostruzione.

Delegation Registry

Nel Delegation Registry vengono salvate le seguenti informazioni:

- **Record di delega**: i record di delega attestano il conferimento temporaneo di un permesso da parte di un soggetto autorizzato a favore di un altro utente operativo.
- **Record di revoca**: così come un permesso può essere delegato, l'utente o l'organizzazione potrebbero anche voler revocare permessi ad un altro utente; i record di revoca attestano la cessazione anticipata o programmata di una delega o di un permesso precedentemente concesso. Anche la revoca viene registrata nel ledger come evento verificabile, in modo da rendere evidente il momento a partire dal quale un determinato permesso non è più valido.

Funzionamento delle deleghe e delle revoche

Per i documenti di delega è sempre il **proprietario** dei dati **a dare il consenso** a seguito di una **proposta di delega** formulata da un utente operativo a cui era stato dato l'accesso in maniera conforme alle policy di accesso globali, alle policy patient-driven o ad un eventuale delega ricevuta.

Campi del record di delega

- *DelegationID* → identificativo della delega;
- *DelegatorDID* → DID del soggetto che delega;
- *DelegateDID* → DID del soggetto delegato;
- *IssuerOrganizationDID* → DID dell'organizzazione che ha prodotto il documento target;
- *TargetDocumentID* → documento oggetto della delega (corrisponde all'identificativo del documento logico all'interno dei record documentali);
- *DelegatedActions* → azioni delegate: read, update, revoke;
- *PurposeOfDelegation* → finalità della delega: consulenza, secondo parere, continuità assistenziale;
- *ValidFrom* → inizio validità;
- *ValidUntil* → fine validità;
- *ParentDelegationRef* → eventuale delega precedente nella catena;
- *Depth* → profondità della delega attuale nella catena;
- *MaxDelegationDepth* → profondità massima della catena di deleghe.

Campi del record di revoca

- *RevocationId* → identificativo della revoca;
- *DelegationId* → identificativo della delega a cui la revoca si riferisce. Serve a collegare il record di revoca alla delega precedentemente concessa;
- *RevocationType* → indica la tipologia di revoca, ad esempio volontaria, automatica per scadenza, o conseguente a violazione della policy;
- *RevokedAt* → data e ora in cui la revoca è stata registrata o è diventata efficace, permette di stabilire da quale momento il permesso non può più essere utilizzato;
- *RevokedBy* → identificativo del soggetto che ha richiesto o disposto la revoca;
- *RevocationReason* → motivazione della revoca, utile per finalità di audit, trasparenza e ricostruzione del processo decisionale;
- *Signature* → firma del soggetto che ha disposto la revoca, necessaria per garantire autenticità, integrità e non ripudio dell'operazione.

Audit registry

Ogni interazione significativa con il sistema documentale viene storicizzata, dunque l'audit registry è il ledger in cui si registrano eventi immutabili strutturati secondo il paradigma "*Chi ha fatto cosa, a chi, quando e con quale autorizzazione*". Questo include le richieste di accesso, le approvazioni delle deleghe, le revoche e, soprattutto. Per preservare la riservatezza ed evitare di appesantire il registro, il ledger non conserva l'audit per intero, che invece verrà memorizzato all'interno di storage distribuiti. Inoltre, il record di audit memorizzato sul registro fa riferimento non ad un singolo evento, ma ad un batch di eventi tra loro correlati e avvenuti in sequenza.

Gli eventi di audit sono collegati anche alle versioni delle policy e dei documenti vigenti al momento dell'operazione, così da consentire agli auditor di ricostruire ex post la correttezza dell'accesso, dell'aggiornamento, della delega o della revoca senza alterare lo stato del sistema.

Campi del record di audit

Ogni record registrato nell'Audit Registry non rappresenta un singolo evento, ma un batch di eventi di audit e il record contiene il *batchId*, ovvero l'identificativo univoco del batch di eventi di audit, e il *CID*, ovvero l'identificatore del contenuto necessario a recuperare tutte le altre informazioni che verranno salvate negli storage. Inoltre, viene inserito anche il *PreviousBatchHash*, ovvero l'hash del batch precedente, ai fini della concatenazione.

L'Identity Registry

È un registro logico ancorato al ledger che supporta la risoluzione e la verifica dei Decentralized Identifiers utilizzati dagli attori del sistema. Per ciascun DID, il registro conserva informazioni minimizzate, come lo stato dell'identificatore, il riferimento CID al DID Document conservato off-chain, l'hash del documento e i dati necessari alla verifica della sua integrità. In questo modo, ogni organizzazione può verificare che un DID sia attivo e che il DID Document recuperato non sia stato alterato. La descrizione approfondita di questo registro è riportata nel livello apposito.

Credential Status Registry

Nel sistema proposto, lo stato delle Verifiable Credentials viene gestito tramite un Credential Status Registry condiviso tra le organizzazioni appartenenti alla rete permissioned. Tale registro consente ai verifier di controllare se una credenziale precedentemente emessa sia ancora accettabile dal sistema, oppure se sia stata revocata o sospesa.

Trust Registry

Il Trust Registry è il componente incaricato di definire e rendere verificabile il perimetro di fiducia della rete. La sua funzione non è attestare lo stato di una singola credenziale, compito demandato al Credential Status Registry, ma stabilire quali soggetti siano riconosciuti dalla governance come issuer, verifier o autorità abilitate, e per quali tipologie di credenziali o operazioni.

Nel sistema proposto, il fatto che un'organizzazione sanitaria appartenga alla rete permissioned non implica automaticamente che essa sia autorizzata a emettere qualsiasi tipo di VC. Un ospedale, ad esempio, potrebbe essere autorizzato a emettere credenziali relative ai propri dipendenti, mentre un ente professionale potrebbe essere autorizzato ad attestare iscrizioni ad albi o qualifiche professionali. Il Trust Registry consente, quindi, di associare a ciascun issuer un insieme esplicito di *credential type* che è autorizzato a emettere.

Il Trust Registry viene consultato durante la verifica di una VC. Dopo aver controllato la firma dell'issuer, il *DID Document*, la validità temporale e lo stato della credenziale, il *verifier* verifica anche che l'issuer sia presente nel Trust Registry e che sia autorizzato a emettere quella specifica tipologia di credenziale. In assenza di tale autorizzazione, la credenziale non deve essere accettata come fonte valida di attributi, anche se la firma crittografica risulta tecnicamente corretta.

Livello identitario e autorizzativo

Il livello identitario e autorizzativo comprende l'insieme dei meccanismi necessari a identificare crittograficamente gli attori del sistema, verificare le attestazioni relative a ruoli e attributi, e determinare se una determinata azione su documenti, credenziali o risorse di governance debba essere autorizzata.

Nel sistema proposto vengono utilizzati **Decentralized Identifiers (DID)** e **Verifiable Credentials** per separare in modo chiaro l'identificazione tecnica degli attori dalla verifica dei loro attributi e dalla decisione autorizzativa. I *DID* non rappresentano, di per sé, un ruolo, una qualifica professionale o un permesso di accesso, ma costituiscono identificatori crittograficamente verificabili associati a chiavi pubbliche e metodi di verifica. Attraverso il relativo *DID Document*, un soggetto può dimostrare il controllo del proprio identificatore, firmare richieste, presentazioni, consensi o credenziali, e consentire ai verifier di controllare l'autenticità delle operazioni eseguite.

Le **Verifiable Credentials**, invece, sono utilizzate per attestare attributi e informazioni rilevanti sugli attori del sistema, come il ruolo professionale, l'organizzazione di appartenenza, la specializzazione, eventuali deleghe o ulteriori qualifiche. Tali credenziali sono emesse da organizzazioni riconosciute e firmate crittograficamente dagli issuer. La loro verifica richiede sia un controllo tecnico della firma, tramite la risoluzione del *DID* dell'issuer e il recupero del relativo *DID Document*, sia un controllo di governance, volto ad accertare che l'issuer sia autorizzato a emettere quello specifico tipo di credenziale.

La decisione finale di autorizzazione non deriva automaticamente dal possesso di un *DID* o di una *Verifiable Credential*. Essa viene invece determinata dal motore di policy, che valuta gli attributi attestati, lo stato di validità e revoca delle credenziali, la legittimità dell'issuer, il contesto della richiesta, la finalità dichiarata, il tipo di documento coinvolto.

L'architettura distingue quindi diversi livelli funzionali.

1. L'**Identity Registry** consente la risoluzione dei *DID* e la verifica dell'integrità dei relativi *DID Document*;
2. Il **DID Document** contiene le chiavi pubbliche e i metodi di verifica associati a un *DID*;
3. Le **Verifiable Credentials** attestano ruoli, attributi;
4. Le **Verifiable Presentations** permettono agli holder di presentare le credenziali in modo verificabile e contestuale;
5. Il **Credential Status Registry** consente di verificare lo stato corrente delle credenziali, ad esempio attive, revocate o sospese;
6. Il **Trust Registry** definisce quali issuer sono riconosciuti e per quali tipi di credenziali;
7. Il **Document Lifecycle Registry** mantiene i riferimenti ai documenti clinici conservati off-chain, includendo *documentId*, *CID*, *hash*, *versione*, *metadati minimizzati* e *DID del paziente associato*;
8. Il **Policy Engine** applica le regole di accesso e governance.

Identity Registry

L'**Identity Registry** è il registro logico incaricato di rendere risolvibili e verificabili i Decentralized Identifiers utilizzati all'interno del sistema. La sua funzione principale non è attestare ruoli, qualifiche, permessi o autorizzazioni, ma fornire le informazioni tecniche necessarie per verificare il controllo crittografico di un *DID* e recuperare in modo affidabile il relativo *DID Document*.

In particolare, l'Identity Registry consente ai verifier e agli altri componenti del sistema di stabilire se un determinato *DID* esiste, se è ancora attivo, dove si trova il *DID Document* associato e se il documento recuperato dallo storage off-chain corrisponde effettivamente alla versione registrata. In questo modo, il registry agisce come punto di ancoraggio verificabile per l'identità tecnica degli attori, senza trasformarsi in un registro anagrafico o autorizzativo.

Il registro mantiene quindi, per ciascun *DID*, un insieme minimo di informazioni tecniche necessarie alla risoluzione, alla verifica di integrità e alla gestione del ciclo di vita dell'identificatore. Il *DID Document* completo non viene memorizzato direttamente sul ledger, ma conservato in uno storage off-chain. Sul ledger viene invece mantenuto un riferimento opaco al documento e il relativo hash, così da ridurre il carico informativo sulla DLT e limitare la divulgazione di informazioni potenzialmente correlabili.

Un record dell'Identity Registry viene rappresentato come segue:

- *did*: identificativo decentralizzato del soggetto;
- *DIDdocumentRef*: CID del *DID Document*, necessario alla risoluzione;
- *documentContentType*: formato del *DID Document*, nel caso specifico *application/did+json*;
- *status*: stato del *DID*, ad esempio attivo o disattivato;
- *version*: versione corrente del *DID Document* associato;
- *previousVersion*: riferimento alla versione precedente, utile per ricostruire la storia degli aggiornamenti;
- *createdAt*: data di creazione del record;
- *updatedAt*: data dell'ultimo aggiornamento;
- *deactivatedAt*: data di eventuale disattivazione del *DID*.

L'Identity Registry supporta inoltre il ciclo di vita del *DID*. Quando il controller del *DID* aggiorna le proprie chiavi, ruota un metodo di verifica o modifica il *DID Document*, il registry viene aggiornato con il nuovo hash, la nuova versione e il riferimento aggiornato al documento. La rotazione delle chiavi non richiede la creazione di un nuovo *DID*, ma richiede l'aggiornamento e il versionamento dello stato associato al *DID Document*. Il mantenimento di riferimenti alle versioni precedenti consente di supportare verifiche storiche e attività di audit, ad esempio per stabilire se una determinata chiave fosse valida al momento dell'emissione di una credenziale o della firma di una richiesta.

L'Identity Registry viene, quindi, consultato ogni volta che il sistema deve verificare una firma, una *VC*, un consenso, una revoca o una delega associata a un *DID*. In tali casi, il verifier risolve il *DID*, recupera il *DID Document* e utilizza le chiavi pubbliche in esso contenute per controllare la firma o la prova crittografica presentata.

DID Document

Il **DID Document** è il documento tecnico associato a un Decentralized Identifier. La sua funzione è descrivere i metodi di verifica e le chiavi pubbliche attraverso cui il controller di un DID può dimostrare il controllo crittografico dell'identificatore e attraverso cui altri soggetti possono verificare firme, autenticazioni, presentazioni, consensi, deleghe o credenziali.

Durante la risoluzione del *DID*, il sistema recupera il *DID Document* completo dallo storage off-chain utilizzando il *CID* registrato nell'Identity Registry. Il ledger non conserva il documento integrale, ma solo il relativo *CID*, lo stato del *DID* e le informazioni di versionamento.

I campi del *DID Document* associato sono i seguenti:

- *id*: DID a cui il documento si riferisce;
- *controller*: entità che controlla il *DID Document*;
- *verificationMethod*: insieme dei metodi di verifica associati al *DID*, contenenti le chiavi pubbliche o il materiale crittografico necessario alla verifica delle firme;
 - *authentication*: specifica quali metodi di verifica possono essere utilizzati per autenticare il controller o il subject del *DID*;
 - *assertionMethod*: specifica quali metodi di verifica possono essere utilizzati per firmare asserzioni, incluse eventuali VC;
 - *keyAgreement*: specifica quali metodi di verifica possono essere utilizzati per lo scambio di chiavi o la cifratura di messaggi destinati al DID.
- *service*: eventuali endpoint di servizio associati al DID, da utilizzare solo quando necessari e preferibilmente in forma opaca o minimizzata.

Infine, è importante sottolineare che la presenza dell'*assertionMethod* nei *verificationMethod* del DID Document non implica direttamente che l'utente corrispondente a tale DID sia effettivamente un issuer autorizzato; infatti, ciò viene gestito separatamente tramite il Trust Registry.

Generazione del DID

Seguendo il modello della Self-Sovereign Identity, la generazione del *DID* avviene a partire dalla creazione del materiale crittografico controllato dal soggetto. Il *DID* non viene assegnato per rappresentare direttamente un ruolo o un'autorizzazione, ma per consentire al soggetto di dimostrare il controllo crittografico del proprio identificatore all'interno della rete.

Il processo di generazione può essere rappresentato come segue:

1. **Generazione della coppia di chiavi:** Il soggetto genera una o più coppie di chiavi crittografiche, associate alle *verification relationship* necessarie, per *authentication*, *assertionMethod* o *keyAgreement*. La scelta di usare chiavi distinte per funzioni diverse è adottata per separare gli ambiti d'uso e limitare l'impatto di una compromissione. La chiave privata rimane sotto il controllo del soggetto, mentre la chiave pubblica viene inserita nel DID Document come metodo di verifica con il relativo periodo di validità.
2. **Creazione dell'identificatore DID:** Viene generato l'identificatore decentralizzato secondo il DID method adottato dalla rete permissioned. L'identificatore non deve contenere dati personali, informazioni sanitarie, ruoli, qualifiche o altri elementi semanticamente riconducibili al soggetto.
3. **Creazione del DID Document:** Viene creato il DID Document associato al DID, contenente le informazioni tecniche necessarie alla verifica crittografica, come le chiavi pubbliche e le relative

verification relationship. Il documento viene mantenuto minimale e non contiene attributi anagrafici, clinici o autorizzativi.

4. **Conservazione off-chain del DID Document:** Il DID Document completo viene conservato off-chain, nello storage dell'organizzazione a cui il soggetto del DID appartiene.
5. **Registrazione sull'Identity Registry:** Sul ledger viene registrato un record nell'Identity Registry contenente il DID, il CID, la versione e i metadati tecnici necessari alla risoluzione.
6. **Verifica della richiesta di registrazione:** La rete verifica che il DID non sia già presente, che il DID Document sia correttamente formato e che il soggetto dimostri il controllo della chiave privata corrispondente alla chiave pubblica indicata nel DID Document. Solo dopo tali controlli il DID viene considerato attivo.

Verifiable Credential

Le **Verifiable Credentials** sono utilizzate nel sistema per attestare in modo crittograficamente verificabile gli attributi degli **utenti operativi**, come medici, infermieri, tecnici di laboratorio, auditor o altri soggetti abilitati a svolgere operazioni sui documenti o sui processi di governance.

Nel modello proposto, le VC non sono utilizzate ordinariamente per attestare il ruolo di paziente, essendo che quest'ultimo è identificato tramite un DID e può utilizzare tale DID per autenticarsi, dimostrare il controllo del proprio identificatore e firmare consensi, revoche, deleghe o richieste di accesso. La legittimazione del paziente ad autorizzare l'accesso ai propri dati non deriva, quindi, dal possesso di una VC di ruolo, ma dall'associazione tra il documento clinico e il DID del paziente indicata nel Document Lifecycle Registry (che consente di verificare che il DID firmatario corrisponda al soggetto documentale o autorizzante associato a quella risorsa nel sistema).

Quando il paziente firma un consenso o una revoca, il sistema verifica che la firma sia stata prodotta tramite una chiave associata al DID registrato come titolare o soggetto autorizzante per quel documento. Tale scelta introduce un trade-off in termini di correlabilità, poiché la presenza del DID del paziente nel Document Lifecycle Registry può consentire di collegare più documenti allo stesso soggetto pseudonimo, ma viene assunta come scelta progettuale compatibile con una rete permissioned, nella quale l'accesso al ledger è limitato ai nodi autorizzati e i documenti clinici sono comunque conservati off-chain in forma cifrata.

Una VC non rappresenta direttamente un'autorizzazione applicativa, ma un insieme di attributi attestati da un issuer riconosciuto. La decisione finale di accesso viene comunque demandata al Policy Engine, che valuta tali attributi rispetto al contesto della richiesta, al tipo di documento, alla finalità dichiarata e alle regole di governance.

Nel sistema considerato, gli **issuer** sono rappresentati dalle organizzazioni sanitarie appartenenti alla rete, che emettono credenziali a favore dei propri utenti operativi. L'issuer firma la credenziale utilizzando una chiave associata al proprio DID, indicata nel relativo DID Document tramite la relazione *assertionMethod*. Il fatto che un'organizzazione appartenga alla rete non implica automaticamente che essa possa emettere qualsiasi tipo di credenziale. La legittimità dell'issuer deve essere valutata tramite un Trust Registry, il quale specifica quali soggetti sono riconosciuti come issuer e per quali tipologie di credenziali. In questo modo, la verifica di una credenziale richiede sia un controllo crittografico della firma, sia un controllo di governance sulla competenza dell'issuer rispetto alla credenziale emessa.

Tipi di VC

Nel sistema possono essere definite diverse tipologie di Verifiable Credential, in base ai ruoli e agli attributi rilevanti per le policy. In particolare, le principali sono:

- *MedicalProfessionalVC*, assegnata ai medici;
- *NurseProfessionalVC*, assegnata agli infermieri;
- *TechnicianProfessionalVC*, assegnata ai tecnici e agli operatori d'ufficio del sistema;
- *AuditorProfessionalVC*, assegnata agli auditor.

L'elenco non è chiuso: ulteriori *credential types* possono essere introdotti tramite policy di governance e registrati nel Trust Registry, indicando quali issuer sono autorizzati a emetterli.

Tutte le VC condividono un insieme di campi comuni, necessari per identificare il soggetto, l'issuer, il periodo di validità e le informazioni essenziali della credential. A tali campi comuni si aggiungono poi attributi specifici, definiti in funzione della categoria professionale dell'utente a cui la VC viene rilasciata.

Struttura VC di base

I campi principali sono:

- *@context*: contesto semantico della credenziale, necessario per interpretare correttamente i campi secondo il modello VC;
- *id*: identificativo univoco della Verifiable Credential;
- *issuerDID*: DID dell'organizzazione che ha emesso la credenziale;
- *type*: obbligatorio per il tipo di VC tra quelli previsti
- *credentialSubject*: insieme degli attributi attestati dall'issuer sul soggetto della credenziale, in particolare:
 - *subjectDID*: DID del soggetto titolare della credenziale;
 - *role*: ruolo attestato dall'issuer, ad esempio medico, infermiere o tecnico di laboratorio;
 - *organizationDID*: DID o riferimento dell'organizzazione di appartenenza;
 - *department*: reparto o unità organizzativa di appartenenza (in caso di utente operativo);
 - *specialization*: specializzazione professionale del soggetto, ad esempio cardiologia, anestesia, radiologia o medicina interna;
 - *constraints*: eventuali vincoli aggiuntivi previsti dalla credenziale, come limiti temporali, limiti organizzativi o limiti sul tipo di documento accessibile.
- *proof*: prova crittografica, cioè la firma digitale dell'issuer sulla credenziale.
- *credentialSchema*: riferimento allo schema della credenziale, utilizzato per verificare che la struttura e i campi della credenziale siano conformi a una tipologia riconosciuta dal sistema;
- *validFrom*: data a partire dalla quale la credenziale è considerata valida;
- *validUntil*: data di scadenza della credenziale.

Emissione di una credenziale

L'emissione di una Verifiable Credential avviene da parte di un'organizzazione riconosciuta come issuer dalla governance della rete. Prima di emettere la credenziale, l'issuer deve verificare internamente la relazione con il soggetto a cui la credenziale viene rilasciata, ad esempio il rapporto di lavoro, l'appartenenza organizzativa, l'iscrizione a un reparto, la qualifica professionale o la delega funzionale.

Il processo di emissione può essere rappresentato come segue:

1. l'utente operativo **presenta il proprio DID**;

2. l'organizzazione sanitaria **verifica l'identità reale e il ruolo** del soggetto secondo le proprie procedure interne;
3. l'organizzazione **verifica di essere autorizzata**, secondo il Trust Registry, a emettere quel tipo di credenziale;
4. l'issuer **costruisce la Verifiable Credential** inserendo gli attributi attestati;
5. l'issuer **firma la credenziale** con una chiave associata al proprio DID e la cui chiave pubblica corrispondente è indicata nel DID Document tramite *assertionMethod*;
6. l'issuer inizializza nel **Credential Status Registry una entry** relativa allo stato della credenziale, senza pubblicare il contenuto della credenziale in chiaro;
7. la credenziale viene **consegnata all'holder**, che la conserva nel proprio wallet.

Conservazione di una VC

La Verifiable Credential non viene conservata in chiaro sul ledger condiviso, bensì viene memorizzata dall'**holder** all'interno di un wallet. L'holder è, quindi, il soggetto che presenta la credenziale quando deve dimostrare un attributo professionale, organizzativo o funzionale.

Credential Status Registry

Il Credential Status Registry sopradescritto fornisce un punto condiviso, verificabile e aggiornato per controllare lo stato delle VC; esso non conserva le VC in chiaro e non contiene gli attributi professionali o organizzativi attestati nelle credenziali stesse.

Una possibile entry del Credential Status Registry è la seguente:

```
{
  "credentialId": "urn:uuid:9f3a2c1e-71a0-4d91-b8b2-13e8f6a2aa90",
  "issuerDID": "did:health:organization:ospedale-1",
  "status": "active",
  "version": 1,
  "createdAt": "2026-01-01T00:00:00Z",
  "updatedAt": "2026-02-01T00:00:00Z",
  "proof": "..."
}
```

All'interno della struttura si ha:

- il *credentialId*, che corrisponde all'ID della VC (necessario a ricercare le informazioni sulla specifica VC all'interno del registry);
- il DID dell'issuer che serve a capire chi ha emesso (e può eventualmente modificare) la VC e anche a ricavare la chiave con cui validare la proof per essere certi che il record non sia stato modificato da un soggetto non autorizzato.

Revoca di una credenziale

Un aspetto essenziale nella gestione delle Verifiable Credentials è la revoca. Una credenziale firmata può restare tecnicamente verificabile anche dopo aver perso validità sostanziale, ad esempio perché il rapporto tra il soggetto e l'organizzazione è cessato o il ruolo è cambiato.

Per questo motivo, ogni decisione di accesso deve essere presa controllando non solo la firma della credenziale, ma anche il suo stato corrente nel Credential Status Registry. La revoca non elimina la credenziale già emessa, che può continuare ad essere conservata dall'holder nel wallet e dagli eventuali sistemi di audit, ma il suo stato viene aggiornato in modo che essa non sia più accettabile per future decisioni autorizzative.

Le informazioni dettagliate sulla revoca non sono pubblicate nel registro condiviso, con lo scopo di minimizzare la divulgazione di informazioni, bensì il ledger si limita registrare l'aggiornamento dello stato e i relativi metadati tecnici mentre i dettagli vengono mantenuti nel registro interno dell'issuer e negli Audit registry.

Verifica di una VC

Quando un verifier riceve una credenziale all'interno di una Verifiable Presentation, non deve limitarsi a controllare la firma crittografica, bensì la verifica deve comprendere una sequenza di controlli tecnici, temporali e di governance.

Solo al termine di questi controlli la credenziale può essere considerata valida come fonte di attributi. Anche in questo caso, la credenziale non determina direttamente l'autorizzazione, ma fornisce input verificati al Policy Engine.

Verifiable Presentation (VP)

L'holder non presenta la VC nella sua forma originaria, ma costruisce una VP, cioè un oggetto firmato dall'holder stesso che contiene la credenziale oppure una selezione verificabile di attributi derivati da essa. La VP consente al verifier di accertare non solo che la credenziale sia stata emessa da un issuer riconosciuto, ma anche che la presentazione sia stata generata dal soggetto legittimato a utilizzarla; infatti, all'interno della VP viene allegata la firma dell'holder.

Il verifier deve, quindi, verificare il legame tra holder della VP e subject della VC; questo perché la firma dell'holder dimostra il controllo della chiave usata per generare la presentazione, ma l'autorizzazione richiede anche che tale holder coincida con il soggetto della credenziale.

Per prevenire attacchi di replay, la VP deve includere elementi contestuali come:

- un *nonce* o challenge generato dal verifier;
- l'identificativo del verifier destinatario;
- il dominio o contesto della richiesta;
- un *timestamp* di creazione;
- un eventuale identificativo della richiesta autorizzativa.

Questi elementi vengono inclusi nel contenuto firmato dall'holder, così che la presentazione risulti valida solo per quella specifica richiesta, in quel determinato contesto e per un intervallo temporale limitato.

Information disclosure sulla credenziale

Quando un utente operativo presenta una Verifiable Credential a un'organizzazione diversa da quella che l'ha emessa, non è sempre necessario rivelare l'intero contenuto della credenziale. Il sistema supporta quindi la presentazione selettiva degli attributi rilevanti per la specifica richiesta, ad esempio ruolo, specializzazione o appartenenza organizzativa. Questo approccio consente di rispettare il principio di minimizzazione dei dati, evitando che il verifier venga a conoscenza di informazioni non rilevanti per la specifica richiesta di accesso.

Nel modello proposto, la **selective disclosure** può essere implementata mediante il meccanismo **SD-JWT**. In questo scenario, SD-JWT consente una gestione più semplice e una verifica efficiente rispetto a meccanismi più complessi (come quelli basati su **Merkle Tree**) pur richiedendo il calcolo di un digest per ciascun attributo selettivamente rivelabile.

In riferimento alla struttura della VC sopra descritta, i campi di cui è necessario andare a effettuare information disclosure sono *role*, *organizationDID*, *department*, *specialization* e *constraints* (presenti all'interno del blocco di *credentialSubject*).

L'issuer che emette la credenziale dovrà andare a creare una disclosure per ognuno di questi campi, composta da [*salt* + *nome del campo* + *valore del campo*]; di tale disclosure viene calcolato un digest crittografico. È necessario utilizzare un *salt* distinto per ciascun attributo, soprattutto quando gli attributi hanno bassa entropia, come *role* = *medico* o *department* = *cardiologia*. In assenza di *salt*, un soggetto esterno potrebbe tentare attacchi a dizionario calcolando gli hash dei valori più probabili e confrontandoli con le disclosure presenti nella VC ricevuta.

Formalmente, sia *C* una credenziale prima della trasformazione SD-JWT, composta da claim pubblici e claim selettivamente rivelabili. Indichiamo con *P* l'insieme dei claim pubblici e con $S = \{a_1, \dots, a_n\}$ l'insieme dei claim soggetti a selective disclosure.

Per ogni claim selettivo $a_i = (name_i, value_i)$, l'issuer genera un salt casuale $salt_i$ e costruisce una disclosure:

$$Disclosure_i = [salt_i, name_i, value_i]$$

Da ciascuna disclosure viene calcolato un digest:

$$digest_i = HASH(Disclosure_i)$$

Il payload firmato dall'issuer contiene i claim pubblici e, al livello JSON appropriato, un array *_sd* contenente i digest dei claim selettivamente rivelabili. Se i claim selettivi appartengono a *credentialSubject*, il payload può essere rappresentato come:

$$Payload_{SD-JWT} = P \cup \{credentialSubject = P_{CS} \cup \{_{sd} = Shuffle(\{digest_1, digest_2, \dots, digest_n\})\}\}$$

Dove P_{CS} è l'insieme dei claim di *credentialSubject* lasciati in chiaro. L'issuer firma questo payload e consegna all'holder l'Issuer-signed JWT insieme all'elenco completo delle disclosure. In fase di presentazione, l'holder invia al verifier solo le disclosure relative agli attributi che intende rivelare; il verifier ricalcola i digest delle disclosure ricevute e verifica che siano presenti nell'array *_sd* del payload firmato.

Se il controllo ha esito positivo, il *verifier* può accertare che gli attributi rivelati appartengono alla credenziale originale emessa dall'issuer, senza ottenere accesso agli attributi non divulgati.

Livello di Storage

Il livello di storage è deputato alla memorizzazione off-chain delle informazioni che, per ragioni di efficienza, riservatezza e minimizzazione dei dati, non vengono salvate integralmente sul ledger. In questo livello sono conservati i contenuti completi di risorse come documenti e policy mentre il ledger mantiene soltanto i riferimenti verificabili a tali risorse.

La descrizione nel dettaglio del meccanismo con cui ogni risorsa viene salvata è demandata alla sezione dedicata ai flussi.

Storage distribuito: IPFS

Nel sistema in esame è necessario gestire diverse risorse off-chain che devono essere rese disponibili a più organizzazioni senza essere salvate integralmente sul ledger. Tra queste risorse rientrano, ad esempio, DID Document, policy complete, documenti cifrati e batch di audit.

A tale scopo, viene introdotto **IPFS** (*InterPlanetary File System*), un sistema distribuito e peer-to-peer per la memorizzazione e il trasferimento di dati basato sul **content addressing**. Ogni risorsa caricata su IPFS viene identificata mediante un **CID** (*Content Identifier*), cioè un identificatore derivato dal contenuto stesso. Se il contenuto viene modificato, il CID risultante cambia e non corrisponde più al riferimento registrato sul ledger.

Nel modello proposto, ogni organizzazione mantiene un proprio nodo IPFS incaricato di conservare le risorse di propria competenza. Quando una risorsa viene caricata su IPFS, il nodo restituisce il relativo CID; tale riferimento viene poi registrato dal componente applicativo autorizzato nel registro logico appropriato.

IPFS non è usato come meccanismo di autorizzazione. Il fatto che un soggetto conosca un CID o possa recuperare un payload cifrato non implica che sia autorizzato a leggerne il contenuto. Nel sistema proposto, **IPFS fornisce content addressing e distribuzione del contenuto**; la riservatezza è garantita dalla cifratura, l'autorizzazione dal Policy Engine, mentre la disponibilità dipende da pinning e replica presso le organizzazioni. Poiché IPFS non fornisce cifratura automatica del contenuto, le risorse sensibili, come documenti clinici, policy patient-driven e batch di audit, vengono cifrate prima della pubblicazione.

Per garantire la persistenza delle risorse off-chain, i nodi IPFS delle organizzazioni adottano meccanismi di *pinning*, attraverso cui un contenuto associato a uno specifico CID viene mantenuto stabilmente sul nodo e non rimosso durante le operazioni di pulizia locale. **Le operazioni di pinning e unpinning sono gestite da un Pinning Manager locale** a ciascuna organizzazione e regolate da policy di governance, in modo da evitare rimozioni non autorizzate e mantenere coerenza tra i riferimenti registrati sul ledger e i contenuti effettivamente disponibili nello storage distribuito.

Per aumentare disponibilità e resilienza, ciascuna organizzazione avrà una replica del proprio nodo IPFS e del relativo Pinning Manager, evitando che l'indisponibilità di un singolo componente renda irrecuperabili risorse necessarie al funzionamento del sistema. Infatti, la presenza di un solo nodo IPFS o di un solo Pinning Manager introdurrebbe un possibile single point of failure e, nel caso del Pinning Manager, anche un potenziale single point of control a livello organizzativo. La **replica dei componenti** permette invece di aumentare la **robustezza complessiva del sistema**, mantenendo coerente il modello distribuito dell'architettura.

Storage delle policy

A livello di governance sono previsti due registri distinti per la gestione delle policy: il **Global Policy Registry** e il **Patient-Driven Policy Registry**. Le policy integrali non vengono salvate direttamente sui registri per ragione di efficienza, minimizzazione e, nel caso delle policy individuali, riservatezza. Il contenuto completo della policy viene conservato off-chain su IPFS, cifrato quando contiene informazioni personali, restrizioni individuali o dettagli non destinati a tutti i partecipanti.

Le policy di accesso globale e di gestione della governance vengono firmate dall'organizzazione che le ha proposte mentre le policy patient-driven vengono firmate dal paziente interessato.

Storage dei documenti

Lo storage off-chain dei documenti è gestito secondo lo stesso principio previsto per le policy patient-driven: **ciascuna organizzazione conserva i documenti di propria competenza** nello storage del nodo IPFS. Poiché tali documenti contengono dati sensibili dei pazienti, essi vengono **memorizzati esclusivamente in forma cifrata**, così da garantirne la riservatezza; inoltre, su ogni documento è apposta la firma dell'utente operativo che l'ha creato.

Per rafforzare l'isolamento tra le risorse e ridurre l'impatto di un'eventuale compromissione, ogni documento è cifrato tramite **una chiave crittografica dedicata**, denominata *Document Encryption Key*. Ciascun paziente conserva nel proprio wallet le *Document Encryption Key* associate ai propri documenti, alle quali può accedere mediante la propria chiave privata.

Inoltre, essendo che il CID utilizzato per recuperare il documento dallo storage è un identificatore creato a partire dal contenuto del documento stesso, non è necessario introdurre un ulteriore meccanismo applicativo di verifica dell'integrità del file recuperato da IPFS; infatti, se il documento cifrato venisse modificato in maniera fraudolenta, il contenuto alterato **produrrebbe un CID differente** e non corrisponderebbe più al riferimento registrato sul ledger.

Storage degli Audit

Così come per le policy patient-driven e per i documenti, anche gli audit log vengono **memorizzati dall'organizzazione interessata dall'evento registrato** (inclusendo anche gli eventi che riguardano il sistema nella sua interezza, ad esempio la generazione di una nuova policy di governance) negli storage dei nodi IPFS delle singole organizzazioni. IPFS garantisce l'immutabilità del singolo oggetto associato al CID, mentre la progressività e l'append-only della sequenza di audit sono garantite dall'ancoraggio dei batch sul ledger e dal collegamento crittografico tra batch successivi.

Per evitare che l'accesso agli audit log diventi indiscriminato, ogni batch di audit viene cifrato con una chiave dedicata. Tale chiave viene resa disponibile solo agli auditor o alle autorità autorizzate, secondo le policy di accesso globale relative agli audit. Anche l'accesso ai batch di audit e il rilascio delle relative chiavi sono registrati come eventi auditabili, dunque, ogni evento di audit viene firmato dal soggetto o dal componente che lo genera.

In particolare, gli eventi di audit non vengono registrati singolarmente sul ledger, ma vengono **ordinati e aggregati periodicamente in batch**. Il batch cifrato viene poi memorizzato sul nodo IPFS dell'organizzazione, che restituisce il relativo CID che a sua volta viene registrato sul ledger.

In fase di verifica, il CID consente di recuperare l'oggetto cifrato corretto e di rilevare eventuali alterazioni del contenuto del batch, degli eventi o del loro ordine.

In particolare, ogni evento sarà formato da un insieme di campi comuni, necessari a identificare l'evento, attribuirlo a un soggetto, collocarlo temporalmente e collegarlo alla policy o alla transazione

che lo ha prodotto. A questi campi comuni possono aggiungersi campi opzionali, utilizzati solo quando l'evento riguarda uno specifico documento, una delega, una credenziale o una policy.

Campi comuni

- *eventId* → identificativo univoco dell'evento;
- *batchId* → identificativo univoco del batch di eventi;
- *actor* → soggetto che ha richiesto o eseguito l'azione e può essere un utente operativo, un'organizzazione o un paziente;
- *action* → operazione richiesta o eseguita, alcuni esempi sono *read*, *update*, *revoke*, *createDelegation*, *registerPolicy*;
- *targetType* → tipo della risorsa coinvolta, può assumere valori come *document*, *delegation*, *credential*, *policy*, *did*, *auditLog*;
- *result* → esito dell'operazione o della decisione autorizzativa; può assumere valori come *allowed*, *denied*, *error*;
- *reasonCode* → motivazione sintetica dell'esito, ad esempio *revokedCredential*, *invalidSignature*, *policyNotSatisfied*, *delegationExpired*;
- *sourceComponent* → componente applicativo che ha generato l'evento, ad esempio *Document Manager*, *Policy Engine* (popolato solo se l'evento è stato generato da un componente applicativo, altrimenti vuoto);
- *timestamp* → momento in cui l'evento si è verificato;
- *policyRef* → riferimento alla policy applicata per valutare o autorizzare l'operazione (in caso di evento di registrazione di una policy di gestione, il campo rimarrà vuoto).
- *previousEventID* → identificativo dell'evento precedente;
- *previousEventHash* → hash dell'evento precedente.

Campi opzionali per eventi documentali

In aggiunta ai campi comuni, nel caso l'evento riguardi un documento, viene aggiunto il campo *documentRef*, ovvero il riferimento al documento coinvolto.

Campi opzionali per eventi di delega

In aggiunta ai campi comuni, nel caso l'evento riguardi creazione, uso o revoca di una delega, viene aggiunto il campo *delegationRef*, ovvero il riferimento alla delega coinvolta.

Campi opzionali per eventi sulle credenziali

In aggiunta ai campi comuni, nel caso l'evento riguardi emissione, presentazione, verifica o revoca di una Verifiable Credential, vengono aggiunti i campi *credentialRef*, ovvero l'identificativo della credenziale coinvolta, e *issuerRef*, ovvero il riferimento all'organizzazione che ha emesso la credenziale.

Campi opzionali per eventi di policy

Questi campi si usano quando l'evento riguarda creazione, aggiornamento, approvazione o revoca di una policy:

- *policyId* → identificativo della policy coinvolta.
- *approvalActors* → autorità che hanno approvato la policy o la modifica.
- *approvalThreshold* → soglia di governance raggiunta.

I campi *approvalActors* e *approvalThreshold* sono valorizzati solo per le policy soggette a un processo di approvazione distribuita, ovvero le policy di accesso globale o di governance; nel caso delle policy patient-driven, tali campi possono rimanere vuoti, poiché la policy deriva direttamente dalla volontà del paziente.

Campi opzionali per eventi di identità/DID

Il campo *DIDRef*, ovvero il DID coinvolto nell'evento, si usa quando l'evento riguarda la registrazione, l'aggiornamento, la disattivazione di un DID.

Storage dei DID Document

Sempre perseguendo l'obiettivo di realizzare un sistema il più possibile decentralizzato, ogni organizzazione mantiene all'interno dello storage del nodo IPFS i DID Document relativi agli utenti operativi e alle altre **figure** del sistema **appartenenti a tale organizzazione**. In questo modo, i DID Document non vengono conservati in un unico repository centralizzato, ma distribuiti presso le organizzazioni responsabili della loro gestione.

Per garantire la persistenza e impedire che il documento venga rimosso, il Pinning Manager dell'organizzazione provvede al pinning del contenuto. Inoltre, il *CID* fornito dal nodo IPFS dalla pubblicazione del *DID Document* viene **registrato sull'Identity Registry** insieme al *DID*, allo stato del DID e ad altri metadati descritti in precedenza, per consentire la risoluzione di un *DID* anche da parte di organizzazioni diverse da quella di appartenenza del soggetto.

Quando il DID Document viene aggiornato, viene pubblicato nuovamente su IPFS e l'Identity Registry viene aggiornato con il nuovo CID, mantenendo il riferimento alle versioni precedenti per finalità di audit.

Flussi operativi

Accesso a documento cifrato da parte di un utente operativo del sistema

Nel sistema proposto, un medico che intende accedere a un documento clinico non conosce necessariamente in anticipo quali attributi professionali debbano essere presentati per ottenere l'autorizzazione. Gli attributi richiesti possono infatti dipendere dal tipo di documento, dalla finalità dichiarata, dall'organizzazione che conserva la risorsa, dal contesto clinico, dalla presenza di un consenso del paziente e dalle regole definite nel *Policy Engine*.

I flussi descritti della soluzione proposta si basano sull'assunzione che il wallet del paziente sia online e non sia stato compromesso. Inoltre, una volta che un utente operativo ha ottenuto l'accesso ai documenti richiesti non è più possibile applicare alcun tipo di restrizione per quell'utente su quel documento.

Inizio della Access Intent Request

Il flusso inizia quando l'utente operativo seleziona, tramite il sistema applicativo, il documento clinico che intende consultare oppure una risorsa documentale a cui desidera accedere. In questa fase il medico non presenta ancora la propria *Verifiable Credential* professionale, bensì invia una richiesta preliminare di accesso, l'**Access Intent Request**.

Questa richiesta ha lo scopo di comunicare al sistema l'intenzione di accedere a una determinata risorsa e contiene i seguenti elementi:

- *preRequestId*, cioè l'identificativo univoco della richiesta preliminare, utilizzato per correlare le fasi successive del processo;
- *requesterDID*, cioè il *DID* dell'utente operativo richiedente;
- *verifierDID*, cioè il *DID* del verifier destinatario della richiesta rappresentato da un nodo dell'organizzazione;
- *documentId*, se il documento è già noto e identificabile;
- *action*, cioè l'azione richiesta;
- eventuali informazioni sulla finalità dichiarata e sul contesto della richiesta, se previste dal sistema applicativo.

La richiesta preliminare viene firmata dall'utente operativo mediante una chiave associata al proprio *DID*. In questa fase, la firma non serve ancora a dimostrare che il soggetto sia effettivamente un medico, né che possieda determinati attributi professionali; serve piuttosto a dimostrare che la richiesta proviene effettivamente dal controller del *requesterDID* indicato (chi riceve la richiesta effettua la risoluzione del *requesterDID* e preleva la chiave pubblica dal *DID Document* ottenuto per verificare la firma apposta).

Quando il verifier riceve la richiesta preliminare, effettua alcuni controlli iniziali. Prima di tutto verifica che la richiesta sia ben formata e che siano presenti i campi minimi sopra citati. Successivamente, il verifier risolve il *DID* dell'utente operativo tramite l'**Identity Registry**; in particolare recupera il **riferimento al DID Document**, controlla che il **DID sia attivo**, e infine **risolve il DID** mediante il *CID* presente nel record sull'*Identity Registry* e ottiene il *DID Document* dallo storage off-chain.

Se tutte le operazioni effettuate hanno esito positivo, il *verifier* controlla che la chiave usata per firmare la richiesta preliminare sia presente nel *DID Document* dell'utente e sia abilitata alla relazione di verifica **authentication**. Se la verifica ha esito positivo, il *verifier* considera il *requesterDID* **tecnicamente autenticato**, ma **non ancora autorizzato**.

Recupero dei metadati documentali e valutazione preliminare delle policy

A questo punto il verifier interroga il **Document Lifecycle Registry** utilizzando il *documentId* indicato nella richiesta preliminare. Dal registry vengono recuperati i metadati minimi del documento che non vengono rivelati all'utente, bensì il *verifier* invia al *Policy Engine* una richiesta di valutazione preliminare che li contiene e che non ha ancora lo scopo di decidere se concedere l'accesso, ma di determinare **quali informazioni siano necessarie per poter prendere una decisione**. Il *Policy Engine* riceve i dati già disponibili sia riguardanti la richiesta che il documento. Il risultato è una **Presentation Requirement**, cioè una descrizione dei requisiti che la *Verifiable Presentation* dovrà soddisfare.

In particolare, la Presentation Requirement contiene:

- il *presentationRequestId*, ovvero un identificativo collegato al *preRequestId* originario;
- il **DID del verifier**;
- il **dominio applicativo** della richiesta;
- il **documento** a cui la richiesta si riferisce;
- l'**azione** richiesta;
- la **finalità** dichiarata;
- una **challenge crittografica**, composta da un *nonce*, da un timestamp di emissione *issuedAt*, da un timestamp di scadenza *expiresAt*.

La Presentation Requirement specifica, inoltre, il **tipo di credenziale accettata** e quali attributi devono essere rivelati; tale richiesta viene effettuata dal *Policy Engine* sulla base della policy applicabili alla specifica richiesta dell'utente. Infine, il *verifier* restituisce al medico la Presentation Requirement **firmata dal verifier** stesso; in questo modo il medico può verificare che la richiesta provenga effettivamente dal soggetto indicato come *verifierDID* e che non sia stata alterata.

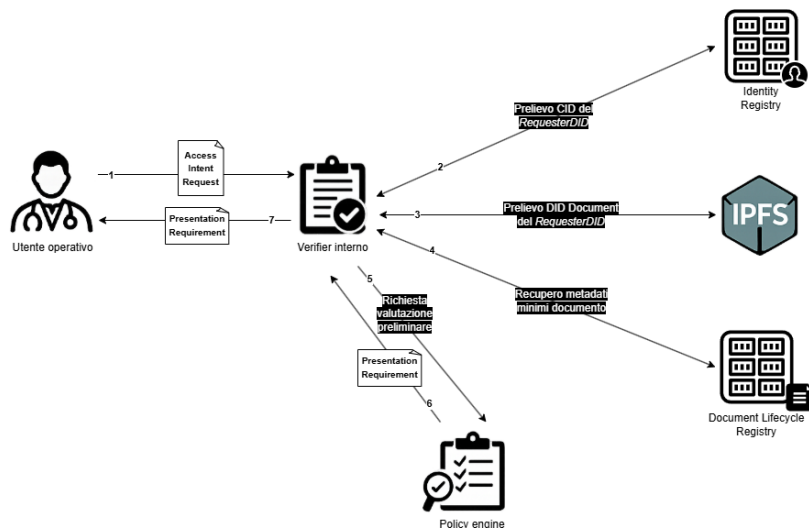


Figura 2, Accesso a documento cifrato da parte di un utente operativo del sistema

Costruzione della Verifiable Presentation

Una volta ricevuta la *Presentation Requirement*, l'utente operativo cerca all'interno del proprio *wallet* una VC in grado di soddisfare i requisiti richiesti, e soprattutto, sceglie quali attributi rendere visibili, in base alle informazioni richieste e al principio di divulgazione selettiva. Una volta selezionata VC e più precisamente gli attributi, il sistema costruisce la *VP*. Quest'ultima contiene:

- il *VPId*, ovvero l'**identificativo** della *VP*;
- il ***DID*** del **utente operativo** nel campo *holder*;
- il ***DID*** del **verifier** nel campo *verifier*;
- il riferimento al *presentationRequestId* ricevuto dal *verifier*;
- il riferimento al *preRequestId* della richiesta preliminare;
- il *nonce* ricevuto nella *Presentation Requirement* e il *timestamp* di creazione della *VP*;
- il *documentId*, l'azione e la finalità dichiarata per collegarla alla specifica richiesta di accesso.

In questo modo la stessa *VP* non può essere riutilizzata per un altro documento, per un'altra azione o per un'altra finalità. All'interno della *VP* viene integrata la rappresentazione della VC, rivelata in modo selettivo secondo il meccanismo della SD-JWT. Infine, il medico firma la *VP* con una chiave associata al proprio DID.

Verifica della Verifiable Presentation

Quando il *verifier* riceve la richiesta completa, controlla innanzitutto che essa corrisponda a una ***Presentation Requirement ancora valida***, poi verifica che il *presentationRequestId* esista, che sia collegato al *preRequestId* originario, che non sia scaduto e che non sia già stato utilizzato.

Controlla che il ***nonce*** contenuto nella *VP* coincida con quello generato nella *Presentation Requirement*. Verifica, inoltre, che il ***verifier*** indicato nella *VP* coincida con il *verifierDID* del sistema ricevente, che il **contesto applicativo** sia corretto e che il **documento**, l'**azione** e la **finalità** siano gli stessi dichiarati nella richiesta preliminare.

Il *verifier* procede quindi a risolvere nuovamente il *DID* dell'utente operativo e verifica che la chiave utilizzata per firmare sia la richiesta sia la *VP* sia effettivamente presente tra i ***verificationMethod*** del relativo *DID Document* e sia autorizzata per l'uso previsto, ad esempio per autenticazione o firma di presentazioni verificabili.

Successivamente, il *verifier* controlla la validità della firma apposta sulla richiesta completa e della proof associata alla *VP*. Se entrambe le verifiche hanno esito positivo, il sistema può concludere che l'utente operativo controlla effettivamente il *DID* indicato come *requesterDID* e che lo stesso soggetto agisce come holder della *VP*.

Verifica della Verifiable Credential

Il *Verifier* passa poi alla verifica della VC contenuta nella *VP*. In primis, il sistema verifica che il **tipo della credenziale** sia uno di quelli richiesti nella *Presentation Requirement*. Successivamente, **risolve il DID** dell'issuer tramite l'Identity Registry, **recupera il DID Document** dell'issuer e controlla che la **chiave** usata per firmare la credenziale sia presente nel *DID Document* dell'issuer e sia **abilitata** tramite *assertionMethod*. Se tale controllo va a buon fine, il *verifier* verifica la firma della credenziale; se è valida, il sistema sa che la credenziale è stata effettivamente emessa dal soggetto indicato come issuer.

Il sistema verifica poi la **validità temporale** della credenziale, controllando che la data della richiesta sia compresa tra *validFrom* e *validUntil*. Successivamente consulta il Credential Status Registry usando il *credentialId* per verificare che la credenziale risulti attiva; se risulta revocata, sospesa o non più valida, la richiesta viene respinta.

Il sistema consulta anche il Trust Registry, essendo che l'issuer non deve solo essere riconosciuto dalla rete, ma deve anche essere **autorizzato a emettere quel tipo specifico di credenziale** professionale.

Infine, il verifier controlla il legame tra la VP e la VC, per cui **l'holder della VP deve coincidere con il subjectDID della credenziale**. Inoltre, l'holder deve coincidere con il *requesterDID* della richiesta di accesso perché questo impedisce che un soggetto presenti una credenziale valida appartenente a un altro medico.

Verifica di una credenziale presentata con SD-JWT

Se la credenziale è presentata tramite SD-JWT, il verifier verifica le **disclosure ricevute**, composte da *salt*, nome del campo e valore del campo, successivamente, il sistema controlla che la VP soddisfi effettivamente la Presentation Requirement. Infatti, una VP può essere crittograficamente valida, ma non sufficiente dal punto di vista autorizzativo. Ad esempio, può essere firmata correttamente, contenere una credenziale valida e provenire da un issuer riconosciuto, ma non rivelare la specializzazione richiesta.

Valutazione dell'accesso in base a policy e deleghe

Una volta validata l'identità applicativa e gli attributi dell'utente, il Policy Engine valuta la richiesta rispetto alle policy applicabili e alle deleghe presenti, nell'ordine specificato precedentemente.

Verifica del permesso dalla policy patient-driven

Per effettuare la verifica di una policy patient-driven, il Policy Engine dovrà effettuare l'accesso al Patient-Driven Policy Registry per confrontare i campi presenti nella VC con i campi delle policy applicabili e verificare se l'utente operativo ha effettivamente l'autorizzazione ad accedere al documento.

In particolare, tra i vari controlli dovrà verificare che il *requesterDID* sia incluso negli *AllowedSpecificDIDs* e che l'azione specificata nella richiesta sia nelle *AllowedActions*. Se ciò non dovesse essere verificato perché non è presente alcuna indicazione, allora si procede con la verifica della presenza di eventuali deleghe, che possono estendere i permessi delle policy patient-driven per un determinato periodo di tempo.

Se, invece, l'accesso o l'azione saranno esplicitamente negati rispettivamente in *RestrictedUsersDID* e *RestrictedActions*, allora verrà comunicato il *deny* all'utente.

Verifica del permesso dalla delega

Se il permesso viene concesso da una delega, affinché la delega possa essere utilizzata, il sistema deve verificare che il soggetto richiedente sia effettivamente il **soggetto delegato**, che il **documento** richiesto coincida con quello indicato nella delega, che l'**azione** richiesta sia inclusa tra quelle **delegate** e che la **finalità** dichiarata rientri nello **scopo** per cui la delega è stata concessa.

Inoltre, è necessario procedere anche alla verifica per **ricercare un eventuale record di revoca** riferito alla delega trovata, in quanto, se questo è presente, il permesso non può essere utilizzato, anche se il periodo compreso tra *ValidFrom* e *ValidUntil* risulterebbe ancora in corso.

Infine, nel caso in cui la delega appartenga a una **catena di deleghe** e quest'ultima risulti *active*, il controllo deve essere comunque esteso anche alle deleghe precedenti, risalendo tramite il campo *ParentDelegationRef*. In questo modo il sistema verifica che non sia stata revocata non solo la delega direttamente utilizzata dal medico, ma anche una delle deleghe da cui essa deriva. Infatti, se una delega precedente nella catena è stata revocata, anche le deleghe successive che dipendono da essa non possono più costituire una base autorizzativa valida.

Aggiornamento del Delegation Registry

Per mantenere coerente e aggiornato il Delegation Registry, nel caso in cui durante la verifica della catena venga individuata la **revoca di una delega precedente**, l'organizzazione competente provvede a registrare ulteriori record di revoca anche per le deleghe successive che dipendono da quella revocata.

Questa esigenza deriva dal fatto che, al momento della revoca di una singola delega, non è sempre efficiente risalire immediatamente a tutte le eventuali **deleghe figlie** presenti sul ledger. Una ricerca completa delle deleghe derivate potrebbe infatti comportare un costo computazionale elevato, soprattutto in presenza di catene articolate o di un numero elevato di record. Per questo motivo, la registrazione esplicita dei record di revoca relativi alle deleghe figlie può avvenire in un **momento successivo**, ad esempio quando tali deleghe vengono intercettate durante una richiesta di accesso o mediante processi periodici di riallineamento.

In questa fase può, quindi, verificarsi una **discrepanza temporanea** tra lo stato apparente di alcune deleghe, che risultano ancora formalmente non revocate se considerate singolarmente, e il loro stato effettivo dal punto di vista autorizzativo.

Verifica dell'accesso tramite policy di accesso globale

Se non esistono policy patient-driven applicabili che concedano o neghino definitivamente l'accesso, e se non è presente una delega valida, il Policy Engine procede alla valutazione delle policy di accesso globale, **confrontando** tra le varie informazioni presenti soprattutto **ruolo e attributi** specificati nei campi della VC con quelli specificati sul Global Policy Registry in *Role* e *Attributes*.

Esito della verifica

Se la richiesta è incompatibile con una policy globale o viene esplicitamente vietata da una policy patient-driven e non è presente alcuna delega, il Policy Engine produce una **decisione di tipo deny**. Tale esito viene registrato nell'Audit Registry come evento di accesso negato, includendo il motivo sintetico del rifiuto, ad esempio delega scaduta, policy non soddisfatta o documento non accessibile nello stato corrente.

Se invece la richiesta è autorizzata dalle policy globali, allora non verrà subito comunicato al Document Manager, ma si procederà prima a **contattare il paziente** nel caso in cui quest'ultimo voglia negare il consenso all'utente operativo di accedere a tale documento. Se il paziente non specifica alcuna restrizione, allora il Policy Engine produce una **decisione strutturata di tipo allow**, contenente i riferimenti alla policy applicata, alla versione documentale, alla credenziale verificata, all'eventuale delega e allo stato di revoca consultato. Tale decisione viene registrata nell'Audit Registry e comunicata al Document Manager.

Prelievo del documento

A questo punto, il Document Manager **recupera dal Document Lifecycle Registry il CID associato alla versione documentale** richiesta e usa tale CID per **prelevare da IPFS il payload cifrato**.

Dopo aver recuperato il documento cifrato, il Document Manager non possiede la Document Encryption Key necessaria per fornire il contenuto in chiaro all'utente operativo che l'ha richiesto. Per questo motivo, il Document Manager invia il documento cifrato all'utente, dopo aver verificato la decisione *allow* del Policy Engine, senza la necessità di richiedere alcun consenso manuale e utilizzando la chiave contenuta nei metodi di *keyAgreement* presenti nel DID Document.

A questo punto, il medico contatta il wallet del paziente e presenta una prova verificabile dell'autorizzazione ottenuta dal sistema, rappresentata da un **Access Grant** firmato dal Policy Engine.

L'Access Grant contiene: *decisionId*, *requesterDID*, *documentId*, *version*, *action*, *purposeOfUse*, *validFrom*, *validUntil*, *policyRefs*, eventuali *delegationRefs*, riferimento allo stato di revoca consultato e firma del componente autorizzato che ha prodotto la decisione. In questo modo, il wallet del paziente non si limita a fidarsi della richiesta del medico, ma può verificare che esista effettivamente una decisione *allow* valida, recente e riferita proprio a quel documento, a quella versione, a quella finalità e a quello specifico richiedente.

Il wallet del paziente risolve quindi il *DID* dell'utente operativo tramite l'Identity Registry, recupera il relativo *DID Document* e verifica che il *requesterDID* indicato nell'Access Grant coincida con il *DID* del medico che sta richiedendo la chiave. Successivamente verifica la firma dell'Access Grant, controlla che l'intervallo temporale di validità non sia scaduto, che il *documentId* e la versione corrispondano al documento richiesto, che l'*action* e la *purposeOfUse* siano coerenti con l'operazione autorizzata e che l'Access Grant non sia già stato utilizzato o presentato fuori contesto.

Solo se tutte queste verifiche hanno esito positivo, il **wallet del paziente rilascia la DEK** necessaria alla decifratura della specifica versione documentale. La DEK viene inviata al medico cifrata tramite la chiave indicata nei metodi di *keyAgreement* del *DID Document* del medico, in modo che **solo il destinatario autorizzato possa decifrarla**. Dopo aver ricevuto la chiave, il medico decifra prima la DEK e poi il documento cifrato recuperato tramite il *CID* registrato nel Document Lifecycle Registry.

Il rilascio della DEK viene infine registrato nell'Audit Registry come evento autonomo, includendo almeno il riferimento all'Access Grant, il *requesterDID*, il *documentId*, la versione documentale, la finalità dichiarata, il *timestamp* e l'esito dell'operazione. In questo modo, non viene auditato soltanto l'accesso al documento cifrato, ma anche il momento in cui la chiave necessaria alla sua lettura è stata effettivamente rilasciata.

Accesso ad un documento cifrato o verifica selettiva da parte di un requester esterno

Quando l'accesso ad un documento non deve essere effettuato da parte di un utente operativo appartenente ad una delle organizzazioni del sistema, ma da un **requester esterno**, allora il flusso operativo subisce dei cambiamenti. Infatti, anche il requester esterno, che ad esempio possiamo individuare come **un'assicurazione sanitaria** che ha interesse nel leggere un referto specifico di un paziente, avrà una Verifiable Credential, la cui struttura non è però la medesima della Verifiable Credential descritta in precedenza, non essendo questo un utente operativo interno.

Dunque, il flusso ha inizio quando il requester esterno presenta una VC che attesta il proprio ruolo e l'organizzazione di appartenenza. In risposta il verifier interno dell'organizzazione **contatterà l'organizzazione del requester esterno** per ottenere le informazioni necessarie a verificare che il requester esterno sia legittimato e che possa quindi accedere ai documenti gestiti dall'organizzazione stessa.

Una volta ottenute le informazioni rilevanti, quali finalità, documento d'interesse e azione richiesta, il verifier le comunicherà al Policy Engine che valuta se la richiesta sia **compatibile con le policy globali** definite, accedendo al Global Policy Registry. Se la verifica ha come esito *allow*, allora si procederà ad effettuare una **richiesta di accesso esplicita al paziente** soggetto del documento a cui il requester esterno vuole accedere. Nel caso in cui il **paziente dà l'assenso**, allora inizia anche in questo caso lo stesso procedimento di **prelievo del documento** descritto nel caso dell'accesso da parte di utente operativo. Se, invece, il paziente non consente l'accesso, allora la decisione verrà comunicata al requester esterno.

In entrambi i casi, l'intera operazione deve essere registrata nell'Audit Registry, in cui deve essere riportato il verifier, la finalità, la policy applicata, l'esito della richiesta e il documento rilasciato o non.

Infine, il flusso descritto sopra è lo stesso anche in caso di verifica selettiva, in cui il requester esterno ha interesse solo nel sapere se un documento esiste, è valido o è stato revocato. In tal caso, però, non è necessaria alcuna interazione con il paziente, ma solo la verifica delle Policy Globali.

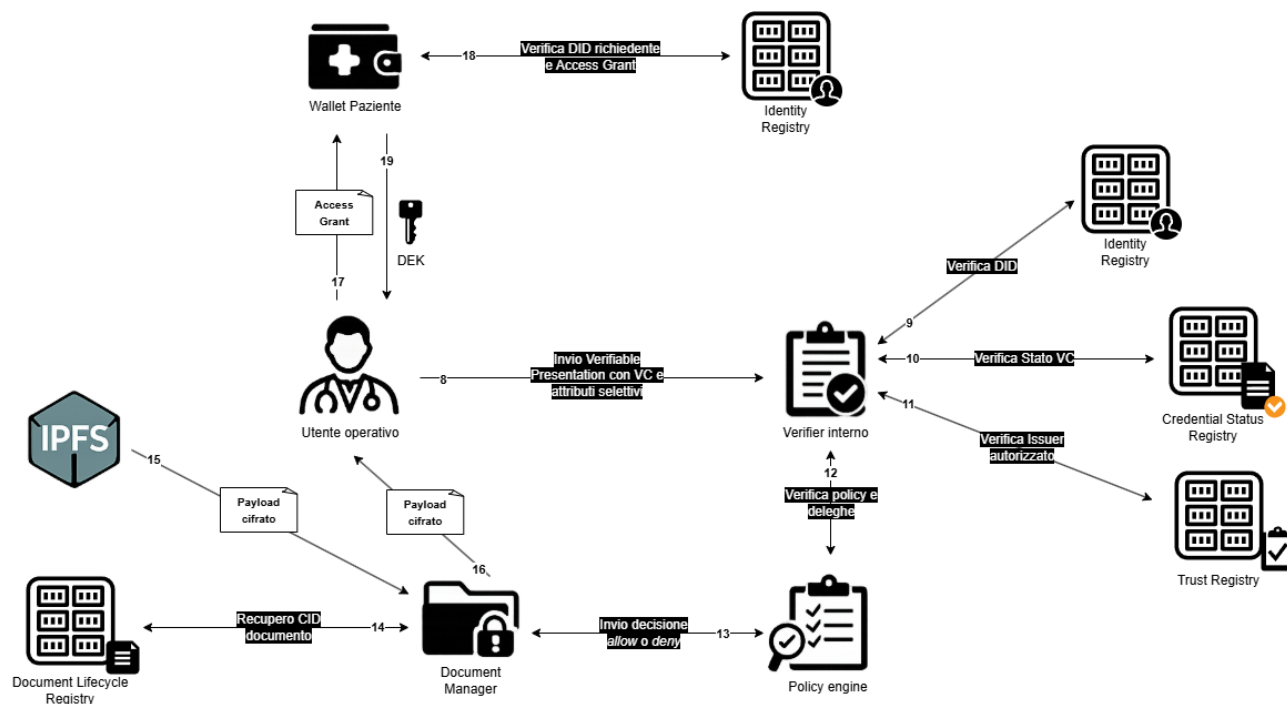


Figura 3, Accesso ad un documento cifrato o verifica selettiva da parte di un requester esterno

Creazione di un nuovo documento e registrazione nel Document Lifecycle Registry

Il flusso di creazione di un documento clinico riguarda il caso in cui un utente operativo autorizzato, produce un nuovo documento sanitario relativo a un paziente e il sistema deve registrarne l'esistenza, l'integrità, la versione e lo stato all'interno del **Document Lifecycle Registry**. In questo flusso il documento non viene mai salvato in chiaro sul ledger. Il contenuto clinico viene prodotto off-chain, firmato dall'utente operativo che lo ha creato, cifrato tramite una chiave dedicata e poi memorizzato su IPFS. Sul ledger viene registrato soltanto il record documentale minimizzato, contenente il riferimento verificabile al payload cifrato e i metadati necessari al lifecycle.

Avvio della richiesta di creazione

Il flusso inizia quando un utente operativo, ad esempio un medico, intende creare un nuovo documento clinico per un paziente. Il medico accede al sistema applicativo dell'organizzazione sanitaria e seleziona l'operazione di creazione; il sistema genera, quindi, una **Create Document Request**, cioè una richiesta formale di creazione del documento.

La richiesta contiene almeno:

- *requestId*, cioè l'identificativo univoco della richiesta;
- *requesterDID*, cioè il DID dell'utente operativo che intende creare il documento;
- *patientDID*, cioè il DID del paziente a cui il documento sarà associato;
- *issuerOrganizationDID*, cioè il DID dell'organizzazione che emette il documento;

- *documentType*, cioè la tipologia del documento;
- *action*, valorizzata a *create*;
- *purposeOfUse*, cioè la finalità dichiarata;
- *timestamp*, cioè il momento di generazione della richiesta.

La richiesta viene firmata dal medico tramite una chiave associata al proprio DID. In questa fase la firma serve a dimostrare che la richiesta proviene effettivamente dal controller del *requesterDID*, ma non è sufficiente a stabilire che il medico sia autorizzato a creare il documento.

Verifica iniziale e presentazione e verifica della VC

Quando il *verifier* interno dell'organizzazione riceve la richiesta, esegue innanzitutto una verifica tecnica dell'identità del richiedente. Successivamente controlla che la richiesta sia correttamente strutturata, che tutti i campi obbligatori siano presenti e che la firma digitale sia verificabile.

Per effettuare questi controlli, il *verifier* risolve il *requesterDID* tramite l'**Identity Registry**, recupera il *CID* del *DID Document* necessario alla risoluzione e ottiene il relativo DID Document da IPFS. A questo punto verifica che il DID sia attivo, che la chiave utilizzata per firmare la richiesta sia presente nel DID Document e che tale chiave sia autorizzata per la relazione di verifica corretta.

Poiché la creazione di un documento clinico rappresenta un'operazione applicativa rilevante, il sistema verifica non solo l'identità tecnica del richiedente, ma anche il ruolo e gli attributi professionali dell'utente operativo.

Per evitare di ripetere nel dettaglio il flusso già descritto per l'accesso ai documenti, viene adottato lo stesso schema generale:

Presentation Requirement → Verifiable Presentation → verifica della VP → verifica della VC → estrazione degli attributi rilevanti.

Il *verifier* richiede quindi al medico una **Verifiable Presentation** contenente una credenziale idonea a dimostrare il possesso degli attributi necessari per la creazione del documento clinico.

Gli attributi richiesti possono includere, ad esempio:

- *role*, ad esempio medico;
- *organizationDID*, ossia l'organizzazione di appartenenza;
- *department*, ossia il reparto;
- *specialization*, se richiesta dal tipo di documento da creare;
- eventuali vincoli specifici indicati nella credenziale.

Il medico genera una **Verifiable Presentation** firmata, utilizzando la **selective disclosure** tramite **SD-JWT**, in modo da rivelare esclusivamente gli attributi necessari alla specifica operazione richiesta.

Il *verifier* procede quindi alla verifica della VP e della VC applicando i controlli già definiti nel flusso di accesso ai documenti. In particolare, verifica:

1. la validità della firma dell'holder;
2. la corrispondenza tra l'holder della VP, il subject della VC e il *requesterDID*;
3. la validità della firma dell'issuer della credenziale;
4. la validità temporale della credenziale;
5. lo stato della credenziale nel **Credential Status Registry**;
6. la legittimità dell'issuer nel **Trust Registry**;
7. l'autorizzazione dell'issuer a emettere quel tipo di credenziale;

8. la corrispondenza tra gli attributi rivelati e quelli richiesti.

Solo dopo il completamento positivo di tali controlli, gli attributi professionali verificati vengono trasmessi al **Policy Engine**.

Valutazione autorizzativa da parte del Policy Engine

A questo punto, il **Policy Engine** valuta se il medico sia effettivamente autorizzato a creare quel tipo di documento clinico per quello specifico paziente e nel contesto indicato, applicando le policy definite dal sistema.

La valutazione tiene conto del *requesterDID*, degli attributi professionali verificati nella *VC*, del *patientDID*, del *documentType*, dell'azione richiesta, in questo caso create, della finalità dichiarata e dell'organizzazione emittente.

L'ordine di valutazione rimane coerente con quello già definito per gli altri flussi: prima vengono considerate le policy **patient-driven**, poi eventuali deleghe e infine le policy globali.

Nel caso della creazione documentale, le policy **patient-driven** possono assumere rilevanza quando il paziente ha definito restrizioni applicabili a una specifica categoria di documenti o a determinati soggetti. Poiché il documento non esiste ancora, il controllo non può essere effettuato su una risorsa documentale già presente, ma viene eseguito sulla base del soggetto richiedente, della categoria documentale e dell'azione richiesta, ossia create.

Se il medico agisce in virtù di una delega, il **Policy Engine** consulta il **Delegation Registry** per verificare che il richiedente corrisponda al soggetto delegato, che l'azione create sia inclusa tra le azioni delegate, ove previsto, e che la delega sia ancora temporalmente valida. Inoltre, controlla che la finalità dichiarata sia coerente con la delega, che non esista un record di revoca e che eventuali deleghe precedenti nella catena siano ancora valide.

Qualora non esista una base autorizzativa derivante da una policy patient-driven o da una delega, il **Policy Engine** valuta le policy di accesso globali presenti nel **Global Policy Registry**. In particolare, verifica che una policy globale consenta a un soggetto con quel ruolo e con quegli attributi professionali di creare quel tipo di documento per la finalità dichiarata.

Se la verifica ha esito positivo, il **Policy Engine** produce una decisione strutturata di tipo *allow*, che viene trasmessa al **Document Manager**.

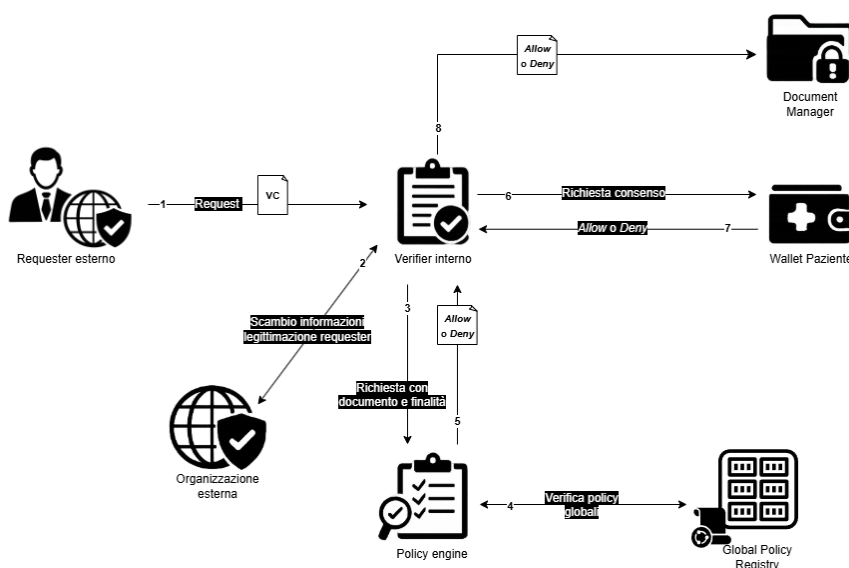


Figura 4, Creazione di un nuovo documento e registrazione nel Document Lifecycle Registry

Generazione, cifratura e memorizzazione del contenuto documentale

Dopo la decisione positiva del **Policy Engine**, il medico può procedere alla produzione del documento clinico. Il documento viene generato **off-chain** e contiene il contenuto clinico vero e proprio.

Oltre al contenuto clinico, il documento include alcuni metadati applicativi interni, tra cui il riferimento al paziente, il tipo di documento, il *DID* dell'autore, il *DID* dell'organizzazione emittente, il *timestamp* di creazione e il riferimento alla decisione autorizzativa che ha consentito la creazione del documento.

Una volta prodotto, il documento viene **firmato dall'utente operativo** che lo ha generato.

Prima della memorizzazione su IPFS, il documento firmato viene cifrato mediante una **Document Encryption Key**, ossia una chiave simmetrica dedicata esclusivamente a quello specifico documento. L'utilizzo di una chiave distinta per ogni documento consente di limitare l'impatto di un'eventuale compromissione, poiché la perdita o la revoca di una singola chiave non comporta automaticamente la compromissione degli altri documenti associati al paziente.

La **Document Encryption Key** viene generata dall'utente operativo che ha prodotto il documento. Successivamente, il documento già cifrato viene consegnato al **Document Manager**, che si occupa del caricamento del payload cifrato sul nodo IPFS dell'organizzazione.

Una volta completato il caricamento, il nodo IPFS restituisce il relativo **CID**, generato a partire dal contenuto cifrato. Il **Pinning Manager** dell'organizzazione provvede quindi a effettuare il pinning del contenuto, così da garantire che il documento cifrato non venga rimosso durante eventuali operazioni di pulizia locale.

A questo punto, il sistema dispone del riferimento necessario per procedere alla registrazione del documento nel **Document Lifecycle Registry**.

Poiché anche il paziente deve conservare nel proprio wallet sia le **Document Encryption Key** associate ai propri documenti sia i documenti stessi, è compito dell'utente operativo trasmettergli il documento creato e la relativa **DEK** utilizzata per cifrarlo.

Prima dell'invio, sia il documento sia la **Document Encryption Key** vengono cifrati utilizzando la chiave di *keyAgreement* presente nel **DID Document** del paziente. In questo modo, solo il paziente, tramite il proprio wallet e il controllo crittografico delle proprie chiavi, può accedere al contenuto ricevuto e alla chiave necessaria per decifrarlo.

Il wallet del paziente conserva quindi la **DEK** in modo protetto, rendendola accessibile esclusivamente attraverso i meccanismi crittografici controllati dal paziente stesso.

Costruzione Record documentale

Il **Document Manager** costruisce il record documentale da registrare sul ledger. Poiché si tratta della creazione della prima versione del documento, viene generato un nuovo *DocumentID*.

Il **Document Manager** invia una transazione alla **DLT** per registrare il nuovo record nel **Document Lifecycle Registry**. La transazione viene firmata dal componente, o dall'organizzazione, autorizzato alla registrazione.

Prima del commit, i peer della rete permissioned verificano la validità tecnica e applicativa della transazione. In caso di esito positivo, la transazione viene confermata e il nuovo record viene scritto sul ledger.

Dopo il commit, il documento entra a far parte dello **stato condiviso del sistema**. La sua esistenza, la versione registrata, lo stato documentale e il riferimento off-chain diventano così verificabili da tutte le organizzazioni autorizzate.

Creazione di una nuova versione di un documento esistente

Il flusso riguarda il caso in cui un utente operativo aggiorni un documento già presente nel sistema, senza creare un nuovo documento logico. In questo scenario, il *DocumentID* rimane invariato, poiché continua a identificare lo stesso documento nel tempo.

L'aggiornamento comporta invece la generazione di una nuova versione documentale, associata a un nuovo *CID*, a un nuovo *timestamp* e a un riferimento esplicito alla versione precedente.

La modifica non avviene mai **in-place**: la versione precedente non viene sovrascritta, ma viene conservata come evidenza storica e posta nello stato *Archived*. La nuova versione diventa invece la versione corrente del documento e viene registrata nello stato *Certified*.

Avvio richiesta e recupero stato del documento

Il flusso inizia quando l'utente operativo seleziona un documento già esistente e richiede di aggiornarlo.

La richiesta può essere rappresentata come una **Update Document Request**, contenente:

- *requestId*, ovvero l'identificativo della richiesta;
- *requesterDID*, ovvero il DID del richiedente;
- *patientDID*, ovvero il DID del paziente;
- *documentID*, ovvero l'identificativo logico del documento di cui si vuole produrre la nuova versione;
- *currentVersionNumber*, cioè la versione attuale che si intende aggiornare;
- *documentType*, ovvero il tipo di documento;
- *issuerOrganizationDID* ovvero il DID dell'organizzazione che ha emesso il documento;
- *action*, l'azione che si vuole compiere valorizzata a *update*;
- *purposeOfUse*, ovvero la finalità;
- *timestamp*, ovvero il timestamp di creazione della nuova versione.

La richiesta viene firmata dall'utente operativo. Come nel flusso di creazione, il verifier controlla il DID del richiedente, richiede una Verifiable Presentation, verifica la VC in essa contenuta, controlla lo stato della credenziale e verifica l'issuer tramite il Trust Registry. Questi passaggi non vengono ripetuti nel dettaglio perché coincidono con quelli già descritti.

Dopo la verifica iniziale dell'identità e degli attributi del richiedente, il **verifier** interroga il **Document Lifecycle Registry** utilizzando il *documentID*.

Il sistema recupera le informazioni relative al documento, tra cui la versione corrente, il relativo *CID*, lo stato della versione corrente, il *PatientDID* associato, il tipo di documento, l'organizzazione issuer e il riferimento a eventuali versioni precedenti.

Prima di consentire l'aggiornamento, il sistema verifica che il documento esista, che sia effettivamente associato al paziente indicato e che la versione da aggiornare corrisponda alla versione corrente. Controlla inoltre che lo stato della versione corrente sia compatibile con l'operazione di *update* e che il documento non si trovi nello stato *Revoked*.

Se il documento risulta già revocato, oppure se l'utente tenta di aggiornare una versione non corrente senza una motivazione ammessa dalla policy, il flusso viene interrotto e l'evento viene registrato nell'**Audit Registry**.

Valutazione autorizzativa del Policy Engine

Il **Policy Engine** valuta se l'utente operativo sia autorizzato ad aggiornare il documento indicato. La valutazione tiene conto degli attributi verificati nella VC, del *documentID*, del *documentType*, della versione corrente, dello stato del documento, del *PatientDID*, dell'azione richiesta, in questo caso update, e della finalità dichiarata.

L'ordine di valutazione delle policy rimane quello già definito in precedenza: prima vengono considerate le policy **patient-driven**, poi eventuali deleghe e infine le policy globali.

Nel caso dell'aggiornamento documentale, le policy **patient-driven** possono limitare la possibilità di modificare specifiche categorie di documenti oppure vietare l'operazione a determinati utenti. Una delega, invece, può autorizzare l'operazione di update solo se include espressamente tale azione, se è ancora valida, se non risulta revocata e se riguarda quello specifico documento o la relativa categoria documentale.

Le policy globali vengono usate per verificare se un soggetto con quel ruolo e quegli attributi possa aggiornare quel tipo di documento in quello stato e per quella finalità. In più, rispetto alla creazione iniziale, il Policy Engine deve controllare la compatibilità con lo stato di lifecycle. In particolare:

- un documento *Certified* può essere aggiornato generando una nuova versione;
- un documento *Archived* non dovrebbe essere aggiornato nel flusso ordinario, perché rappresenta una versione storica;
- un documento *Revoked* non può essere aggiornato come se fosse ancora valido.

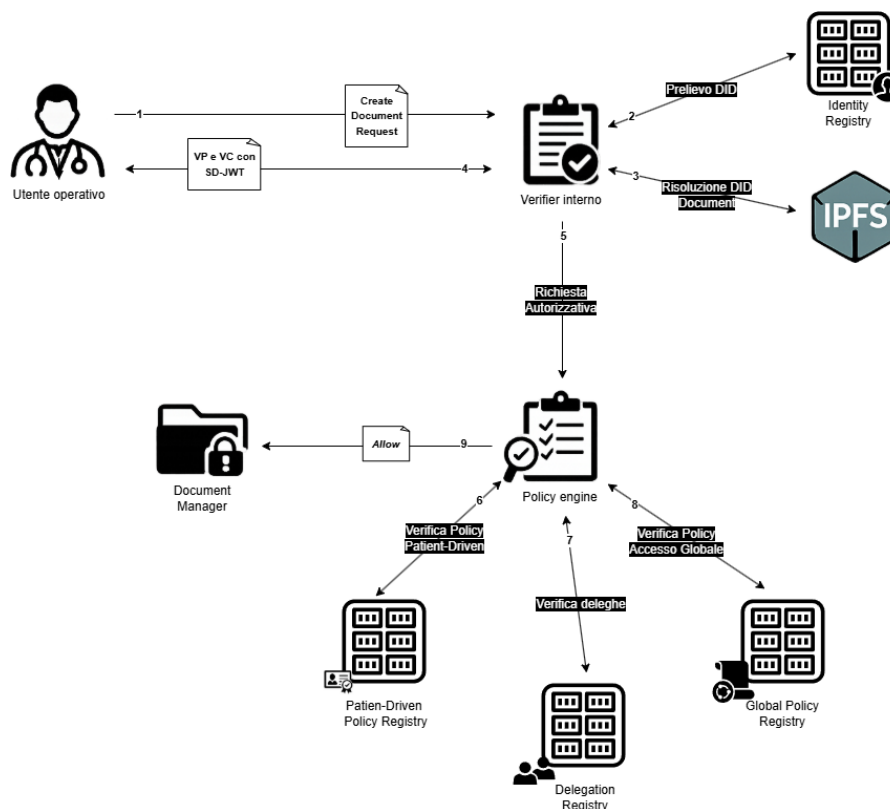


Figura 5, Creazione di una nuova versione di un documento esistente (parte 1)

Produzione, cifratura e memorizzazione

Dopo l'autorizzazione, l'utente operativo genera il nuovo contenuto documentale e lo firma. È importante che il documento aggiornato mantenga un riferimento logico alla versione precedente, senza però sovrascriverla. Il contenuto della versione precedente resta infatti conservato off-chain ed è ancora identificabile tramite il proprio *CID*, mentre la nuova versione viene trattata come un payload autonomo e avrà quindi un *CID* differente.

La nuova versione firmata viene cifrata tramite una nuova **Document Encryption Key**, dedicata esclusivamente alla versione aggiornata e generata dall'utente operativo che ha prodotto la modifica. Dopo la cifratura, il payload cifrato viene inviato al **Document Manager**, che lo trasmette al nodo IPFS dell'organizzazione competente.

Il nodo IPFS restituisce quindi un nuovo *CID*, calcolato sul contenuto cifrato della nuova versione. Successivamente, il **Pinning Manager** dell'organizzazione effettua il pinning del nuovo payload cifrato, così da garantirne la persistenza. La versione precedente rimane comunque disponibile su IPFS, poiché il suo *CID* continua a essere referenziato dal record precedente nel **Document Lifecycle Registry**.

L'utilizzo di una nuova **DEK** per ogni versione è necessario perché ciascuna versione documentale rappresenta un payload autonomo, con un proprio *CID*, un proprio stato di lifecycle e una propria gestione crittografica.

Come già previsto nel flusso di creazione di un nuovo documento, anche in questo caso il documento aggiornato e la relativa **DEK** devono essere inviati al paziente cifrati con la chiave di *keyAgreement* presente nel suo **DID Document**.

Costruzione nuovo record

A questo punto, il **Document Manager** costruisce il nuovo record documentale. A differenza del flusso di creazione iniziale, il *DocumentID* non viene modificato, poiché continua a identificare lo stesso documento logico nel tempo. Cambiano invece il numero di versione, il *CID* associato al nuovo contenuto cifrato e il riferimento alla versione precedente.

Il nuovo record contiene almeno il *DocumentID*, uguale a quello del documento originario, il *PatientDID*, il *DocumentType*, il *CreatorDID*, il nuovo *CID*, corrispondente alla versione cifrata aggiornata, lo stato documentale valorizzato a *Certified*, il *VersionNumber*, incrementato rispetto alla versione precedente, il *PreviousVersionPointer*, valorizzato con il riferimento alla versione immediatamente precedente, e il *CreationTimestamp*, che indica il momento di creazione della nuova versione.

Contestualmente alla registrazione della nuova versione, il sistema aggiorna anche lo stato operativo della versione precedente. Quest'ultima passa da *Certified* ad *Archived* mediante la creazione di un nuovo record documentale in cui viene modificato soltanto lo stato.

Questa operazione viene effettuata dal **Document Manager**. Dopo il commit sul ledger, la nuova versione diventa la versione corrente del documento.

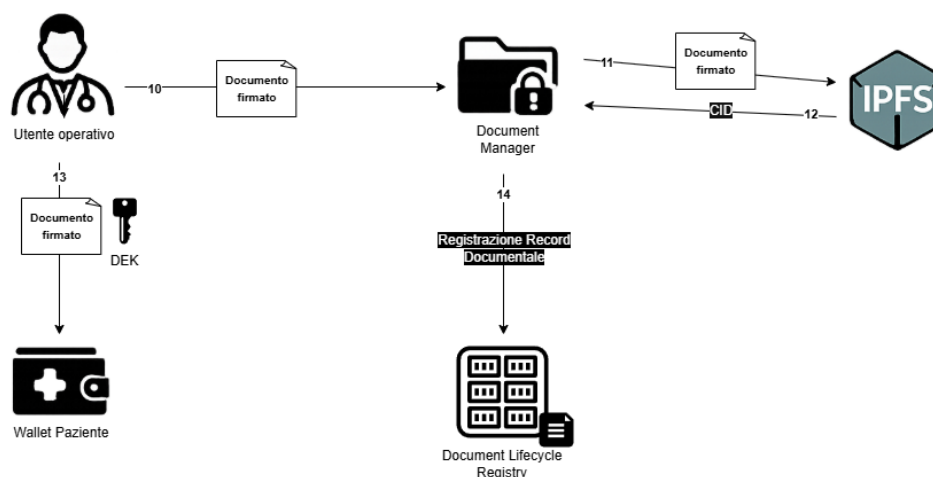


Figura 6, Creazione di una nuova versione di un documento esistente (parte 2)

Creazione di un record di delega

Il flusso di creazione di un record di delega riguarda il caso in cui un soggetto già autorizzato a operare su un documento clinico intenda conferire temporaneamente a un altro soggetto operativo un insieme limitato di permessi. La delega non viene considerata un'autorizzazione autonoma e permanente, ma un permesso derivato, circoscritto, revocabile e subordinato sia alle policy del sistema sia al consenso del paziente.

Nel sistema proposto, la delega non può essere registrata unilateralmente dal medico delegante. Anche se il delegante dispone già di un accesso valido al documento, il record di delega viene inserito nel **Delegation Registry** solo dopo una serie di verifiche. In particolare, il sistema controlla l'identità e gli attributi del delegante, l'identità del soggetto delegato, la possibilità effettiva per il delegante di delegare quella specifica azione, la compatibilità della delega con le policy globali e patient-driven, nonché la presenza del consenso esplicito del paziente. Solo al termine di questi controlli la transazione può essere confermata e registrata sulla **DLT permissioned**.

Il flusso ha inizio quando un utente operativo, ad esempio un medico cardiologo, intende delegare a un altro professionista sanitario una o più azioni su un documento clinico oppure su una specifica categoria documentale.

La richiesta contiene almeno:

- *proposalId*, cioè l'identificativo univoco della proposta di delega;
- *delegatorDID*, cioè il DID del soggetto che propone la delega;
- *delegateDID*, cioè il DID del soggetto che dovrebbe ricevere la delega;
- *targetDocumentID*, cioè il documento oggetto della delega;
- *issuerOrganizationDID*, cioè l'organizzazione che ha prodotto o gestisce il documento target;
- *delegatedActions*, cioè le azioni che si intendono delegare, ad esempio read o update;
- *purposeOfDelegation*, cioè la finalità della delega, ad esempio consulenza, secondo parere o continuità assistenziale;
- *validFrom*, cioè l'inizio della validità della delega;
- *validUntil*, cioè la fine della validità della delega;
- *parentDelegationRef*, se la proposta deriva da una delega precedente;
- *timestamp*, cioè il momento di generazione della proposta.

La richiesta viene firmata dal delegante utilizzando una chiave associata al proprio *DID*. In questa fase, la firma serve a dimostrare che la proposta di delega proviene effettivamente dal controller del *delegatorDID*.

Verifica tecnica del delegante

Quando il **verifier** interno dell'organizzazione riceve la proposta di delega, esegue innanzitutto una verifica tecnica del *DID* del delegante. A tale scopo, risolve il *delegatorDID* tramite l'**Identity Registry**, recupera da IPFS il relativo **DID Document** utilizzando il *CID* registrato sul ledger e controlla che il *DID* sia attivo, che il *DID Document* recuperato sia integro, che la chiave utilizzata per firmare la proposta sia presente nel documento e che sia abilitata per l'autenticazione o per la firma della richiesta. Infine, verifica la validità della firma apposta sulla proposta di delega.

Se questi controlli hanno esito positivo, il delegante viene considerato tecnicamente autenticato. Tuttavia, questa verifica non è ancora sufficiente per autorizzare la creazione della delega, poiché dimostra soltanto che la proposta proviene effettivamente dal soggetto associato al *delegatorDID*.

Poiché la delega riguarda un permesso operativo su un documento clinico, il sistema deve verificare anche il ruolo e gli attributi professionali del delegante. Anche in questo caso viene applicato il pattern già definito nei flussi precedenti: **Presentation Requirement** → **Verifiable Presentation** → **verifica della VP** → **verifica della VC** → **estrazione degli attributi rilevanti**.

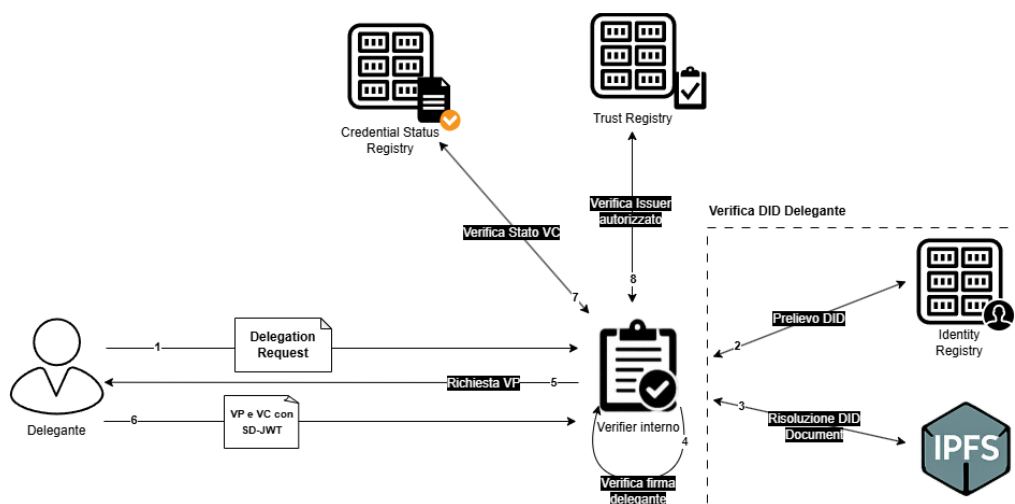


Figura 7, Creazione di un record di delega (parte 1)

Il delegante presenta quindi una **Verifiable Presentation** contenente una credenziale idonea a dimostrare il proprio ruolo e i propri attributi professionali. Il **verifier** controlla la validità della *VP*, verificando la firma dell'holder e la corrispondenza tra l'holder della *VP*, il subject della *VC* e il *delegatorDID*.

Successivamente, il **verifier** procede con la verifica della credenziale presentata. In particolare, controlla la firma dell'issuer, la validità temporale della credenziale, il suo stato nel **Credential Status Registry**, la legittimità dell'issuer nel **Trust Registry** e l'autorizzazione dell'issuer a emettere quella specifica tipologia di credenziale. Verifica, inoltre, gli attributi rivelati tramite **SD-JWT**.

Una volta completati positivamente questi controlli, gli attributi verificati vengono inoltrati al **Policy Engine**, che li utilizza per stabilire se il delegante possa effettivamente proporre quella delega.

Verifica del soggetto delegato

Prima di procedere con la valutazione autorizzativa, il sistema verifica anche il *delegateDID*. Questa verifica è necessaria per evitare che venga creata una delega verso un'identità inesistente, disattivata o non risolvibile all'interno del sistema.

Il **verifier** interroga quindi l'**Identity Registry** per accertare che il *delegateDID* sia effettivamente presente e che il DID risulti attivo. Successivamente recupera il relativo **DID Document** e verifica che sia correttamente accessibile e coerente con il CID registrato sul ledger.

Se questi controlli hanno esito positivo, il soggetto delegato viene considerato tecnicamente identificabile dal sistema. Questa verifica, tuttavia, non implica ancora che la delega sia autorizzata, ma conferma soltanto che il destinatario della delega corrisponde a un'identità valida e risolvibile.

Valutazione della delegabilità da parte del Policy Engine

A questo punto, il **Policy Engine** valuta se il delegante possa effettivamente creare una delega. La verifica non riguarda ancora l'accesso del delegato al documento, ma la possibilità di registrare una delega valida all'interno del sistema.

Per effettuare questa valutazione, il **Policy Engine** considera il *delegatorDID*, gli attributi verificati del delegante, il *delegateDID*, gli eventuali attributi verificati del delegato, il *targetDocumentID*, il *DocumentType*, il *PatientDID*, lo stato del documento, le azioni che si intendono delegare, la finalità della delega, il periodo di validità, le policy patient-driven applicabili e le deleghe già esistenti.

Il **Policy Engine** consulta innanzitutto il **Patient-Driven Policy Registry** per verificare se il paziente abbia definito restrizioni rilevanti rispetto alla delega proposta. Tali policy possono incidere in diversi modi: ad esempio, il paziente può aver vietato la delega per una determinata categoria documentale, può aver escluso specifici soggetti dalla possibilità di ricevere deleghe oppure può aver limitato le azioni delegabili.

Se il delegante agisce sulla base di una delega ricevuta, la proposta deve indicare il campo *parentDelegationRef*. In questo caso, il **Policy Engine** consulta il **Delegation Registry** per verificare la delega sorgente e accertare che la nuova delega possa effettivamente derivare da essa.

Il sistema controlla innanzitutto che la delega sorgente esista e che il delegante della nuova proposta coincida con il *DelegateDID* indicato nella delega originaria. Verifica, inoltre, che la delega sorgente sia attiva, non sia scaduta e non risulti associata ad alcun record di revoca. Se esiste una catena di deleghe precedente, il sistema controlla anche che tutti i passaggi della catena siano ancora validi.

Il **Policy Engine** verifica poi che la nuova delega non estenda il perimetro dei permessi ricevuti. In particolare, le azioni delegate devono essere uguali o più restrittive rispetto a quelle originarie, la finalità deve essere coerente o più specifica rispetto a quella iniziale, il periodo di validità non deve superare quello della delega sorgente, il documento oggetto della nuova delega deve coincidere con quello previsto dalla delega originaria oppure rientrare nel suo ambito e che la profondità della delega appena da inserire sia nel limite della profondità massima della catena di deleghe di cui fa parte.

Questo controllo è essenziale per evitare che una delega derivata attribuisca al nuovo delegato poteri più ampi rispetto a quelli concessi al delegante. Se una delle deleghe precedenti nella catena risulta revocata o non più valida, la nuova proposta viene respinta.

Produzione della decisione e richiesta consenso paziente

Se la proposta non soddisfa le condizioni richieste, il **Policy Engine** produce una decisione di tipo *deny*. L'esito negativo viene registrato nell'**Audit Registry** e nessun record di delega viene inserito nel **Delegation Registry**.

Se invece i controlli hanno esito positivo, il **Policy Engine** produce una decisione di tipo *allow-to-request-consent*. Questo esito non determina ancora la creazione della delega, ma indica soltanto che la proposta è compatibile con le policy del sistema e può quindi essere sottoposta al paziente per l'acquisizione del consenso esplicito.

Poiché il documento contiene dati sensibili del paziente, la delega può essere registrata solo dopo l'acquisizione del suo consenso esplicito. Per questo motivo, una volta ottenuta la decisione preliminare positiva del **Policy Engine**, il sistema genera una richiesta di consenso da sottoporre al paziente.

La richiesta contiene le informazioni necessarie per permettere al paziente di comprendere in modo chiaro l'oggetto e il perimetro della delega. In particolare, include il *proposalId*, il *delegatorDID*, il *delegateDID*, il *targetDocumentID*, il *documentType*, le azioni che si intendono delegare, la finalità della delega, il periodo di validità indicato tramite *validFrom* e *validUntil*, l'eventuale *parentDelegationRef*, il riferimento alla decisione preliminare del **Policy Engine**, il verifier o l'organizzazione richiedente, un timestamp e una challenge o nonce per prevenire attacchi di replay.

Il paziente riceve la richiesta tramite il proprio wallet oppure attraverso il canale applicativo previsto dall'organizzazione di riferimento.

Il wallet del paziente presenta la richiesta di delega in una forma comprensibile, mostrando chiaramente chi sta proponendo la delega, chi riceverebbe i permessi, quale documento o categoria documentale è coinvolta, quali azioni verrebbero delegate, per quale finalità e per quale periodo di tempo.

Se il paziente approva la proposta, il wallet genera un consenso firmato. Il consenso contiene le informazioni necessarie a collegare in modo univoco l'approvazione alla proposta ricevuta, tra cui il *consentId*, il *proposalId*, il *patientDID*, il *delegatorDID*, il *delegateDID*, il *targetDocumentID*, le azioni delegate, la finalità della delega, il periodo di validità indicato tramite *validFrom* e *validUntil*, il *timestamp*, il *nonce* o il riferimento alla challenge ricevuta, la decisione di consenso valorizzata ad *approved* e la firma del paziente.

La firma viene prodotta utilizzando una chiave associata al **DID Document** del paziente, così da garantire che il consenso provenga effettivamente dal soggetto titolare del *patientDID* e che non possa essere riutilizzato in modo fraudolento in un contesto diverso.

Costruzione record di delega

Dopo l'acquisizione di un consenso valido da parte del paziente, il **Document Manager** costruisce il record da inserire nel **Delegation Registry**. Questo record rappresenta la delega effettivamente spendibile nel sistema e costituisce la base autorizzativa che potrà essere verificata nei successivi flussi di accesso o di operazione sul documento.

Il record contiene gli elementi necessari a identificare la delega e a definirne il perimetro e il periodo di validità. Inoltre, se la delega deriva da una delega precedente, viene indicato anche il *ParentDelegationRef*. Sono poi inclusi il riferimento al consenso firmato del paziente, il riferimento alla decisione del **Policy Engine**, il timestamp di creazione e lo stato iniziale della delega, valorizzato ad *active*.

Il record non contiene il contenuto clinico del documento né informazioni sensibili non necessarie. Deve limitarsi agli elementi indispensabili per verificare, in un momento successivo, se il delegato possa utilizzare quella delega come base autorizzativa.

Prima del commit, i peer della rete **permissioned** verificano la correttezza tecnica e applicativa della transazione. Se i controlli hanno esito positivo, la transazione viene ordinata, validata e inserita nel ledger attraverso il normale processo di consenso della **DLT permissioned**. Dopo il commit, il record di delega diventa parte dello stato condiviso e verificabile del sistema.

Stato iniziale della delega

Una volta registrata, la delega assume lo stato *active*. Questo stato indica che la delega può essere utilizzata dal *DelegateDID* come possibile base autorizzativa in future richieste di accesso, aggiornamento o esecuzione di altre azioni compatibili con il perimetro definito nel record.

Tuttavia, una delega attiva non garantisce automaticamente l'accesso al documento. Nel momento in cui il soggetto delegato tenta di utilizzarla, il sistema deve comunque verificare che la delega esista, che non sia scaduta, che non sia stata revocata e che l'intera catena di deleghe sia ancora valida. Deve inoltre controllare che l'azione richiesta rientri tra quelle delegate, che la finalità dichiarata sia coerente con quella autorizzata, che il documento si trovi in uno stato compatibile con l'operazione richiesta e che non siano intervenute nuove policy patient-driven o globali tali da impedire l'accesso.

La delega, quindi, non rappresenta un accesso incondizionato, ma una base autorizzativa verificabile, che deve essere rivalutata ogni volta che viene utilizzata.

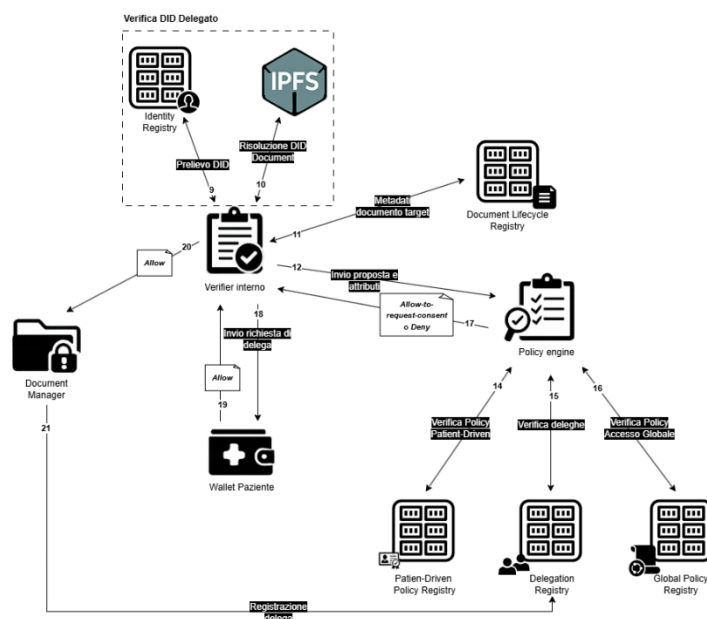


Figura 8, Creazione di un record di delega (parte 2)

Revoca di una delega

Il flusso di revoca di una delega riguarda il caso in cui una delega precedentemente registrata nel **Delegation Registry** debba essere resa non più spendibile prima della sua naturale scadenza. La revoca può derivare, ad esempio, da una violazione, da una decisione del paziente, da una decisione dell'organizzazione competente oppure dal verificarsi di una condizione prevista dalle policy.

La revoca non comporta l'eliminazione del record di delega originario. La delega resta presente nella storia del sistema, ma viene affiancata da un record di revoca che ne modifica lo stato effettivo ai fini autorizzativi. In questo modo, il sistema conserva la tracciabilità dell'evento e può distinguere correttamente tra una delega mai esistita, una delega esistente e ancora valida, una delega scaduta, una delega esplicitamente revocata.

Avvio della richiesta di revoca

Il flusso ha inizio quando un soggetto autorizzato richiede la revoca di una delega già presente nel **Delegation Registry**. La richiesta può provenire dal paziente titolare del documento, dall'organizzazione competente oppure da un componente applicativo autorizzato, ad esempio nei casi di revoca automatica per scadenza, violazione o altra condizione prevista dalle policy.

La revoca può inoltre essere richiesta dal soggetto delegante, qualora le policy del sistema gli consentano di revocare una delega precedentemente conferita. In ogni caso, la richiesta non produce automaticamente la revoca, ma avvia il flusso di verifica necessario ad accertare che il richiedente sia effettivamente autorizzato a renderla non più spendibile.

Il richiedente invia una richiesta che contiene almeno:

- *revocationRequestId*, cioè l'identificativo univoco della richiesta di revoca;
- *delegationId*, cioè l'identificativo della delega da revocare;
- *revokedBy*, cioè il DID del soggetto o dell'organizzazione che richiede la revoca;
- *revocationType*, cioè la tipologia di revoca, ad esempio volontaria, automatica per scadenza o conseguente a violazione della policy;
- *revocationReason*, cioè la motivazione sintetica della revoca;
- *timestamp*, cioè il momento in cui la richiesta viene generata.

La richiesta di revoca viene firmata dal soggetto che la presenta. La firma consente di garantire l'autenticità, l'integrità e il non ripudio della richiesta, assicurando che essa provenga effettivamente dal soggetto indicato e che non sia stata alterata.

Quando il **verifier** interno dell'organizzazione riceve la richiesta, esegue innanzitutto una verifica tecnica del DID del richiedente. A tale scopo, risolve il campo *revokedBy* tramite l'**Identity Registry**, recupera da IPFS il relativo **DID Document** utilizzando il CID registrato sul ledger e verifica che il DID esista, sia attivo e che il DID Document sia correttamente recuperabile.

Il verifier controlla inoltre che il DID Document ottenuto corrisponda al riferimento registrato nell'**Identity Registry**, che la chiave utilizzata per firmare la richiesta sia presente nel DID Document e che la firma apposta sulla richiesta sia valida. Solo dopo l'esito positivo di questi controlli il richiedente può essere considerato tecnicamente autenticato ai fini del flusso di revoca.

Verifica del ruolo del richiedente

Se la revoca è richiesta dal paziente, il sistema non richiede la presentazione di una **Verifiable Credential** di ruolo. In questo caso, la legittimazione deriva direttamente dalla titolarità del documento:

il DID del soggetto che firma la richiesta deve coincidere con il *PatientDID* associato al documento oggetto della delega.

Se invece la revoca è richiesta da un utente operativo o da un rappresentante di un'organizzazione, la sola autenticazione tecnica del *DID* non è sufficiente. Il sistema richiede quindi anche la presentazione di una **Verifiable Presentation** contenente una **Verifiable Credential** idonea a dimostrare il ruolo e gli attributi necessari per richiedere la revoca.

Anche in questo caso viene applicato il pattern già definito nei flussi precedenti: **Presentation Requirement → Verifiable Presentation → verifica della VP → verifica della VC → estrazione degli attributi rilevanti**.

Recupero della delega da revocare e del documento associato

Il **verifier** interroga il **Delegation Registry** utilizzando il *delegationId* indicato nella richiesta. Dal registro vengono recuperati i dati della delega originaria, necessari a verificare che la delega esista e a ricostruirne il contesto autorizzativo.

Se non viene trovata alcuna delega corrispondente, il flusso viene interrotto e l'evento viene registrato nell'**Audit Registry**. Se invece la delega esiste, il sistema controlla che non sia già presente un record di revoca associato allo stesso *DelegationID*.

Nel caso in cui la delega risulti già revocata, il sistema non crea una revoca duplicata. Può invece restituire un esito di tipo *alreadyRevoked* e registrare il tentativo nell'**Audit Registry**.

Durante questa fase non viene recuperato il contenuto clinico del documento. Ai fini della revoca sono sufficienti i metadati necessari a identificare il paziente titolare, l'organizzazione competente, la delega interessata e lo stato del documento associato.

Valutazione autorizzativa della revoca

Il **Policy Engine** valuta quindi se il richiedente abbia effettivamente il diritto di revocare la delega. La valutazione tiene conto del DID del richiedente, del suo ruolo e dei suoi attributi verificati, qualora si tratti di un utente operativo o di un'organizzazione, e dei dati relativi alla delega, tra cui il *DelegationID*, il *DelegatorDID*, il *DelegateDID*, il *TargetDocumentID*, il *PatientDID* e l'organizzazione competente. Vengono inoltre considerati il tipo di revoca richiesto, la motivazione dichiarata, le policy di governance, le policy patient-driven applicabili e le eventuali policy globali relative alla delegabilità e alla revoca.

La revoca viene autorizzata se almeno una delle condizioni previste dal sistema risulta soddisfatta. Nel caso in cui la richiesta provenga dal paziente, il controllo principale consiste nel verificare che il *DID* firmatario coincida con il *PatientDID* associato al documento nel **Document Lifecycle Registry**. Poiché la delega incide sui dati clinici del paziente, quest'ultimo può revocare il consenso precedentemente concesso, nei limiti stabiliti dalle policy di sistema.

Se invece la revoca è richiesta dall'organizzazione competente, il **Policy Engine** verifica che tale organizzazione sia effettivamente collegata al documento o alla delega, ad esempio perché indicata come *IssuerOrganizationDID*.

Nel caso di un utente operativo, il sistema controlla che il soggetto disponga di una base autorizzativa idonea a richiedere la revoca, ad esempio perché coincide con il delegante originario o perché possiede un ruolo e attributi compatibili con le policy applicabili.

Costruzione del record di revoca

Se la verifica autorizzativa ha esito positivo, il sistema costruisce un record di revoca da inserire nel **Delegation Registry**. Tale record consente di rendere la delega non più spendibile ai fini autorizzativi, senza eliminare il record originario dal ledger.

Il record di revoca contiene le informazioni necessarie a identificare in modo univoco l'evento di revoca e a collegarlo alla delega interessata. In particolare, include il *RevocationId*, cioè l'identificativo univoco della revoca, il *DelegationId*, che indica la delega a cui la revoca si riferisce, e il *RevocationType*, che specifica la tipologia di revoca, ad esempio volontaria, automatica per scadenza oppure conseguente a una violazione.

Prima del commit, i peer della rete permissioned verificano la correttezza tecnica e applicativa della transazione. Se i controlli hanno esito positivo, la transazione viene ordinata, validata e inserita nel ledger.

Effetto della revoca sulla delega

Una volta registrato il record di revoca, la delega originaria risulta effettivamente nello stato *revoked*. Questo stato può essere considerato uno stato derivato, poiché il record originario della delega resta immutato, mentre la sua non validità deriva dalla presenza di un record di revoca ad essa collegato.

Da questo momento, ogni richiesta di accesso o di operazione che tenti di utilizzare quella delega come base autorizzativa deve essere respinta dal **Policy Engine**. Per questo motivo, il sistema deve verificare sempre non solo che la delega esista, ma anche che non siano presenti record di revoca associati al relativo *DelegationID*.

Revoca di deleghe derivate

Un aspetto particolarmente importante riguarda le catene di delega. Se la delega revocata ha generato deleghe successive, cioè deleghe figlie che indicano la delega revocata come *ParentDelegationRef*, anche queste ultime non possono più essere considerate valide come base autorizzativa. La revoca della delega sorgente produce quindi effetti sull'intera catena di deleghe che da essa deriva.

Nel sistema proposto, tale effetto viene gestito tramite un controllo dinamico in fase di accesso. Quando un soggetto tenta di utilizzare una delega derivata, il **Policy Engine** risale la catena attraverso il campo *ParentDelegationRef* e verifica che ogni delega precedente sia ancora valida, non scaduta e non revocata.

Se anche una sola delega precedente risulta revocata, la delega derivata viene considerata non spendibile ai fini autorizzativi. In questo caso, la richiesta viene respinta e lo stato effettivo della delega derivata viene aggiornato da *active* a *revoked*, così da riflettere la perdita della base autorizzativa da cui dipendeva.

Revoca di una Verifiable Credential

La revoca di una Verifiable Credential avviene quando l'issuer stabilisce che una credenziale precedentemente emessa non debba più essere accettata come fonte valida di attributi. Una credenziale revocata può continuare a **risultare crittograficamente verificabile**, perché la firma dell'issuer sulla VC originaria può rimanere valida. Tuttavia, **non deve più essere accettata nelle decisioni autorizzative**, poiché il suo stato corrente nel Credential Status Registry non è più *Active*.

Avvio della richiesta di revoca

Il flusso inizia quando l'issuer della credenziale rileva la necessità di modificare lo stato di una Verifiable Credential. L'attore che richiede la revoca genera una **Credential Revocation Request**, contenente almeno:

- *revocationRequestId*, cioè l'identificativo univoco della richiesta di revoca;
- *credentialId*, cioè l'identificativo della credenziale da revocare;
- *requestByDID*, cioè il DID dell'organizzazione che richiede la revoca;
- *effectiveFrom*, cioè il momento a partire dal quale il nuovo stato deve produrre effetti;
- *reasonCode*, cioè una motivazione sintetica e minimizzata della revoca;
- *timestamp*, cioè il momento di generazione della richiesta.

La richiesta viene firmata dal soggetto revocante con una chiave associata al proprio DID che garantisce autenticità, integrità e non ripudio della richiesta.

Verifica del soggetto revocante

Quando il sistema riceve la richiesta di revoca, il componente incaricato della gestione delle credenziali verifica innanzitutto il DID del soggetto revocante. A tale scopo, risolve il *requestByDID* tramite l'**Identity Registry** e recupera il relativo **DID Document** dallo storage off-chain, utilizzando il CID registrato sul ledger.

Il sistema controlla quindi che il DID sia presente nell'**Identity Registry**, che risulti nello stato *Active*, che la chiave utilizzata per firmare la richiesta sia presente nel **DID Document** e sia associata alla corretta relazione di verifica. Infine, verifica che la firma della richiesta sia valida.

Se uno di questi controlli fallisce, la richiesta di revoca viene respinta. L'evento viene comunque registrato nell'**Audit Registry** come tentativo non valido di aggiornamento dello stato di una credenziale. Se invece la verifica tecnica ha esito positivo, il soggetto revocante viene considerato autenticato, ma non ancora autorizzato a modificare lo stato della credenziale.

Dopo aver verificato l'identità tecnica del soggetto revocante, il sistema deve stabilire se tale soggetto sia effettivamente autorizzato a revocare la credenziale indicata. In particolare, deve verificare che il richiedente coincida con l'organizzazione issuer della **Verifiable Credential**.

Per questo motivo, il sistema interroga il **Credential Status Registry** utilizzando il *credentialId* e recupera il record associato alla VC, all'interno del quale è indicato l'*issuerDID*. Se l'*issuerDID* coincide con il *requestByDID*, il soggetto richiedente corrisponde all'issuer originario della credenziale e può quindi procedere alla revoca.

Se invece la richiesta proviene da un soggetto diverso dall'issuer originario, la revoca viene respinta e l'esito viene registrato nell'**Audit Registry**.

Verifica dello stato corrente della credenziale

Prima di aggiornare il **Credential Status Registry**, il sistema verifica lo stato corrente della credenziale, così da evitare aggiornamenti incoerenti o duplicati.

Se la credenziale si trova nello stato *Active*, può essere revocata oppure sospesa, in base al tipo di operazione richiesta. Se invece risulta già nello stato *Revoked*, il sistema non modifica nuovamente il suo stato, ma registra la richiesta come evento di audit, così da mantenere traccia del tentativo. Nel caso in cui la credenziale sia nello stato *Suspended*, il sistema può invece aggiornarla a *Revoked*, rendendo la revoca definitiva ai fini delle successive verifiche.

Costruzione del record

Se tutti i controlli hanno esito positivo, il componente incaricato costruisce un record nel **Credential Status Registry** per l'aggiornamento.

Viene creato il nuovo record di stato e l'aggiornamento viene inviato alla **DLT** sotto forma di transazione destinata al **Credential Status Registry**. La transazione è firmata dal soggetto autorizzato alla modifica dello stato della credenziale e viene sottoposta ai controlli previsti dalla rete permissioned. Se tali controlli hanno esito positivo, la transazione viene ordinata, validata e inserita nel ledger e, da quel momento, il nuovo stato della credenziale diventa parte dello stato condiviso e verificabile del sistema.

La credenziale non viene eliminata dal wallet dell'holder, principalmente per ragioni di auditing e tracciabilità. Tuttavia, se il suo stato non è Active, essa non può più essere utilizzata come fonte valida di attributi nei successivi processi di verifica.

Infine, l'intero flusso di revoca viene registrato nell'**Audit Registry**. In questo modo, un auditor autorizzato può ricostruire chi ha disposto la revoca, quale credenziale è stata coinvolta, quale nuovo stato è stato applicato, quando la revoca è diventata efficace e quale base autorizzativa ha giustificato l'operazione.

Casi di revoca specifici

Compromissione della chiave dell'holder

Se la revoca deriva dalla compromissione della chiave dell'holder, l'aggiornamento del **Credential Status Registry** non è sufficiente. In questo caso, infatti, il sistema deve intervenire anche sull'identità dell'holder, così da ripristinare un assetto crittografico sicuro.

Viene quindi eseguita la rotazione delle chiavi nel **DID Document** dell'holder. Il nuovo DID Document viene pubblicato su IPFS e il relativo CID viene aggiornato nell'**Identity Registry**, in modo che le successive risoluzioni del DID restituiscano la versione corretta e aggiornata del documento.

Il DID Document viene inoltre versionato, così da mantenere traccia dell'evoluzione dell'identità nel tempo e consentire eventuali verifiche retrospettive. Una volta completata la rotazione, può essere emessa una nuova **Verifiable Credential** associata al nuovo stato dell'identità e alle nuove chiavi dell'holder.

Issuer non più autorizzato

Se la revoca della singola credenziale dipende dal fatto che l'issuer non è più autorizzato a emettere credenziali di una determinata tipologia, o credenziali in generale, la governance deve intervenire anche sul **Trust Registry**. Questo può accadere, ad esempio, quando l'issuer abbia tenuto un comportamento ritenuto scorretto o non più compatibile con i requisiti di fiducia previsti dal sistema.

In questo caso, oltre alla revoca della credenziale interessata, viene aggiornata l'autorizzazione dell'issuer nel **Trust Registry**, rimuovendo o limitando la sua capacità di emettere credenziali.

Da quel momento, durante la verifica di una **Verifiable Credential**, il verifier non considera più valide le credenziali emesse da quell'issuer per le tipologie non autorizzate. Ciò vale anche se la firma crittografica della credenziale risulta formalmente corretta, poiché la validità tecnica della firma non è sufficiente in assenza di una legittimazione dell'issuer nel **Trust Registry**.

Creazione delle policy di governance e di accesso globale

Il flusso di creazione di una **policy di governance** e quello relativo a una **policy di accesso globale** seguono lo stesso schema generale. L'unica differenza, come già evidenziato, riguarda il quorum

necessario per rendere effettiva la policy proposta: nel caso delle policy di governance è richiesta una **supermajority**, mentre per le policy di accesso globale è sufficiente la **maggioranza**.

La creazione o la modifica di queste policy avviene quindi attraverso un processo **multi-authority**, nel quale più organizzazioni partecipano alla valutazione e all'approvazione della proposta.

Il flusso inizia quando un'organizzazione autorizzata propone una nuova policy, o la modifica di una policy esistente. Il contenuto completo della policy viene salvato off-chain tramite IPFS e alla proposta viene associato il relativo CID, necessario per recuperare il contenuto integrale.

La policy viene inoltre firmata dall'organizzazione proponente. In questo modo, le altre organizzazioni coinvolte nel processo di governance possono verificare che il contenuto recuperato tramite il CID corrisponda effettivamente alla policy proposta e che non sia stato alterato dopo la sua presentazione.

Proposta della policy da parte dell'organizzazione

L'organizzazione proponente costruisce una **Policy Proposal Request**, contenente almeno:

- *proposalId*, cioè l'identificativo univoco della proposta;
- *policyId*, cioè l'identificativo della policy (rimane lo stesso della versione precedente in caso di nuova versione della policy);
- *policyType*, valorizzato a *governance* oppure *globalAccess*;
- *policyVersion*, cioè la versione proposta;
- *previousPolicyVersion*, se si tratta di aggiornamento;
- *proposedByDID*, cioè il DID dell'organizzazione proponente;
- *CID*, identificatore del contenuto per recuperare il contenuto integrale della policy proposta mediante IPFS.

La proposta viene **firmata dall'organizzazione proponente** tramite una chiave associata al proprio DID.

Verifica dell'identità dell'organizzazione proponente

Quando la proposta viene ricevuta dal sistema, viene innanzitutto verificata l'identità tecnica dell'organizzazione proponente. A tale scopo, il sistema risolve il *proposedByDID* tramite l'**Identity Registry** e recupera da IPFS il relativo **DID Document**, utilizzando il CID registrato sul ledger.

Una volta ottenuto il DID Document, il sistema controlla che il DID sia nello stato Active, che la chiave utilizzata per firmare la proposta sia presente nel documento e sia associata alla corretta relazione di verifica, e che la firma apposta sulla proposta sia valida.

Solo dopo l'esito positivo di questi controlli, l'organizzazione proponente può essere considerata tecnicamente autenticata ai fini del processo di governance.

Votazione della policy

Ogni organizzazione votante recupera il contenuto della policy tramite il CID indicato nella proposta e ne verifica la correttezza prima di esprimere il proprio voto. In particolare, controlla che il contenuto non sia stato alterato, che la firma dell'organizzazione proponente sia valida e che la versione proposta sia coerente con lo stato attuale della policy o con le regole di versionamento previste dal sistema.

Dopo queste verifiche, ciascuna organizzazione avente diritto, secondo quanto stabilito dalle policy di governance già definite, può esprimere un voto favorevole o contrario sulla proposta.

Ogni voto contiene:

- *voteId*, ovvero l'identificativo univoco del voto;
- *proposalId*, ovvero l'identificativo della proposta fatta in precedenza;
- *policyId*, cioè l'identificativo della policy proposta;
- *policyVersion*;
- *voterDID*, ovvero il DID dell'organizzazione votante;
- *voteValue*, ad esempio *approve* oppure *reject*;
- *timestamp* di quando viene fornito il voto.

Il voto viene firmato dall'organizzazione votante utilizzando una chiave associata al proprio DID. Per ogni voto ricevuto, il sistema esegue una serie di controlli finalizzati a garantirne la validità e a impedire votazioni non autorizzate o duplicate.

In particolare, verifica che il *voterDID* sia valido e attivo, che l'organizzazione votante sia autorizzata a partecipare a quello specifico processo decisionale e che il voto sia effettivamente riferito alla proposta corretta. Il sistema controlla inoltre la validità della firma apposta sul voto e accerta che la stessa organizzazione non abbia già espresso un voto per la medesima proposta.

Solo se tutti questi controlli hanno esito positivo, il voto viene considerato valido e può essere conteggiato ai fini del raggiungimento del quorum previsto.

Finalizzazione della proposta

Quando viene raggiunto il quorum richiesto, la proposta viene finalizzata. La finalizzazione consiste nel trasformare una proposta approvata in una decisione formalizzata, pronta per essere registrata sul ledger. A tal fine, il sistema costruisce un oggetto di finalizzazione contenente le informazioni relative alla proposta, alla policy interessata e all'esito della votazione.

Se la proposta viene respinta, la policy non viene registrata come attiva nel registro competente, ovvero il **Global Policy Registry**. Il contenuto salvato su IPFS viene rimosso localmente dall'organizzazione proponente, poiché non produce effetti sullo stato condiviso del sistema.

Se invece la proposta viene approvata, il Document Manager invia una transazione alla **DLT** per registrare la policy nel registro previsto. Il record contiene i campi definiti in precedenza e consente di rendere la policy parte dello stato verificabile della rete.

Prima del commit, i peer della rete permissioned verificano che la proposta sia stata effettivamente approvata, che il quorum richiesto sia stato raggiunto e che i voti espressi siano validi. Controllano inoltre che la policy sia firmata correttamente, che il CID sia presente, che la versione proposta sia coerente, che l'eventuale policy precedente esista e che la transizione di stato sia ammessa.

Se tutti i controlli hanno esito positivo, la transazione viene ordinata, validata e inserita nel ledger. Da quel momento, la policy diventa parte dello stato condiviso e verificabile del sistema.

Pinning e replica della policy

Dopo la registrazione della policy nel **Global Policy Registry**, il **Pinning Manager** dell'organizzazione proponente effettua il pinning del contenuto sul proprio nodo IPFS.

Successivamente, le altre organizzazioni recuperano la policy tramite il CID registrato sul ledger, la memorizzano sui rispettivi nodi IPFS e provvedono a loro volta al pinning del contenuto tramite i propri **Pinning Manager**.

In questo modo, la policy approvata rimane disponibile in maniera distribuita e non dipende esclusivamente dal nodo IPFS dell'organizzazione proponente.

Gestione della versione precedente

Il flusso descritto si applica anche all'aggiornamento di una policy di governance o di una policy di accesso globale già esistente. In questo caso, la versione precedente della policy non viene eliminata. Il sistema provvede invece ad aggiornarne lo stato, impostandola come *revoked* e registrando il motivo della revoca insieme alle informazioni correlate.

Inoltre, ogni azione rilevante sul ciclo di vita di una policy genera un evento nell'**Audit Registry**, così da assicurare tracciabilità e verificabilità dell'intero processo. In particolare, se la policy non viene approvata, il contenuto viene rimosso dallo storage IPFS dell'organizzazione proponente, ma negli audit verrà comunque inserito il contenuto della policy oltre alle altre informazioni sulla votazione.

Creazione di una policy patient-driven

Il flusso relativo alle **policy patient-driven** presenta alcune differenze rispetto a quello previsto per le policy di accesso globale e per le policy di governance. Le policy patient-driven, infatti, non derivano da un processo decisionale multi-authority, ma esprimono direttamente la **volontà del paziente**. Per questo motivo, non è necessario il raggiungimento di alcun quorum.

Il flusso ha inizio quando il paziente definisce una nuova policy individuale oppure modifica una policy patient-driven già esistente.

Invio della richiesta del paziente

Il paziente genera una **Patient-Driven Policy Request**, contenente:

- *policyRequestId*, ovvero l'id della richiesta;
- *patientDID*, il DID del paziente;
- *policyId*, l'id della policy che si vuole inserire;
- *policyVersion*, ovvero la versione della policy;
- *validFrom*;
- *validUntil*;
- *previousPolicyVersion*, se si tratta di modifica;
- *timestamp*;
- contenuto della policy.

La richiesta viene firmata dal paziente tramite una chiave associata al proprio DID; di quest'ultimo viene effettuata la verifica come descritto già per il flusso delle policy di governance e di accesso globale. La richiesta viene inviata al Policy Engine dell'organizzazione di riferimento cifrata con la chiave di *keyAgreement*.

Verifica della corrispondenza tra paziente richiedente e soggetto del documento

Se la policy riguarda uno specifico documento, il sistema interroga il **Document Lifecycle Registry** utilizzando il *targetDocumentID*. In questo modo recupera i metadati necessari a verificare l'associazione tra il documento e il paziente.

Il controllo principale consiste nel verificare che il *patientDID* firmatario della richiesta coincida con il *patientDID* associato al documento. Se non vi è corrispondenza, la richiesta viene respinta, poiché il paziente non risulta legittimato a definire o modificare una policy relativa a quel documento.

Verifica di compatibilità con le policy globali

Prima di registrare una **policy patient-driven**, il sistema deve verificare che essa non ampli il perimetro dei permessi già definiti dalle policy globali. La volontà del paziente può infatti introdurre restrizioni,

limitazioni o consensi specifici, ma non può autorizzare azioni, soggetti o finalità che il sistema non considera ammissibili a livello globale.

Per questo motivo, il **Policy Engine** consulta il **Global Policy Registry** e confronta la policy proposta dal paziente con le policy globali applicabili. In particolare, verifica che gli utenti autorizzati dal paziente possiedano ruoli e attributi compatibili con le regole globali, che le azioni consentite siano già previste, che la finalità indicata sia ammessa e che la categoria documentale rientri nel perimetro autorizzativo generale.

Il sistema controlla inoltre che il periodo di validità non violi eventuali vincoli generali e che la policy patient-driven non abiliti soggetti, azioni o condizioni non previste dal sistema.

Se la policy proposta si limita a introdurre restrizioni, limitazioni o consensi entro il perimetro definito dalle policy globali, può essere accettata. Se invece tenta di estendere tale perimetro, la richiesta viene respinta.

Salvataggio off-chain della policy patient-driven

Una volta superati i controlli previsti, il contenuto completo della **policy patient-driven** viene conservato off-chain sul nodo IPFS dell'organizzazione di riferimento. Poiché tale policy può contenere informazioni sensibili relative alle preferenze e alle restrizioni definite dal paziente, il contenuto non viene memorizzato in chiaro, ma in forma cifrata.

La policy cifrata viene quindi inviata al nodo IPFS dell'organizzazione dal Document Manager, che restituisce il relativo CID. Tale identificativo consente di riferire il contenuto off-chain senza esporlo direttamente sul ledger. Successivamente, il **Pinning Manager** dell'organizzazione effettua il pinning della policy, così da garantirne la persistenza e impedirne la rimozione durante eventuali operazioni di pulizia locale.

Registrazione nel Patient-Driven Policy Registry

Il flusso descritto è coerente anche con l'aggiornamento di una policy patient-driven già esistente, con l'aggiunta della modifica dello stato della versione precedente che deve essere revocata.

Dopo il salvataggio off-chain, il sistema costruisce il record da inserire nel **Patient-Driven Policy Registry**, contenente le informazioni già descritte in precedenza. La transazione viene quindi inviata alla **DLT** e sottoposta ai controlli dei peer della rete permissioned.

Prima del commit, i peer verificano che la firma del paziente sia valida, che il DID del paziente sia attivo e che il paziente sia effettivamente associato al documento oggetto della policy. Controllano inoltre che la policy proposta non ampli il perimetro dei permessi globali, che il CID sia presente, che la versione sia coerente, che l'eventuale versione precedente esista e che la transizione di stato sia ammessa.

Se tutti i controlli hanno esito positivo, la transazione viene validata e registrata nel **Patient-Driven Policy Registry**. Da quel momento, la policy diventa parte dello stato condiviso e verificabile del sistema, pur mantenendo il proprio contenuto completo conservato off-chain in forma cifrata.

Il flusso descritto si applica anche all'aggiornamento di una policy patient-driven già esistente. In questo caso, la nuova versione viene registrata nel registro, mentre la versione precedente non viene eliminata, ma viene posta nello stato *revoked*, così da preservare la tracciabilità storica e consentire la ricostruzione delle decisioni autorizzative effettuate nel tempo.

Emissione di una Verifiable Credential

Il flusso di emissione di una **Verifiable Credential** riguarda il caso in cui un'organizzazione riconosciuta dalla governance della rete rilasci a un utente operativo una credenziale attestante ruoli, attributi professionali, appartenenza organizzativa o ulteriori qualifiche necessarie per interagire con il sistema.

L'emissione della VC è effettuata da un issuer riconosciuto, come ad esempio un'organizzazione sanitaria appartenente alla rete permissioned, un ospedale, un laboratorio o un ente autorizzato ad attestare specifiche qualifiche professionali.

La sola partecipazione di un'organizzazione alla rete, tuttavia, non implica automaticamente la possibilità di emettere qualsiasi tipo di credenziale. Prima dell'emissione, il sistema deve verificare che l'issuer sia effettivamente autorizzato, tramite il **Trust Registry**, a rilasciare quella specifica tipologia di **Verifiable Credential**.

Avvio della richiesta di emissione

Il flusso inizia quando un soggetto operativo, ad esempio un medico, un infermiere, un tecnico di laboratorio o un auditor, richiede all'organizzazione di appartenenza l'emissione di una Verifiable Credential. Il soggetto deve già possedere un DID attivo registrato nell'Identity Registry. La richiesta di emissione può essere rappresentata come una **Credential Issuance Request**, contenente almeno:

- *issuanceRequestId*, cioè l'identificativo univoco della richiesta;
- *holderDID*, cioè il DID del soggetto che riceverà la credenziale;
- *issuerDID*, cioè il DID dell'organizzazione chiamata a emettere la credenziale;
- *requestedCredentialType*, cioè la tipologia di credenziale richiesta;
- *requestedAttributes*, cioè gli attributi che dovrebbero essere attestati, ad esempio ruolo, reparto o specializzazione;
- *timestamp*, cioè il momento di generazione della richiesta;
- eventuali riferimenti interni alla relazione tra soggetto e organizzazione, ad esempio rapporto di lavoro, incarico, reparto o qualifica.

La richiesta viene firmata dal soggetto richiedente utilizzando una chiave associata al proprio DID. La firma consente di dimostrare che la richiesta proviene effettivamente dal controller del holderDID, ma non è ancora sufficiente per stabilire che gli attributi richiesti debbano essere attestati tramite una **Verifiable Credential**.

Quando l'organizzazione riceve la richiesta, verifica innanzitutto l'identità tecnica del soggetto richiedente. A tale scopo, il sistema risolve il *holderDID* tramite l'**Identity Registry**, recupera il CID del relativo **DID Document**, ottiene il documento da IPFS e ne verifica la coerenza con il riferimento registrato sul ledger.

Il sistema controlla quindi che il DID sia presente nell'**Identity Registry**, che risulti nello stato *active*, che la chiave utilizzata per firmare la richiesta sia presente nel **DID Document** e che sia abilitata alla corretta relazione di verifica, ad esempio *authentication*. Infine, verifica la validità della firma apposta sulla richiesta.

Se questi controlli hanno esito positivo, il soggetto richiedente viene considerato tecnicamente autenticato. Tuttavia, questa verifica non comporta ancora l'emissione della credenziale, ma consente soltanto di proseguire con le successive valutazioni relative agli attributi richiesti e all'autorizzazione dell'issuer.

Verifica interna degli attributi da attestare

Dopo la verifica tecnica del DID, l'organizzazione deve stabilire se gli attributi richiesti possano effettivamente essere attestati. Questa verifica non avviene tramite il ledger, ma attraverso i sistemi interni e le procedure organizzative dell'issuer.

Ad esempio, l'organizzazione può controllare che il soggetto sia effettivamente un proprio dipendente, collaboratore o professionista abilitato, che il ruolo dichiarato sia corretto e che il reparto indicato corrisponda alla sua posizione organizzativa. Può inoltre verificare che la specializzazione professionale sia adeguatamente documentata, che eventuali vincoli temporali, funzionali o organizzativi siano corretti e che il soggetto non risulti sospeso internamente o privo della qualifica richiesta.

Questa fase è essenziale perché la **Verifiable Credential** trasferisce nel sistema decentralizzato un'attestazione prodotta dall'organizzazione. L'issuer, quindi, si assume la responsabilità degli attributi che decide di firmare e rendere verificabili all'interno della rete.

Costruzione della Verifiable Credential

Se i controlli precedenti hanno esito positivo, l'issuer procede alla costruzione della **Verifiable Credential**.

La credenziale deve contenere esclusivamente gli attributi necessari rispetto allo scopo per cui viene emessa, nel rispetto del principio di minimizzazione dei dati. Non devono quindi essere inclusi dati clinici, informazioni relative ai pazienti o elementi non necessari alla verifica del ruolo, dell'appartenenza organizzativa e degli attributi professionali del soggetto.

Preparazione della credenziale per selective disclosure

Nel modello proposto, la **Verifiable Credential** può essere emessa in una forma compatibile con **SD-JWT**, così da consentire all'holder di rivelare selettivamente solo gli attributi necessari durante una futura **Verifiable Presentation**.

Per ciascun attributo rivelabile in modo selettivo, l'issuer genera una disclosure composta da un salt casuale, dal nome del campo e dal relativo valore. Successivamente, calcola il digest crittografico di ciascuna disclosure.

I digest vengono inseriti nella VC firmata dall'issuer, mentre le disclosure complete vengono consegnate all'holder insieme alla credenziale. In questo modo, quando l'holder utilizzerà la VC, potrà presentare al verifier soltanto le disclosure necessarie per la specifica richiesta.

Il verifier ricalcolerà i digest delle disclosure ricevute e controllerà che siano presenti tra quelli firmati dall'issuer. Così facendo, potrà verificare la correttezza degli attributi rivelati senza venire a conoscenza degli attributi non presentati.

Firma della Verifiable Credential

Dopo aver costruito il payload della credenziale, l'issuer firma la **Verifiable Credential** utilizzando una chiave privata associata al proprio DID e abilitata alla relazione di verifica *assertionMethod*.

La firma dell'issuer garantisce l'autenticità della credenziale, l'integrità degli attributi attestati, il non ripudio dell'emissione e la possibilità per soggetti diversi dall'issuer di verificarne la validità crittografica. Tuttavia, la firma non è sufficiente, da sola, a rendere la credenziale automaticamente accettabile nelle future decisioni autorizzative. Ogni verifier dovrà infatti controllare anche il **Trust Registry**, per verificare la legittimazione dell'issuer, e il **Credential Status Registry**, per accertare lo stato corrente della credenziale.

Dopo la firma della VC, l'issuer inizializza lo stato della credenziale nel **Credential Status Registry**. Sul ledger non viene pubblicato il contenuto completo della credenziale, ma soltanto una entry minimizzata e verificabile, necessaria a supportare i futuri controlli di validità.

Il record contiene almeno il *credentialId*, cioè l'identificativo della VC, l'issuerDID, che identifica l'organizzazione emittente, lo status, inizialmente valorizzato ad *active*, e la *version*, inizialmente pari a 1. Include inoltre il *createdAt*, corrispondente al timestamp di creazione del record, l'*updatedAt*, inizialmente coincidente con il momento dell'emissione, l'*updatedBy*, valorizzato con il DID dell'issuer, e la *proof*, ossia la firma dell'issuer sull'aggiornamento di stato.

La transazione di inizializzazione viene quindi inviata alla **DLT permissioned**. I peer della rete verificano la correttezza tecnica della transazione, la firma dell'issuer e la coerenza dell'operazione con le regole del sistema. Dopo il commit, lo stato della credenziale diventa parte dello stato condiviso e verificabile della rete.

Consegna della credenziale all'holder

Una volta firmata la **Verifiable Credential** e registrato il relativo stato iniziale, la credenziale viene consegnata all'holder, che la conserva nel proprio wallet o nel proprio credential repository.

Nel caso di una credenziale basata su **SD-JWT**, il wallet conserva l'Issuer-signed JWT, le disclosure associate agli attributi che possono essere rivelati selettivamente e gli eventuali metadati necessari per la futura presentazione della credenziale.

La VC non viene pubblicata in chiaro sul ledger e non viene resa accessibile in modo indiscriminato alle altre organizzazioni. Sarà l'holder a presentarla quando dovrà dimostrare il possesso di un ruolo, di un attributo professionale o di una qualifica nell'ambito di una specifica richiesta operativa.

L'issuer può inoltre conservare nei propri sistemi interni un registro delle credenziali emesse, utile per finalità di audit, revoca, sospensione o gestione del ciclo di vita della credenziale. Questo registro interno non sostituisce il **Credential Status Registry** condiviso, ma lo affianca con informazioni amministrative più dettagliate, mantenute sotto la responsabilità dell'organizzazione emittente.

Registrazione di un DID e del DID Document associato

Il flusso di registrazione di un DID riguarda il processo attraverso cui un soggetto del sistema, sia esso un utente operativo, un'organizzazione o un paziente, rende utilizzabile il proprio identificatore decentralizzato all'interno della rete.

Enrollment del paziente

Un caso particolare da gestire è quello del paziente che non dispone di una **Verifiable Credential** di ruolo. Per rendere il modello affidabile, è necessaria una fase iniziale di **enrollment**, durante la quale l'organizzazione sanitaria di riferimento verifica l'identità reale del paziente e la associa al DID che quest'ultimo utilizzerà nel sistema.

L'enrollment rappresenta quindi la fase in cui viene stabilito il legame iniziale tra il paziente reale e il patientDID, che sarà successivamente utilizzato nei documenti clinici e nelle operazioni firmate dal paziente.

Durante questa fase, il paziente presenta il proprio DID e il relativo **DID Document**. Successivamente deve dimostrare di controllare la chiave privata associata a quel DID. A tal fine, l'organizzazione sanitaria genera una challenge contenente un *nonce*, il DID dell'organizzazione, il *patientDID* e un *timestamp*. Il paziente firma la challenge con la propria chiave privata e la restituisce all'organizzazione.

L'organizzazione verifica la firma utilizzando il **DID Document** presentato nella fase iniziale di registrazione. Se la firma è valida, il paziente dimostra di controllare effettivamente la chiave privata associata al DID; in caso contrario, il DID non viene accettato ai fini dell'enrollment.

Dopo aver verificato sia l'identità reale del paziente sia il controllo crittografico del DID, l'organizzazione sanitaria registra internamente l'associazione tra il paziente reale e il *patientDID*.

Creazione del DID e del DID Document associato

Ad eccezione fatta per la fase di enrollment, tutte le fasi di registrazione del DID e del relativo DID Document descritte da questo punto in poi saranno comuni per tutti gli utenti del sistema.

Seguendo il modello della Self-Sovereign Identity, il soggetto genera il proprio materiale crittografico, ovvero tre chiavi distinte per le principali relazioni di verifica previste dal DID Document: una chiave per *authentication*, usata per dimostrare il controllo del DID e firmare richieste di autenticazione; una chiave per *assertionMethod*, usata dai soggetti autorizzati a firmare asserzioni o Verifiable Credentials; una chiave per *keyAgreement*, usata per lo scambio di chiavi o per ricevere contenuti cifrati.

Le chiavi private rimangono sotto il controllo del soggetto o del wallet mentre le chiavi pubbliche vengono inserite nel DID Document, insieme alle altre informazioni tecniche in esso contenute precedentemente descritte.

Prima della pubblicazione, il DID Document viene firmato dal soggetto controller del DID; questa firma serve a dimostrare che il soggetto che richiede la registrazione controlla effettivamente le chiavi private corrispondenti alle chiavi pubbliche inserite nel DID Document.

Storage del DID Document

Il DID Document completo non viene salvato sul ledger, ma viene pubblicato sul nodo IPFS dell'organizzazione competente. Il nodo IPFS restituisce il CID del DID Document che verrà poi registrato sull'Identity Registry per consentire a tutte le organizzazioni del sistema di recuperare la versione corretta del DID Document.

Inoltre, il Pinning Manager dell'organizzazione effettua il pinning del DID Document, così da garantirne la persistenza e impedire che venga rimosso durante operazioni di pulizia locale.

Pubblicazione sull'Identity Registry

Dopo la pubblicazione del **DID Document** su IPFS, l'organizzazione competente costruisce una **DID Registration Request** da inviare alla DLT. La richiesta contiene le informazioni necessarie a identificare il DID da registrare e il relativo documento associato.

Prima di procedere con la registrazione nell'**Identity Registry**, il componente applicativo incaricato esegue una serie di controlli preliminari. In particolare, verifica che il DID non sia già presente nel registro, che il **DID Document** sia recuperabile tramite il *CID* indicato e che risulti correttamente formato. Controlla inoltre che il campo *id* contenuto nel DID Document coincida con il DID da registrare e che l'eventuale firma del documento sia valida.

Il sistema verifica anche che il soggetto dimostri il controllo della chiave privata corrispondente alla chiave pubblica indicata nel DID Document. Infine, accerta che il documento non contenga dati personali o sanitari non ammessi, poiché il DID Document deve includere solo le informazioni tecniche necessarie alla risoluzione dell'identità e alla verifica crittografica.

Nel caso di un paziente, il sistema verifica anche che l'enrollment sia stato effettuato da un'organizzazione sanitaria autorizzata. Nel caso di un utente operativo, invece, l'organizzazione verifica

internamente l'esistenza e la correttezza del rapporto con il soggetto, ad esempio in termini di appartenenza, ruolo o qualifica professionale.

Se tutti i controlli hanno esito positivo, viene inviata una transazione alla **DLT** per registrare il nuovo DID nell'**Identity Registry**. Il record registrato contiene almeno il DID, il *DIDDocumentRef*, cioè il CID del **DID Document**, il *documentContentType*, valorizzato a *application/did+json*, lo status, valorizzato ad *Active*, e la *version*, che indica la versione corrente del DID Document. Include inoltre il riferimento alla versione precedente, valorizzato a null nel caso di creazione di un nuovo DID Document, il timestamp di creazione, il timestamp di aggiornamento e l'eventuale timestamp di disattivazione del DID, inizialmente vuoto.

La transazione viene firmata dal soggetto o dall'organizzazione autorizzata e viene sottoposta al normale processo di consenso della **DLT permissioned**. Dopo il commit, il DID diventa risolvibile dagli altri componenti del sistema.

Da quel momento, un verifier può recuperare il **DID Document** tramite il CID registrato, verificare lo stato del DID e utilizzare le chiavi pubbliche contenute nel documento per controllare firme, Verifiable Presentation, consensi, revoche o altre richieste operative.

In caso di rotazione delle chiavi, non è necessario generare un nuovo DID, poiché l'identificativo del soggetto rimane invariato. La rotazione comporta invece la creazione di una nuova versione del **DID Document**, all'interno della quale viene registrata la nuova chiave pubblica.

Questa soluzione consente di mantenere la continuità dell'identità digitale del soggetto e, allo stesso tempo, di ricostruire lo stato storico delle chiavi. In questo modo, il sistema può verificare anche a posteriori quali chiavi fossero valide al momento della firma di una **Verifiable Credential**, di una **Verifiable Presentation** o di un'altra operazione rilevante, garantendo verificabilità ex post e supporto alle attività di audit.

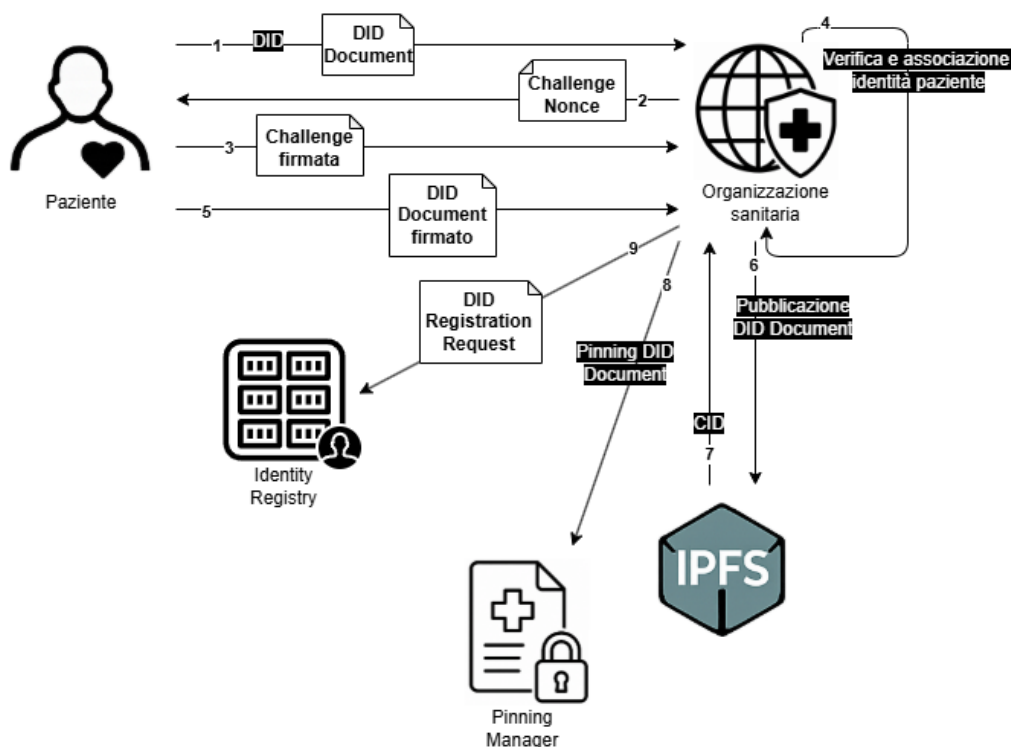


Figura 9, Registrazione di un DID e del DID Document associato

Auditabilità e ricostruzione ex post

Il flusso di auditabilità e ricostruzione ex post riguarda il caso in cui un auditor autorizzato debba ricostruire, dopo il verificarsi di una o più operazioni, la sequenza degli eventi che hanno portato a una determinata decisione o transizione di stato. La ricostruzione ex post può riguardare, ad esempio, un accesso ordinario a un documento, un accesso negato, una creazione o modifica documentale, una delega, una revoca, l'emissione o revoca di una credenziale, la registrazione di una policy.

Il principio fondamentale è che l'audit non si basa su log locali modificabili unilateralmente, ma su eventi ancorati al ledger tramite l'Audit Registry, collegati crittograficamente a batch off-chain cifrati e riferiti agli stati registrati nei vari registry del sistema.

Avvio della richiesta di audit

La richiesta può essere rappresentata come una **Audit Reconstruction Request**, contenente almeno:

- *reconstructionRequestId*, cioè l'identificativo univoco della richiesta di ricostruzione;
- *requesterDID*, cioè il DID dell'auditor che deve effettuare l'audit;
- *requesterOrganizationDID*, se il richiedente agisce per conto di un'organizzazione;
- *targetType*, cioè il tipo di risorsa o evento da ricostruire, ad esempio document, delegation, credential, policy.
- *targetRef*, cioè il riferimento alla risorsa coinvolta, ad esempio *DocumentID*, *DelegationID*, *CredentialID*, *PolicyID* o *DID*;
- *timeWindow*, cioè l'intervallo temporale oggetto della ricostruzione;
- *purposeOfAudit*, cioè la finalità dell'audit, ad esempio verifica ordinaria, indagine su anomalia;
- *requestedEvidenceScope*, cioè il livello di evidenza richiesto, ad esempio solo metadati di audit, batch di audit completi, verifica di policy, verifica di document lifecycle;
- *timestamp*;
- *nonce*, per evitare replay.

La richiesta viene firmata dal soggetto che la presenta, così da garantirne autenticità, integrità e non ripudio.

Verifica del soggetto richiedente

Quando il sistema riceve la richiesta, il **verifier** interno dell'organizzazione competente esegue innanzitutto la verifica tecnica dell'identità del richiedente. A tale scopo, risolve il requesterDID tramite l'**Identity Registry**, recupera da IPFS il relativo **DID Document** utilizzando il CID registrato sul ledger e controlla la validità degli elementi necessari alla verifica.

In particolare, il sistema verifica che il DID sia presente nell'**Identity Registry** e che risulti nello stato Active. Controlla inoltre che la chiave utilizzata per firmare la richiesta sia presente nel **DID Document** e sia abilitata alla corretta relazione di verifica. Successivamente verifica la validità della firma apposta sulla richiesta e accerta che il nonce non sia già stato utilizzato, così da prevenire eventuali tentativi di replay.

Se tutti i controlli hanno esito positivo, il richiedente viene considerato tecnicamente autenticato. Tuttavia, questa verifica non è ancora sufficiente per consentire l'accesso agli elementi di audit, ma permette soltanto di proseguire con la successiva valutazione autorizzativa.

Valutazione autorizzativa della richiesta di audit

Il **Policy Engine** valuta se il richiedente sia autorizzato a eseguire la ricostruzione ex post richiesta. È importante distinguere l'accesso agli elementi di audit dall'accesso ai documenti clinici, poiché l'autorizzazione a ricostruire un evento non implica automaticamente il diritto di leggere il contenuto sanitario in chiaro.

Di norma, l'auditor accede ai metadati, ai riferimenti documentali, alle decisioni autorizzative, agli stati dei registri e agli eventi registrati nell'**Audit Registry**. L'accesso al contenuto clinico cifrato viene autorizzato solo se strettamente necessario e se previsto dalle policy di governance e di accesso globale relative agli audit.

Se la richiesta non è compatibile con le policy, il **Policy Engine** produce una decisione di tipo *deny*, indicando in forma sintetica il motivo del rifiuto. L'esito viene poi registrato nell'**Audit Registry**.

Se invece la richiesta è autorizzata, il **Policy Engine** produce una decisione strutturata di tipo *audit-allow*. Tale decisione contiene l'identificativo della decisione, indicato come *decisionId*, l'identificativo della richiesta originaria, ossia il *reconstructionRequestId*, il *requesterDID* del soggetto richiedente, il periodo di validità della decisione tramite *validFrom* e *validUntil*, e i *policyRefs*, cioè i riferimenti alle policy che hanno giustificato l'esito positivo della valutazione.

Questa decisione non autorizza un accesso generale al sistema, ma consente soltanto la consultazione degli elementi necessari alla ricostruzione ex post, entro il perimetro indicato dalla decisione stessa.

Individuazione dei batch di audit rilevanti

Dopo l'autorizzazione, l'auditor interroga l'**Audit Registry** per individuare i batch di audit pertinenti alla ricostruzione richiesta.

L'**Audit Registry** non contiene il contenuto completo dei log, ma conserva riferimenti verificabili ai relativi batch. Per ciascun batch rilevante vengono quindi recuperati il *batchId* e il relativo CID, che consente di localizzare il contenuto conservato off-chain.

Se la ricostruzione riguarda eventi distribuiti tra più organizzazioni, il sistema può individuare batch conservati su nodi IPFS differenti. In questo modo, l'auditor può ricostruire il quadro degli eventi rilevanti senza che i log completi debbano essere pubblicati direttamente sul ledger.

Recupero e decifrazione dei batch di audit

Una volta individuati i batch rilevanti, il sistema utilizza i CID registrati nell'**Audit Registry** per recuperarli da IPFS. Poiché i batch di audit sono conservati off-chain in forma cifrata, il semplice recupero del payload non è sufficiente per accedere al loro contenuto.

Per consentire la lettura dei batch, il sistema deve quindi attivare il meccanismo di rilascio della relativa chiave di cifratura. Tale rilascio avviene soltanto se la decisione *audit-allow* lo consente, in particolare sulla base del perimetro definito nel campo *requestedEvidenceScope*.

Quando l'accesso alla chiave è autorizzato, quest'ultima viene rilasciata all'auditor in forma cifrata, utilizzando la chiave di *keyAgreement* associata al suo DID. In questo modo, solo l'auditor autorizzato può decifrare la chiave del batch e accedere agli elementi di audit necessari alla ricostruzione richiesta.

Verifica dell'integrità degli eventi nei batch recuperati

Dopo il recupero e la decifrazione dei batch, il sistema verifica la coerenza interna degli eventi contenuti in ciascun batch. Gli eventi, collegati tra loro tramite riferimenti e hash agli eventi precedenti, devono risultare disposti nell'ordine corretto e coerenti con la sequenza registrata.

Il sistema controlla quindi che la catena degli eventi non presenti interruzioni, sostituzioni, omissioni o riordinamenti non giustificati. Questa verifica consente di accertare che il batch non sia stato alterato e che la sequenza degli eventi di audit possa essere utilizzata come evidenza affidabile nella ricostruzione ex post.

Recupero dello stato storico dei registri coinvolti

Dopo aver verificato i batch di audit, il sistema ricostruisce lo stato dei registri rilevanti nel momento in cui è avvenuta l'operazione oggetto di audit. A seconda del tipo di evento da analizzare, possono essere consultati il **Document Lifecycle Registry**, il **Global Policy Registry**, il **Patient-Driven Policy Registry**, il **Delegation Registry**, il **Credential Status Registry**, il **Trust Registry** e l'**Identity Registry**.

La ricostruzione deve avere natura temporale e non può basarsi esclusivamente sullo stato corrente dei registri. Infatti, una condizione oggi non più valida poteva esserlo nel momento in cui l'operazione è stata eseguita. Ad esempio, una credenziale attualmente revocata poteva risultare ancora valida al momento dell'accesso, così come una policy oggi sostituita poteva essere quella applicabile al tempo della decisione autorizzativa.

Per questo motivo, il sistema recupera le versioni storiche applicabili e ricostruisce il contesto esistente al timestamp dell'evento. In particolare, considera la versione della policy indicata nell'evento, la versione del documento coinvolta, lo stato della credenziale al momento della verifica, lo stato della delega al momento del suo utilizzo, lo stato del DID e del relativo **DID Document** al momento della firma, nonché lo stato del **Trust Registry** al momento della verifica dell'issuer.

In questo modo, l'audit non si limita a osservare la situazione attuale del sistema, ma consente di ricostruire con precisione il contesto autorizzativo effettivamente esistente quando l'operazione è stata autorizzata, negata o eseguita.

Produzione del report di ricostruzione

Al termine delle verifiche, il componente di audit produce un **Audit Reconstruction Report**, il cui contenuto deve essere limitato alle sole informazioni necessarie rispetto alla finalità dell'audit, nel rispetto del principio di minimizzazione dei dati.

Il report contiene gli elementi necessari a identificare la ricostruzione effettuata e a renderla verificabile. In particolare, include il *reportId*, cioè l'identificativo del report, il *reconstructionRequestId*, relativo alla richiesta originaria, il DID dell'auditor richiedente, l'ambito della ricostruzione e il periodo analizzato. Riporta inoltre i batch di audit consultati, i registri interrogati, gli eventi ricostruiti, le policy applicabili al momento dell'operazione, lo stato storico delle credenziali, delle deleghe e dei documenti coinvolti, nonché l'esito della ricostruzione ed eventuali anomalie rilevate.

Il report include anche i riferimenti crittografici agli elementi verificati e il timestamp di produzione, così da consentire controlli successivi sull'integrità e sulla correttezza della ricostruzione.

Una volta prodotto, il report viene firmato dall'auditor o dal componente autorizzato che lo genera. Può inoltre essere conservato off-chain in forma cifrata su IPFS; in questo caso, il relativo CID viene registrato nell'**Audit Registry**, rendendo verificabile anche l'esistenza e la produzione del report stesso.

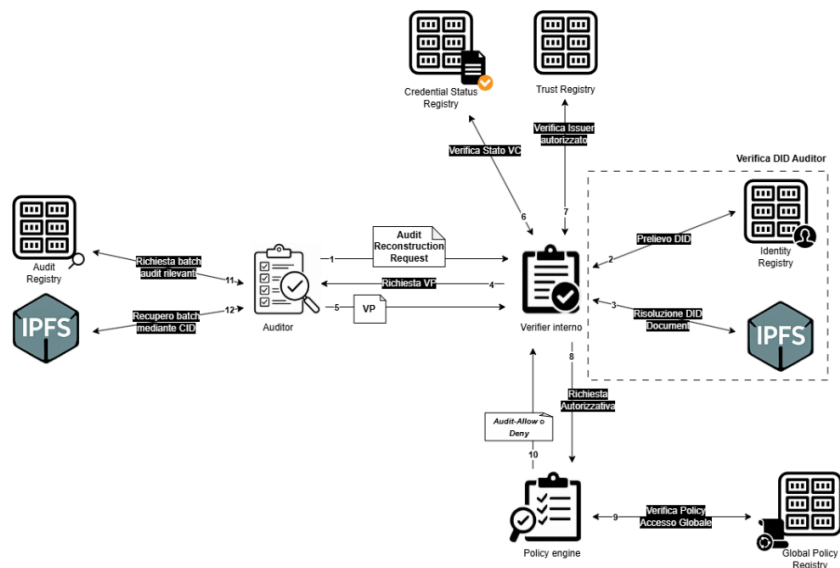


Figura 10, Auditabilità e ricostruzione ex post

Revoca di un documento

Il flusso di revoca di un documento riguarda il caso in cui una versione documentale precedentemente registrata nel **Document Lifecycle Registry** non debba più essere considerata valida per le operazioni ordinarie del sistema. La revoca può riguardare il documento logico nel suo complesso oppure una specifica versione documentale certificata.

È importante precisare che la revoca non comporta la cancellazione fisica del documento dallo storage off-chain né la rimozione del relativo record dal ledger. Il documento e le sue versioni rimangono conservati ai fini di audit, tracciabilità e ricostruzione storica, ma il loro stato cambia in modo tale da impedire che vengano utilizzati come documenti validi nei flussi ordinari di accesso, aggiornamento o certificazione.

La revoca può essere necessaria, ad esempio, quando un documento è stato prodotto per errore o quando contiene informazioni non più attendibili.

Avvio della richiesta di revoca

Il flusso inizia quando un soggetto autorizzato richiede la revoca di un documento o di una sua versione. La richiesta può provenire, a seconda delle policy applicabili, dall'organizzazione che ha emesso o certificato il documento, da un utente operativo con ruolo adeguato, da un'autorità di governance.

La richiesta può essere rappresentata come una **Document Revocation Request**, contenente almeno:

- *revocationRequestId*, cioè l'identificativo univoco della richiesta di revoca;
- *requesterDID*, cioè il DID del soggetto che richiede la revoca;
- *documentId*, cioè l'identificativo logico del documento;
- *targetVersion*, cioè la versione documentale da revocare, specificata solo se si vuole revocare una specifica versione e non il documento logico;
- *patientDID*, cioè il DID del soggetto a cui il documento si riferisce;
- *revocationReason*, cioè una motivazione sintetica e minimizzata;
- *timestamp*;
- *nonce*, per prevenire attacchi di replay.

La richiesta viene firmata dal soggetto che la presenta.

Verifica tecnica del richiedente

Quando il verifier interno dell'organizzazione riceve la richiesta, esegue innanzitutto la verifica tecnica del *requesterDID*.

Il sistema risolve il *DID* tramite l'**Identity Registry** ed effettua tutti i controlli relativi a questo già descritti in precedenza.

Se il richiedente è un utente operativo, il sistema richiede anche una **Verifiable Presentation** contenente una *Verifiable Credential* idonea a dimostrare ruolo, appartenenza organizzativa e attributi necessari per richiedere la revoca. La *VC* viene verificata controllando firma dell'issuer, validità temporale, stato nel **Credential Status Registry** e legittimazione dell'issuer nel **Trust Registry**.

Recupero dello stato documentale corrente

Dopo la verifica tecnica del richiedente, il sistema interroga il **Document Lifecycle Registry** usando *documentId* e, se indicato, *targetVersion*.

Prima di procedere, il sistema verifica che il documento esista, che la versione indicata sia effettivamente registrata, che il documento non sia già nello stato *Revoked* e che la transizione di stato richiesta sia ammessa. Se il documento o la versione risultano inesistenti, già revocati o non compatibili con la richiesta, il flusso viene interrotto e l'evento viene registrato nell'**Audit Registry**.

Valutazione autorizzativa da parte del Policy Engine

Il **Policy Engine** valuta se il richiedente sia autorizzato a revocare il documento. Se la richiesta non soddisfa le condizioni previste, il Policy Engine produce una decisione di *deny*, registrata nell'**Audit Registry** con una motivazione sintetica.

Se invece la richiesta è conforme alle policy, il Policy Engine produce una decisione strutturata di tipo *allow*, contenente almeno *decisionId*, *requesterDID*, *documentId*, *targetVersion*, *policyRefs*, *timestamp* e, infine, la firma dell'organizzazione autorizzata.

Costruzione del record di revoca documentale

Dopo una decisione positiva, il Document Manager costruisce un nuovo record da inserire nel **Document Lifecycle Registry**.

Il record di revoca contiene almeno l'identificativo univoco dell'evento di revoca, ovvero il *DocumentRevocationID*, l'identificativo del documento logico coinvolto, cioè *DocumentID*, e la versione revocata, indicata come *TargetVersion*.

Nel record devono essere indicati anche il *DID* del soggetto o dell'organizzazione che ha disposto la revoca, il motivo della revoca, i riferimenti alle policy che la giustificano e il riferimento alla decisione del **Policy Engine**. Infine, il record deve contenere la data e l'ora della revoca, indicate con *RevokedAt*, e la *Signature*, cioè la firma che garantisce l'autenticità e l'integrità dell'evento.

Effetti della revoca sul lifecycle

Una volta registrata la revoca sul ledger, il documento o la versione interessata non possono più essere utilizzati come validi nei flussi ordinari. Se viene revocata una singola versione, quella versione assume lo stato *Revoked*, mentre le altre versioni mantengono il proprio stato.

La revoca di un documento produce effetti anche sulle autorizzazioni collegate. In particolare, eventuali deleghe/policy riferite al documento o alla versione revocata non devono più essere utilizzabili come base autorizzativa. Il Policy Engine, in fase di accesso, quindi verifica sempre non solo lo stato della delega/policy, ma anche lo stato corrente del documento a cui la delega si riferisce.

Effetti sull'accesso al contenuto

La revoca viene interpretata come blocco degli effetti futuri: dopo la revoca, il sistema non deve più rilasciare nuove chiavi, nuovi Access Grant o nuove autorizzazioni relative alla versione revocata. Inoltre, eventuali richieste successive di accesso saranno respinte dal Policy Engine perché il documento non si trova più in uno stato compatibile con l'operazione richiesta.

Registrazione nell'Audit Registry

L'intero flusso di revoca viene registrato nell'**Audit Registry**. In questo modo, un auditor autorizzato può ricostruire chi ha richiesto la revoca, chi l'ha approvata, quale documento o versione è stata coinvolta, quale policy è stata applicata, quando la revoca è diventata efficace e quali effetti ha prodotto sulle autorizzazioni successive.

WP3 – Security Analysis

Analisi rispetto alle proprietà di sicurezza definite in WP1

Integrità documentale e integrità dello stato

La proprietà di integrità richiede che nessun soggetto non autorizzato possa alterare documenti, versioni, policy, revoche o deleghe senza che la modifica venga rilevata.

I documenti non sono salvati in chiaro sul ledger, ma vengono conservati off-chain, cifrati e identificati tramite CID. Poiché il CID è derivato dal contenuto, un'alterazione del payload cifrato produrrebbe un identificatore diverso da quello registrato nel Document Lifecycle Registry. Di conseguenza, un documento alterato non risulterebbe coerente con il riferimento registrato sul ledger.

Inoltre, ogni documento è firmato dall'utente operativo che lo produce. Questo consente di verificare non solo che il contenuto non sia stato alterato, ma anche che esso sia stato effettivamente generato dal soggetto indicato come creatore.

Anche lo stato del documento è protetto. Il Document Lifecycle Registry mantiene una sequenza versionata di record documentali, in cui ogni nuova versione conserva un riferimento esplicito alla precedente.

L'integrità dello stato globale è rafforzata dalla DLT permissioned; le transazioni che modificano policy, deleghe, revoche, stati documentali o credential status non vengono accettate passivamente, ma sono sottoposte sia al consenso BFT sia a controlli applicativi. I peer verificano firme, ruoli, quorum, stato corrente, assenza di revoche e compatibilità con il lifecycle della risorsa coinvolta.

Autenticità delle operazioni

La proprietà di autenticità richiede che ogni operazione rilevante sia riconducibile a un soggetto o a un componente autorizzato.

Il design soddisfa questa proprietà tramite DID e relativi DID Document, firme digitali e Verifiable Credentials. Ogni attore del sistema possiede un DID associato a uno o più metodi di verifica; quando un soggetto firma una richiesta, un consenso, una delega, una revoca o una Verifiable Presentation, il sistema risolve il DID, recupera il DID Document tramite il CID registrato nell'Identity Registry e verifica che la chiave usata sia presente e abilitata alla relazione corretta.

Il sistema distingue correttamente tra autenticazione tecnica e autorizzazione applicativa: il possesso di un DID valido dimostra il controllo crittografico dell'identificatore, ma non attribuisce automaticamente ruoli, qualifiche o permessi. Tali attributi sono attestati tramite Verifiable Credentials emesse da issuer riconosciuti e controllate rispetto al Trust Registry e al Credential Status Registry.

Il sistema soddisfa, quindi, la proprietà di autenticità, a condizione che:

- il DID sia attivo;
- il DID Document sia integro e aggiornato;
- la chiave privata non sia compromessa;
- la firma sia verificata;
- eventuali credenziali siano valide, non revocate e rilasciate da issuer autorizzati.

In caso di compromissione della chiave privata, l'autenticità crittografica rimane formalmente valida, ma non corrisponde più necessariamente alla volontà reale del soggetto.

Non ripudio

Il non ripudio richiede che un soggetto non possa negare arbitrariamente di aver compiuto o approvato una determinata operazione.

Il design supporta questa proprietà attraverso firme digitali e registrazione auditabile degli eventi. Le operazioni rilevanti, come creazione documentale, delega, voto di governance ed emissione di credenziali, sono firmate dal soggetto o dal componente responsabile; inoltre, gli eventi corrispondenti sono registrati nell'Audit Registry e ancorati al ledger tramite batch cifrati.

Questo consente di ricostruire ex post:

- chi ha richiesto l'operazione;
- quale chiave era valida al momento della firma;
- quale policy era applicabile;
- quale versione del documento o della delega era coinvolta;
- quale esito è stato prodotto;
- quale componente ha eseguito o validato l'operazione.

Il non ripudio è particolarmente forte per le operazioni registrate sul ledger, perché una volta confermata una transazione non può essere modificata unilateralmente da una singola organizzazione. Anche per gli audit log off-chain, l'ancoraggio tramite CID consente di rilevare alterazioni successive.

Autorizzazione per operazioni sui documenti

La proprietà di autorizzazione richiede che un'operazione venga accettata solo se il soggetto possiede, al momento dell'uso, i diritti necessari rispetto a documento, azione, contesto, finalità, policy e stato corrente.

Il design proposto si basa su un modello ibrido RBAC/ABAC, scelta è adeguata al dominio sanitario, perché il solo ruolo non è sufficiente. Non basta sapere che un soggetto è medico: occorre sapere se è il medico corretto, con la specializzazione corretta, rispetto al paziente corretto, per una finalità ammessa e nel momento corretto.

L'autorizzazione è valutata dal Policy Engine, che consulta gli attori necessari per la verifica e l'accesso viene concesso solo se gli attributi della VC presentata soddisfano le policy applicabili e se non esistono restrizioni o revoche.

Correttezza della delega

Il sistema deve permettere il trasferimento temporaneo di diritti, anche lungo catene di deleghe, senza consentire ampliamenti indebiti.

Il design soddisfa questa proprietà attraverso il Delegation Registry; ogni delega è registrata come un fatto concreto, contenente delegante, delegato, documento, azioni delegate, finalità, intervallo temporale, eventuale delega sorgente, stato, riferimento alla decisione del Policy Engine.

La delega non è un permesso permanente, ma un diritto derivato, limitato e revocabile.

Il sistema controlla che:

- il delegante sia tecnicamente autenticato e possieda attributi e diritti sufficienti;

- il delegato abbia un DID valido e attivo;
- il paziente abbia espresso consenso esplicito;
- la delega non ecceda le azioni, la finalità o la durata del diritto originario;
- in caso di sub-delega, la delega sorgente consenta la sub-delega.

La proprietà è soddisfatta, purché venga mantenuta una distinzione tra stato registrato e stato effettivo della delega.

Quindi ogni delega derivata mantiene un *ParentDelegationRef*, e in fase di uso il Policy Engine risale l'intera catena; questo impedisce che una delega figlia resti valida se una delega precedente è stata revocata o è scaduta. Infatti, una delega può risultare apparentemente *active* se considerata isolatamente, ma essere effettivamente non spendibile perché una delega sorgente è stata revocata. Dunque, il Policy Engine non autorizza mai un accesso controllando solo la delega finale, ma verifica sempre l'intera catena tramite *ParentDelegationRef*.

Correttezza della revoca e propagazione degli effetti

La revoca riguarda credenziali, deleghe, consensi, policy e documenti. Il design gestisce in modo esplicito quattro forme di revoca: revoca di VC, di deleghe, di policy e di documento.

La revoca di una credenziale è gestita tramite Credential Status Registry. Una VC può rimanere crittograficamente verificabile anche dopo la perdita di validità sostanziale; per questo motivo il sistema non si limita alla verifica della firma, ma consulta sempre lo stato corrente della credenziale perché se quest'ultima è revocata o sospesa, non può più essere usata come fonte valida di attributi.

La revoca delle deleghe è gestita tramite record di revoca nel Delegation Registry; il record di delega originario non viene eliminato, ma la sua efficacia viene neutralizzata dalla presenza di un record di revoca ad esso collegato.

La revoca delle policy è gestita tramite versionamento. Quando una policy viene aggiornata, la versione precedente non viene eliminata, ma viene impostata come *revoked*.

Il limite principale riguarda la revoca di un documento. Infatti, una volta che il documento è stato consegnato all'utente insieme alla DEK, il sistema non può garantire che l'utente non ne abbia già salvato una copia o che non possa continuare a consultarla localmente. La revoca può impedire accessi futuri al documento e bloccare nuovi rilasci delle chiavi di decifratura, ma non può eliminare le informazioni che sono già state ottenute. Questo limite è inevitabile in qualsiasi sistema in cui un soggetto riceve i dati e le chiavi necessarie per decifrarli.

Coerenza del lifecycle documentale

Il documento è modellato come entità dinamica e versionata; infatti, viene mantenuto un DocumentID stabile che si riferisce al documento logico, mentre ogni versione ha un CID, un numero di versione, uno stato e un riferimento alla versione precedente.

Gli stati principali sono *Certified*, *Archived*, *Revoked*. Il passaggio da una versione alla successiva non sovrascrive la precedente, bensì la nuova versione diventa *Certified*, mentre la precedente diventa *Archived*. Questo consente di distinguere tra versione corrente e storia documentale.

La coerenza è garantita dai controlli del Document Lifecycle Registry e dai peer della DLT prima del *commit*. Una transizione viene accettata solo se è compatibile con lo stato corrente.

Accountability e auditabilità

Il sistema offre buone garanzie di auditabilità. L'Audit Registry non conserva tutti i log in chiaro sul ledger, ma registra riferimenti verificabili a batch off-chain cifrati, identificati da *CID*, hash e collegamento al batch precedente. Gli eventi interni ai batch contengono riferimenti a soggetto, azione, risorsa, esito, timestamp, policy applicata, documento, delega, credenziale o *DID* coinvolto.

Questa struttura consente di ricostruire ex post: accessi autorizzati/negati, creazioni e aggiornamenti documentali, deleghe e revoche, modifiche di policy.

Il design è particolarmente solido perché l'audit non si basa soltanto sullo stato corrente; infatti, durante la ricostruzione, l'auditor recupera lo stato storico dei registri al momento dell'evento.

Disponibilità e tempestività operativa

Il sistema affronta la disponibilità dei documenti e, più in generale, di tutte le risorse salvate off-chain, attraverso nodi IPFS per ogni organizzazione, pinning dei contenuti e replica del nodo IPFS e del Pinning Manager; infatti, tali scelte riducono il rischio di single point of failure.

Tuttavia, la disponibilità non è comunque del tutto garantita a causa della gestione dello storage del meccanismo di IPFS; il sistema proposto prevede un componente applicativo dedicato al pinning delle risorse e, inoltre, tali risorse sono salvate in replica su più nodi del sistema. Nonostante ciò, l'unpinning arbitrario di una risorsa dal nodo IPFS non può comunque essere in alcun modo impedito, motivo per cui è necessaria anche la replica.

Nel caso specifico, però, delle policy patient-driven, per garantire la privacy sulle informazioni del paziente queste vengono salvate solo dalle organizzazioni che subiscono tale policy; dunque, non è garantita la replica del contenuto. In questo contesto, tale eventualità viene considerata un trade-off accettabile.

Inoltre, anche se più nodi hanno memorizzata la stessa risorsa, comunque potrebbe verificarsi come caso limite l'unpinning su tutti. Tali situazioni non sono però mitigabili mediante progettazione del sistema, perché sono problematiche intrinseche della gestione degli storage con IPFS.

Per quanto concerne, invece, la tempestività, in base al funzionamento descritto del sistema, per le operazioni critiche come l'accettazione di una delega è necessario che il wallet del paziente sia online e non compromesso. Questo è un limite significativo perché se il wallet del paziente è offline o indisponibile, un accesso clinicamente necessario potrebbe non essere possibile. Tale scelta è dettata dal contesto di riferimento considerato, per consentire al paziente di avere pieno controllo sulle operazioni che vengono effettuate in relazione ai suoi dati. Dunque, anche in questo caso, può essere considerato un trade-off dettato dal contenuto sanitario considerato.

Confidenzialità e minimizzazione

La confidenzialità è affrontata tramite cifratura off-chain dei documenti, delle policy patient-driven e dei batch di audit, ovvero di tutte le risorse che potrebbero contenere dati sensibili degli utenti. Il ledger non contiene documenti clinici in chiaro, ma solo riferimenti, *CID*, stati e metadati minimizzati.

Il sistema usa inoltre SD-JWT per supportare selective disclosure sulle Verifiable Credentials. Un utente operativo può rivelare solo gli attributi necessari alla richiesta, ad esempio ruolo e specializzazione, senza mostrare l'intera credenziale. Queste scelte sono coerenti con il principio di minimizzazione. Tuttavia, la minimizzazione non è completa.

Analisi degli attacchi principali

Collusione tra organizzazioni

Le organizzazioni partecipano alla governance, votano policy, emettono credenziali, validano transazioni e possono influenzare il funzionamento del sistema. Il design mitiga la collusione su due livelli:

- A **livello infrastrutturale**, la DLT permissioned usa un consenso BFT. Questo impedisce a una minoranza bizantina di alterare unilateralmente il ledger o finalizzare transazioni non valide.
- A **livello applicativo**, le policy di governance richiedono una supermajority, mentre le policy di accesso globali richiedono maggioranza semplice. I peer verificano che gli approvatori siano distinti, autorizzati e non duplicati.

Privilege escalation da parte di utenti operativi

Un utente operativo compromesso o malevolo può tentare di ottenere accesso a documenti non autorizzati, usare credenziali non valide o aggirare le policy.

Il design mitiga questo attacco tramite una catena di controlli, tra cui risoluzione e verifica del DID, verifica della VP con relativa VC associata, controlli sui registri che si occupano delle credenziali (ovvero Credential Status Registry e Trust Registry), valutazione del ruolo e degli attributi in relazione alle policy e ad eventuali deleghe, ed infine audit dell'esito.

Un utente non può:

- auto-attribuirsi un ruolo, perché i ruoli sono attestati da VC firmate da issuer riconosciuti ed in più si assume che gli issuer non producano credenziali false.
- usare una VC revocata, perché lo stato viene controllato nel Credential Status Registry.
- usare una credenziale emessa da un issuer non autorizzato, perché il Trust Registry viene consultato.
- riutilizzare una VP, perché la presentazione include tutti i campi necessari ad evitare attacchi di replay.

La privilege escalation rimane possibile solo se una delle assunzioni viene violata.

Abuso di diritti delegati

Un delegato malevolo può tentare di usare una delega oltre lo scopo previsto, dopo la scadenza, dopo la revoca o per azioni non autorizzate. Durante l'uso, il sistema verifica che il richiedente coincida con il delegato, l'azione richiesta e il documento sia quello indicato nella delega, la delega sia temporalmente valida e non revocata ed infine che la delega non sia in contrasto con nuove policy.

Dunque, quest'attacco è mitigato.

Replay

Un attaccante può tentare di riutilizzare una prova, una credenziale, una delega, un Access Grant o una policy precedentemente valida, ma non più valida al momento dell'uso.

Il sistema riduce il rischio di replay attack utilizzando meccanismi come *nonce*, *timestamp* e challenge nelle Verifiable Presentation. Inoltre, controlla lo stato aggiornato della credenziale, verifica che l'Access Grant sia ancora valido nel suo intervallo temporale e tiene traccia delle richieste o delle presentazioni già utilizzate, così da evitare che vengano riusate più volte.

Tampering documentale e compromissione dello storage off-chain

In primo luogo, il sistema proposto mitiga l'alterazione silenziosa del contenuto: poiché il *CID* dipende dal contenuto stesso, un file modificato produrrebbe un *CID* differente e non corrisponderebbe più al riferimento registrato sul ledger. Inoltre, il Document Lifecycle Registry associa ogni documento alla specifica versione, al relativo *CID*, allo stato di lifecycle e ai metadati minimi necessari alla verifica, permettendo di distinguere la versione corrente dalle versioni precedenti e impedendo che una copia obsoleta venga accettata come valida per le operazioni ordinarie. Essendo il documento firmato dal creatore, la verifica della firma rafforza ulteriormente l'autenticità e la non ripudiabilità dell'operazione.

Di conseguenza, anche se un attaccante riuscisse a far recuperare un contenuto diverso dallo storage, la verifica del *CID*, dello stato documentale, della versione e della firma farebbe emergere l'incoerenza con lo stato certificato del sistema.

Inoltre, gli eventi rilevanti e i fallimenti sono registrati nell'Audit Registry, così che l'indisponibilità o il recupero anomalo di una versione documentale non rimangano eventi silenziosi, ma siano rilevabili e ricostruibili ex post.

Man-in-the-Middle

Un attaccante esterno può intercettare, modificare o iniettare messaggi tra gli attori.

Il design mitiga questo attacco perché le richieste e le risposte rilevanti sono firmate. Anche se un messaggio viene intercettato, una modifica altererebbe la firma. Inoltre, *VP*, *Access Grant* e richieste includono *nonce*, *timestamp*, *verifier*, contesto, *documentId*, *action* e *purposeOfUse*. Questo impedisce a un attaccante di riutilizzare messaggi validi in un contesto diverso.

La confidenzialità dei contenuti è protetta dalla cifratura. I documenti sono cifrati con *DEK*, e le *DEK* sono inviate cifrate con la chiave di *keyAgreement* del destinatario. Anche se un attaccante intercetta il payload cifrato o la chiave cifrata, non dovrebbe poterli decifrare senza la chiave privata corrispondente.

Compromissione di chiavi e wallet

La compromissione delle chiavi è una delle minacce più difficili. Se un attaccante ottiene la chiave privata di un utente, può generare firme valide mentre se ottiene una *DEK*, può decifrare il documento corrispondente. Il sistema definito prevede alcune mitigazioni ovvero: chiavi distinte per *authentication*, *assertionMethod* e *keyAgreement*, revoca delle credenziali, controllo dello stato del *DID*, audit delle operazioni, e infine, *DEK* differenti per ogni documento.

In particolare, la separazione delle chiavi permette di ridurre l'impatto di una compromissione parziale; infatti, una chiave di *authentication* compromessa non permette automaticamente di firmare credenziali, essendo che per *assertionMethod* è prevista una chiave differente.

Inoltre, l'uso di una *DEK* diversa per ogni documento limita i danni conseguenti la compromissione della chiave stessa, perché in questo modo viene violato solo un documento del paziente, non tutti (a differenza di quello che sarebbe successo se avessimo avuto una chiave per paziente).

Rimangono però limiti importanti. Anche se l'accesso al wallet è protetto tramite autenticazione multi-fattore e le chiavi private sono conservate mediante meccanismi di protezione locale del dispositivo o del wallet, la compromissione del wallet o del dispositivo non può essere esclusa completamente. Se il wallet del paziente venisse compromesso dopo l'autenticazione, o se l'attaccante riuscisse a ottenere il controllo operativo del dispositivo, potrebbe rilasciare *DEK* a soggetti non autorizzati o decifrare documenti per cui il paziente dispone delle chiavi. Analogamente, se il wallet di un medico fosse

compromesso, l'attaccante potrebbe presentare credenziali, richiedere accessi o firmare operazioni apparentemente valide.

Requester esterno malevolo

Un requester esterno, ad esempio un'assicurazione, può tentare di ottenere più informazioni del necessario o correlare richieste diverse. Nel sistema è previsto che il requester esterno presenti una VC specifica (rilasciata dall'organizzazione di appartenenza del requester che è anch'essa esterna al sistema), che venga verificato da un verifier interno e che il Policy Engine valuti finalità, documento e azione. Inoltre, in caso di accesso al documento, è richiesto il consenso esplicito del paziente.

Questa logica protegge dall'accesso diretto e non autorizzato al documento; inoltre, per soddisfare pienamente la proprietà di minimizzazione, è prevista anche la possibilità che il requester esterno non acceda al contenuto del documento, ma effettui una verifica selettiva, come esistenza, validità o stato di revoca di un documento, senza accedere al contenuto. In tal caso, riceverà solo una prova o un'attestazione limitata, ad esempio: *"Il documento esiste"*.

Questo permettere di soddisfare richieste legittime senza rivelare l'intero contenuto clinico.

Auditor curious o malevolo

Gli auditor devono poter ricostruire eventi, ma non devono trasformare l'audit in sorveglianza generalizzata. Il design mitiga questo rischio limitando l'accesso tramite policy di accesso globale specifiche per gli audit. L'auditor deve presentare una richiesta firmata, con finalità, target, finestra temporale e *requestedEvidenceScope* e il Policy Engine produce una decisione *audit-allow* solo se la richiesta è compatibile con le policy. I batch di audit sono cifrati, e la chiave viene rilasciata solo entro il perimetro autorizzato.

Debolezze e limiti del design

Dipendenza dal wallet del paziente

Un limite rilevante è l'assunzione che il wallet del paziente sia online e non compromesso. Questo rafforza il controllo del paziente, ma potrebbe introdurre un problema di disponibilità nel momento in cui può essere necessario l'accesso e il paziente non è disponibile o non può interagire con il wallet. Nel caso specifico in esame, però, si assume che il contesto sanitario non sia emergenziale, ma ordinario e che dunque il paziente sia nelle condizioni di poter rispondere ed interagire con il wallet.

Revoca non retroattiva dopo rilascio della DEK

Una volta che una *DEK* è stata rilasciata, il sistema non può impedire che il destinatario la conservi. Questo significa che la revoca blocca accessi futuri, ma non cancella conoscenza già acquisita. Questo limite è inevitabile nei sistemi che rilasciano contenuti o chiavi agli utenti finali.

Correlabilità dei metadati on-chain

Il ledger non contiene documenti in chiaro, ma contiene metadati che in ambito sanitario possono essere sensibili. In questo caso, è possibile effettuare inferenze anche senza accesso al contenuto. Nel sistema proposto, questo viene considerato un trade-off tra verificabilità, auditabilità e riservatezza. L'inserimento di informazioni nel registro aumenta la trasparenza e consente il controllo dell'integrità e della sequenza delle operazioni, ma introduce inevitabilmente una superficie informativa osservabile.

Una delle alternative considerate consiste nell'utilizzare *HMAC* come identificativi nei registri al posto dei DID; questa soluzione può ridurre l'esposizione immediata dell'identità e rendere più difficile il

riconoscimento diretto degli attori coinvolti. Tuttavia, non elimina completamente il problema perché anche identificativi pseudonimi o derivati crittograficamente possono risultare correlabili se vengono riutilizzati, se compaiono in sequenze ricorrenti o se sono associati a pattern temporali e operativi distinguibili.

Pertanto, l'uso di *HMAC* può mitigare alcune forme di identificazione diretta, ma non risolve in modo definitivo il rischio di inferenza e correlazione.

Gestione quorum applicativo

A livello applicativo si può comunque verificare il caso limite per cui il quorum venga raggiunto da un insieme di organizzazioni tutte malevole, ma questa è una limitazione fisiologica dei sistemi di governance multi-authority.

WP4 – Implementation and Evaluation

Obiettivo del prototipo

Il prototipo ha l'obiettivo di dimostrare la fattibilità tecnica delle componenti centrali del modello proposto e di valutarne il comportamento in termini di latenza, throughput e overhead di storage.

L'implementazione si concentra sui registri logici principali introdotti nel WP2: *Document Lifecycle Registry*, *Delegation Registry*, *Policy Registry* e *Audit Registry*. Tali registri sono implementati mediante smart contract, così da rappresentare su blockchain lo stato condiviso, verificabile e resistente alla manomissione del sistema.

Il prototipo è disponibile su 

Scelte implementative e assunzioni

Rispetto al design teorico descritto nel WP2, il prototipo introduce alcune semplificazioni implementative. In particolare, viene utilizzata *Ethereum* come infrastruttura principale per implementare i registri logici del sistema e per conservare i riferimenti verificabili ai contenuti documentali, anche se nella soluzione viene descritto l'uso di una DLT Permissioned e, dunque, la scelta più adatta sarebbe stata l'implementazione mediante *Hyperledger Fabric*.

Inoltre, nel prototipo non viene implementato la cifratura documentale. Questa semplificazione implica che il prototipo non valuta le garanzie di confidenzialità offerte dalla cifratura dei documenti. In un sistema reale, il CID dovrebbe essere calcolato sul documento cifrato, mentre nell'implementazione viene calcolato sul contenuto in chiaro. Dunque, viene implementato solo ciò che serve a dimostrare il legame tra documento, record on-chain e versione documentale.

Con riferimento alle policy e alla loro gestione, la soluzione prevede il versionamento delle policy stesse; tuttavia, tale funzionalità non è stata implementata nella Proof of Concept. Anche questa scelta è stata adottata per motivi di semplificazione.

Infine, sebbene nel modello sia stato definito un controllo degli accessi basato su ABAC e RBAC, nell'implementazione della Proof of Concept fornita il controllo viene effettuato esclusivamente sulla base del ruolo. Tale scelta è stata adottata per evitare di ampliare eccessivamente la gestione implementativa delle organizzazioni.

Architettura implementata

DataTypes.sol

Il contratto *DataTypes.sol* non rappresenta un registry autonomo, ma una libreria condivisa che definisce le strutture semantiche comuni utilizzate dagli altri contratti. La sua funzione principale è uniformare il significato degli stati, dei ruoli, delle azioni e dei codici di audit all'interno dell'intero prototipo. In particolare, è possibile suddividerla nelle seguenti sezioni:

Stati e ruoli

In primis, vengono definiti gli **stati documentali** associati all'enum *DocumentState*, ovvero *Certified*, *Archived* e *Revoked*, che rappresentano il ciclo di vita del documento clinico.

Inoltre, è presente l'enum *OrganizationStatus*, utilizzato per rappresentare lo **stato delle organizzazioni** partecipanti al sistema. Gli stati previsti sono *Active*, *Suspended* e *Revoked*, oltre allo stato iniziale *None*; tali stati, consentono di modellare il ciclo di vita delle organizzazioni autorizzate a partecipare

alla rete, distinguendo tra organizzazioni pienamente operative, temporaneamente sospese o definitivamente revocate. Questa informazione è rilevante perché solo le organizzazioni riconosciute e attive dovrebbero poter partecipare alle operazioni di governance, gestione delle policy e rilascio di credenziali.

Sono presenti enum riguardanti gli **stati** anche in riferimento alle **policy**, alle **deleghe**, ai **DID** e alle **credenziali**, rispettivamente *PolicyProposalStatus*, *DelegationProposalStatus*, *DIDStatus* e *CredentialStatus*.

Per quanto concerne i **ruoli**, vengono definiti quelli applicativi principali: *Patient*, *Doctor*, *Nurse*, *Technician*, *Auditor* ed *ExternalRequester*.

Governance

Per la governance del sistema viene definito l'enum *GovernanceActionType* che identifica le **principali operazioni** sottoposte a **governance**, come aggiunta, sospensione, revoca o riattivazione di un'organizzazione, la registrazione di policy di governance, l'impostazione dei permessi globali e la definizione della profondità massima delle deleghe.

Un altro aspetto importante è la **definizione delle azioni** legate alla **gestione del sistema** e dei documenti, come *READ*, *CREATE*, *UPDATE*, *REVOKE* e *DELEGATE*, usate in modo trasversale dai contratti per verificare i permessi, registrare eventi di audit e valutare le richieste di accesso.

Credenziali

Come descritto nel WP2, un soggetto dimostra il proprio ruolo tramite una VC emessa da una delle organizzazioni fidate; a tale scopo vengono definiti anche i principali tipi di Verifiable Credential previsti dal sistema, come *MedicalProfessionalVC*, *NurseProfessionalVC*, *TechnicianProfessionalVC*, *AuditorProfessionalVC* ed *ExternalRequesterVC*.

Inoltre, un elemento rilevante della libreria è la struttura *PresentedCredential*, che rappresenta in forma minimale la credenziale presentata al sistema. Al suo interno, il campo *format* identifica la tipologia di formato utilizzato per la credenziale, mentre il campo *id* ne rappresenta l'identificativo univoco. L'emittente e il titolare della credenziale sono definiti rispettivamente dai campi *issuer* e *subject*; in particolare, il valore di *issuer* consente di verificare, tramite il Trust Registry, che l'ente sia un

```
struct PresentedCredential {
    string format;
    string id;
    string issuer;
    string subject;
    string credentialType;
    string sdJwtCredential;
    string[] selectivelyDisclosableClaims;
}
```

Figura 11 Frammento di Codice di DataTypes

soggetto fidato e autorizzato. La tipologia specifica di credenziale professionale viene invece indicata nel campo *credentialType*. Per quanto riguarda la gestione della sicurezza e dei dati, il campo *sdJwtCredential* racchiude l'intera credenziale firmata in formato SD-JWT, permettendo così di controllarne l'integrità e l'autenticità. Infine, l'array di stringhe *selectivelyDisclosableClaims* contiene l'elenco dei claim e degli attributi specifici che l'utente ha deciso di presentare e rivelare al sistema, sfruttando la funzionalità di divulgazione selettiva nativa del formato.

Audit

Infine, in tale libreria vengono definiti anche i **target di audit**, i **risultati delle operazioni** e i **reason code** (motivo preciso per cui l'evento è stato generato). Questo permette agli eventi registrati nell'*Audit Registry* di avere una struttura omogenea e facilmente interpretabile.

AuditRegistry.sol

Il contratto *AuditRegistry.sol* implementa il **registro di audit del sistema**. Come già descritto in precedenza, nel prototipo la logica del registro di audit viene semplificata: invece di gestire log aggregati off-chain e ancorati on-chain tramite CID, le operazioni rilevanti vengono rappresentate direttamente tramite **eventi Solidity**.

La componente principale del contratto è l'evento *AuditEvent*, utilizzato per **registrare le informazioni essenziali** associate a un'operazione: l'attore coinvolto, l'azione eseguita, il target dell'operazione, l'esito, il motivo dell'esito e il *timestamp*.

Un aspetto rilevante del contratto è il controllo sugli **emitter autorizzati**; infatti, solo gli indirizzi abilitati, specificati nel mapping *authorizedEmitters*, possono invocare la funzione *logEvent*, evitando che un *address* qualsiasi possa **generare eventi di audit** arbitrari. Nel costruttore, il deployer viene salvato come *bootstrapAdmin* e viene anche inserito tra gli *authorizedEmitters*, mentre ulteriori emitter possono essere **autorizzati o rimossi durante la fase di bootstrap**. Questa fase può essere successivamente bloccata tramite *lockBootstrap*, rendendo stabile la configurazione degli indirizzi autorizzati. È importante evidenziare che il campo *actor* viene passato come parametro alla funzione *logEvent* e non coincide necessariamente con *msg.sender*; questa scelta è utile perché, nel prototipo implementato, l'evento può essere generato da un registro autorizzato, ma **l'operazione può essere stata richiesta o causata da un altro soggetto**, come un medico, un paziente o un'organizzazione.

Infine, si precisa che il contratto non conserva gli audit log in una struttura dati interna, ma li rende disponibili tramite eventi *Solidity*.

Il contratto risulta, quindi, centrale per l'**osservabilità del prototipo**, poiché consente agli altri registri del sistema di registrare le operazioni critiche in modo uniforme e controllato.

La seguente tabella riassume le principali componenti del contratto.

Categoria	Elemento	Tipo / Firma	Descrizione
Dati	<i>bootstrapAdmin</i>	address public immutable	Indirizzo dell'account che effettua il deploy del contratto. Ha il ruolo iniziale di amministratore del bootstrap.
Dati	<i>bootstrapLocked</i>	bool public	Indica se la fase di bootstrap è stata chiusa. Quando vale true, non è più possibile modificare gli emitter autorizzati.
Dati	<i>authorizedEmitters</i>	mapping(address => bool)	Mapping che associa a ogni indirizzo un valore booleano che indica se tale indirizzo è autorizzato a emettere eventi di audit.
Modifier	<i>onlyBootstrapAdmin</i>	modifier	Permette l'esecuzione della funzione solo all'indirizzo <i>bootstrapAdmin</i> .
Modifier	<i>onlyDuringBootstrap</i>	modifier	Permette l'esecuzione della funzione solo finché la fase di bootstrap non è stata bloccata.
Modifier	<i>onlyAuthorizedEmitter</i>	modifier	Permette l'esecuzione della funzione solo agli indirizzi presenti tra gli emitter autorizzati.
Operazione	<i>constructor</i>	constructor()	Inizializza il contratto impostando come <i>bootstrapAdmin</i> l'indirizzo che effettua il deploy e autorizzandolo come primo emitter.
Operazione	<i>setAuthorizedEmitter</i>	setAuthorizedEmitter(address emitter, bool authorized)	Consente al bootstrap admin, durante la fase di bootstrap, di autorizzare o rimuovere un indirizzo dalla lista degli emitter autorizzati.
Operazione	<i>lockBootstrap</i>	lockBootstrap()	Blocca definitivamente la fase di bootstrap. Dopo questa operazione non è più possibile modificare gli emitter autorizzati.

Categoria	Elemento	Tipo / Firma	Descrizione
Operazione	<i>logEvent</i>	logEvent(address actor, bytes32 action, bytes32 targetId, bytes32 targetType, bytes32 result, bytes32 reasonCode)	Permette a un emitter autorizzato di registrare un evento di audit contenente attore, azione, target, risultato, motivazione e timestamp.
Evento	<i>AuditEvent</i>	event	Evento principale del registro di audit. Notifica un'azione rilevante eseguita nel sistema.
Evento	<i>AuthorizedEmitterSet</i>	event	Evento emesso quando un indirizzo viene autorizzato o rimosso dalla lista degli emitter autorizzati.
Evento	<i>BootstrapLocked</i>	event	Evento emesso quando la fase di bootstrap viene chiusa definitivamente.

Tabella 2 Riferimenti AuditRegistry

OrganizationRegistry.sol

Il contratto *OrganizationRegistry.sol* implementa il **registro delle organizzazioni riconosciute** dal sistema. Nel modello descritto, tali organizzazioni partecipano alla governance, contribuiscono alla validazione delle operazioni e costituiscono il perimetro di fiducia della rete permissioned. Nell'implementazione, questa logica viene rappresentata attraverso indirizzi *Ethereum* associati a uno specifico stato organizzativo.

Ogni organizzazione registrata è **descritta** da un record contenente l'indirizzo dell'organizzazione, un *organizationCID* riferito a contenuti e metadati off-chain, lo stato corrente dell'organizzazione e i *timestamp* relativi alla creazione e all'ultimo aggiornamento. Lo stato consente di distinguere le organizzazioni attive da quelle sospese o revocate, permettendo agli altri contratti di verificare se un determinato indirizzo appartiene effettivamente a un soggetto abilitato.

```
struct OrganizationRecord {
    address organizationAddress;
    bytes32 organizationCID;
    DataTypes.OrganizationStatus status;
    uint256 createdAt;
    uint256 updatedAt;
}
```

Figura 12 Frammento di Codice di OrganizationRegistry

Il contratto fornisce a *PolicyGovernance* le informazioni necessarie per verificare che solo le **organizzazioni attive** possano partecipare ai processi di governance, proporre policy e votare.

Un ulteriore elemento rilevante è il mantenimento del **numero di organizzazioni attive** tramite *activeOrganizationCount*. Anche questo valore viene utilizzato da *PolicyGovernance* per calcolare i quorum, facendo sì che la **composizione effettiva del network** incida direttamente sulle **soglie decisionali**.

Un aspetto rilevante del contratto è la distinzione tra **fase di bootstrap** e **fase di governance ordinaria**. Il *bootstrapAdmin* viene utilizzato solo per inizializzare il prototipo, configurare il contratto di governance e registrare eventualmente le prime organizzazioni. Una volta completata questa fase, il bootstrap può essere bloccato tramite *lockBootstrap*, impedendo ulteriori modifiche dirette da parte dell'amministratore iniziale; da questo momento in poi, tali modifiche devono essere eseguite attraverso il contratto *PolicyGovernance*.

Per definire quali sono le funzioni che possono essere richiamate solo dal *bootstrapAdmin* o dalla governance del sistema sono stati implementati **due modifier**, *onlyBootstrapAdmin*, usato per funzioni strettamente amministrative di inizializzazione, e *onlyBootstrapOrGovernance*, usato per le funzioni che modificano le organizzazioni. In particolare, il *bootstrapAdmin* viene associato a *msg.sender* all'interno del costruttore.

Ogni modifica rilevante viene registrata anche tramite **eventi Solidity specifici**; inoltre, il contratto interagisce con l'*AuditRegistry*, invocando *logEvent* per produrre un evento di audit associato alle operazioni principali legate allo stato delle organizzazioni, come l'aggiunta o la sospensione di un'organizzazione.

La seguente tabella riassume le principali componenti del contratto.

Categoria	Elemento	Tipo / Firma	Descrizione
Interfaccia	<i>IAuditRegistryForOrganizations</i>	interface	Interfaccia utilizzata per interagire con il contratto <i>AuditRegistry.sol</i> e registrare eventi di audit relativi alle operazioni sulle organizzazioni.
Struct	<i>OrganizationRecord</i>	struct	Struttura che rappresenta una singola organizzazione registrata nel sistema. Contiene indirizzo, CID, stato e timestamp di creazione e aggiornamento.
Dati	<i>bootstrapAdmin</i>	address public immutable	Indirizzo dell'account che effettua il deploy del contratto. Ha poteri amministrativi solo nella fase iniziale di bootstrap.
Dati	<i>governanceContract</i>	address public	Indirizzo del contratto di governance autorizzato a gestire le organizzazioni dopo la chiusura del bootstrap.
Dati	<i>bootstrapLocked</i>	bool public	Indica se la fase di bootstrap è stata bloccata. Quando vale true, le modifiche non possono più essere eseguite direttamente dal bootstrap admin.
Dati	<i>auditRegistry</i>	<i>IAuditRegistryForOrganizations</i> public	Riferimento al contratto di audit utilizzato per registrare le operazioni critiche.
Dati	<i>organizations</i>	mapping(address => <i>OrganizationRecord</i>)	Mapping che associa a ogni indirizzo il relativo record dell'organizzazione.
Dati	<i>organizationList</i>	address[] private	Array contenente gli indirizzi delle organizzazioni registrate, utile per enumerarle.
Dati	<i>activeOrganizationCount</i>	uint256 public	Numero di organizzazioni attualmente attive nel sistema.
Modifier	<i>onlyBootstrapAdmin</i>	modifier	Consente l'esecuzione della funzione solo al bootstrap admin.
Modifier	<i>onlyBootstrapOrGovernance</i>	modifier	Consente l'esecuzione della funzione al bootstrap admin durante la fase iniziale oppure al contratto di governance dopo la configurazione.
Operazione	<i>constructor</i>	constructor(address auditRegistryAddress)	Inizializza il contratto impostando il bootstrap admin e il riferimento al contratto di audit.
Operazione	<i>setGovernanceContract</i>	setGovernanceContract(address governanceContractAddresses)	Imposta l'indirizzo del contratto di governance durante la fase di bootstrap.
Operazione	<i>lockBootstrap</i>	lockBootstrap()	Blocca definitivamente la fase di bootstrap. Può essere eseguita solo dopo aver impostato il contratto di governance.
Operazione	<i>addOrganization</i>	addOrganization(address organizationAddress, bytes32 organizationCID)	Registra una nuova organizzazione, salvandone indirizzo, CID, stato iniziale e timestamp. L'organizzazione viene inizialmente impostata come attiva.
Operazione	<i>suspendOrganization</i>	suspendOrganization(address organizationAddress)	Sospende un'organizzazione attiva, aggiornandone lo stato e riducendo il numero di organizzazioni attive.

Categoria	Elemento	Tipo / Firma	Descrizione
Operazione	<i>revokeOrganization</i>	revokeOrganization(address organizationAddress)	Revoca un'organizzazione attiva o sospesa. Se l'organizzazione era attiva, il contatore delle organizzazioni attive viene decrementato.
Operazione	<i>reactivateOrganization</i>	reactivateOrganization(address organizationAddress)	Riattiva un'organizzazione sospesa, aggiornandone lo stato e incrementando il numero di organizzazioni attive.
Operazione	<i>isActiveOrganization</i>	isActiveOrganization(address organizationAddress)	Restituisce true se l'organizzazione indicata è attualmente attiva.
Operazione	<i>getOrganization</i>	getOrganization(address organizationAddress)	Restituisce il record completo associato a una specifica organizzazione registrata.
Operazione	<i>getOrganizationCount</i>	getOrganizationCount()	Restituisce il numero totale di organizzazioni presenti nell'array organizationList.
Operazione	<i>getOrganizationAt</i>	getOrganizationAt(uint256 index)	Restituisce l'indirizzo dell'organizzazione presente in una determinata posizione dell'array.
Evento	<i>OrganizationAdded</i>	event	Evento emesso quando una nuova organizzazione viene registrata.
Evento	<i>OrganizationSuspended</i>	event	Evento emesso quando un'organizzazione viene sospesa.
Evento	<i>OrganizationRevoked</i>	event	Evento emesso quando un'organizzazione viene revocata.
Evento	<i>OrganizationReactivated</i>	event	Evento emesso quando un'organizzazione sospesa viene riattivata.
Evento	<i>GovernanceContractSet</i>	event	Evento emesso quando viene impostato il contratto di governance.
Evento	<i>BootstrapLocked</i>	event	Evento emesso quando la fase di bootstrap viene bloccata.

Tabella 3 Riferimenti OrganizationRegistry

PolicyGovernance.sol

Il contratto *PolicyGovernance.sol* rappresenta il modulo di governance multi-organizzazione del sistema. Pur non corrispondendo direttamente a uno dei registri descritti nella soluzione, costituisce una componente centrale del prototipo, poiché implementa il meccanismo attraverso cui le policy non vengono modificate da un singolo amministratore, ma tramite un processo condiviso di proposta, voto, raggiungimento del quorum ed esecuzione.

La logica multi-authority, secondo cui le organizzazioni riconosciute partecipano alla definizione delle regole comuni del sistema, viene rappresentata tramite la struttura *PolicyProposal*. Tale struttura consente di modellare sia modifiche alle policy globali di accesso sia decisioni di governance relative alla composizione e alla configurazione del network. Le azioni supportate includono l'impostazione di permessi globali, l'aggiunta, sospensione, revoca o riattivazione di organizzazioni, la registrazione di policy di governance e la modifica della profondità massima delle deleghe.

Il processo di governance è articolato in tre fasi. La prima fase è la proposta: solo un'organizzazione attiva può creare una proposta, garantendo che il potere di iniziativa appartenga esclusivamente ai partecipanti riconosciuti dal sistema. La seconda fase è il voto: anche in questo caso, solo le organizzazioni attive possono votare e ogni organizzazione può esprimere un solo voto per ciascuna proposta; per gestire ciò, viene usato il mapping *hasVoted* che evita duplicazioni e riflette il requisito

secondo cui gli approvatori devono essere distinti e autorizzati. La terza fase è la finalizzazione: una proposta viene approvata o respinta solo se viene raggiunto il quorum previsto.

Il contratto distingue due soglie decisionali. Per le policy globali di accesso viene utilizzata la maggioranza semplice, calcolata come $\left(\frac{n}{2}\right) + 1$, dove n rappresenta il numero di organizzazioni attive. Per le decisioni più critiche, come quelle relative alla governance o alla gestione delle organizzazioni, viene invece richiesta una supermajority pari a $\text{ceil}\left(\frac{2n}{3}\right)$. Nel contratto tale valore è implementato come $\frac{(2n + 2)}{3}$, così da ottenere l'arrotondamento per eccesso nonostante la divisione intera di Solidity.

Quando una proposta viene approvata, *PolicyGovernance* esegue l'azione prevista mentre per le modifiche operative, il contratto chiama il registro competente: *PolicyRegistry* per i permessi globali e per la profondità massima delle deleghe, e *OrganizationRegistry* per l'aggiunta, sospensione, revoca o riattivazione delle organizzazioni. Nel caso della registrazione di una policy di governance, invece, si effettua l'approvazione della proposta e l'ancoraggio del relativo *policyCID*. Ad esempio, una modifica ai permessi globali viene applicata tramite *PolicyRegistry*, mentre l'aggiunta, la sospensione, la revoca o la riattivazione di un'organizzazione viene eseguita tramite *OrganizationRegistry*.

Inoltre, le fasi principali del processo, come la creazione della proposta, il voto, l'approvazione, il rigetto e l'esecuzione, vengono registrate tramite *AuditRegistry.sol*.

La seguente tabella riassume le principali componenti del contratto.

Categoria	Elemento	Tipo / Firma	Descrizione
Interfaccia	<i>IOrganizationRegistryForGovernance</i>	interface	Interfaccia verso <i>OrganizationRegistry.sol</i> . Permette di verificare se un'organizzazione è attiva, ottenere il numero di organizzazioni attive e gestire aggiunta, sospensione, revoca e riattivazione delle organizzazioni.
Interfaccia	<i>IPolicyRegistryForGovernance</i>	interface	Interfaccia verso <i>PolicyRegistry.sol</i> . Permette alla governance di aggiornare i permessi globali e la profondità massima di delega.
Interfaccia	<i>IAuditRegistryForGovernance</i>	interface	Interfaccia verso <i>AuditRegistry.sol</i> , usata per registrare eventi di audit relativi alle operazioni di governance.
Struct	<i>PolicyProposal</i>	struct	Struttura che rappresenta una proposta di governance. Contiene identificativo, tipo di azione, proponente, timestamp, stato, parametri della proposta, voti favorevoli, voti contrari ed eventuali dati per l'esecuzione.
Dati	<i>organizationRegistry</i>	<i>IOrganizationRegistryForGovernance public</i>	Riferimento al registro delle organizzazioni, utilizzato per verificare i soggetti abilitati e per eseguire modifiche sulle organizzazioni.
Dati	<i>policyRegistry</i>	<i>IPolicyRegistryForGovernance public</i>	Riferimento al registro delle policy, utilizzato per applicare le modifiche approvate dalla governance.
Dati	<i>auditRegistry</i>	<i>IAuditRegistryForGovernance public</i>	Riferimento al registro di audit, utilizzato per tracciare proposte, voti, approvazioni, rigetti ed esecuzioni.

Categoria	Elemento	Tipo / Firma	Descrizione
<i>Dati</i>	<i>nextProposalId</i>	uint256 public	Contatore usato per assegnare un identificativo progressivo alle nuove proposte.
<i>Dati</i>	<i>proposals</i>	mapping(uint256 => PolicyProposal) private	Mapping che associa l'identificativo di una proposta al relativo record.
<i>Dati</i>	<i>hasVoted</i>	mapping(uint256 => mapping(address => bool)) public	Mapping che impedisce a una stessa organizzazione di votare più volte sulla medesima proposta.
<i>Dati</i>	<i>approvers</i>	mapping(uint256 => address[]) private	Mapping che memorizza gli indirizzi delle organizzazioni che hanno votato a favore di una proposta.
<i>Dati</i>	<i>rejecters</i>	mapping(uint256 => address[]) private	Mapping che memorizza gli indirizzi delle organizzazioni che hanno votato contro una proposta.
<i>Modifier</i>	<i>onlyActiveOrganization</i>	modifier	Consente l'esecuzione della funzione solo agli indirizzi che risultano organizzazioni attive nel registro.
<i>Operazione</i>	<i>constructor</i>	constructor(address organizationRegistryAddress, address policyRegistryAddress, address auditRegistryAddress)	Inizializza il contratto collegandolo ai registri delle organizzazioni, delle policy e di audit. Imposta inoltre il primo identificativo di proposta.
<i>Operazione</i>	<i>proposeGlobalAccessPolicy</i>	proposeGlobalAccessPolicy(...)	Crea una proposta per modificare una policy globale di accesso, specificando ruolo, tipo di documento, azione, permesso e CID della policy.
<i>Operazione</i>	<i>proposeAddOrganization</i>	proposeAddOrganization(address organizationAddress, bytes32 organizationCID, bytes32 policyCID)	Crea una proposta per aggiungere una nuova organizzazione al registro.
<i>Operazione</i>	<i>proposeSuspendOrganization</i>	proposeSuspendOrganization(address organizationAddress, bytes32 policyCID)	Crea una proposta per sospendere temporaneamente un'organizzazione.
<i>Operazione</i>	<i>proposeRevokeOrganization</i>	proposeRevokeOrganization(address organizationAddress, bytes32 policyCID)	Crea una proposta per revocare un'organizzazione attiva o sospesa.
<i>Operazione</i>	<i>proposeReactivateOrganization</i>	proposeReactivateOrganization(address organizationAddress, bytes32 policyCID)	Crea una proposta per riattivare un'organizzazione sospesa.
<i>Operazione</i>	<i>proposeGovernancePolicy</i>	proposeGovernancePolicy(bytes32 policyCID)	Crea una proposta per registrare una policy di governance, ancorata tramite policyCID.
<i>Operazione</i>	<i>proposeSetMaxDelegationDepth</i>	proposeSetMaxDelegationDepth(uint256 newMaxDelegationDepth, bytes32 policyCID)	Crea una proposta per modificare la profondità massima di delega consentita dal sistema.
<i>Operazione</i>	<i>vote</i>	vote(uint256 proposalId, bool support)	Permette a un'organizzazione attiva di votare una proposta pendente. Il voto può essere favorevole o contrario e viene registrato anche nel registro di audit.
<i>Operazione</i>	<i>finalize</i>	finalize(uint256 proposalId)	Finalizza una proposta pendente. Se i voti favorevoli raggiungono il quorum, la proposta viene approvata ed eseguita; se lo raggiungono i voti contrari, viene respinta.

Categoria	Elemento	Tipo / Firma	Descrizione
Operazione	<i>getRequiredQuorum</i>	getRequiredQuorum(DataTypes.GovernanceActionType actionType)	Calcola il quorum richiesto in base al tipo di azione proposta. Le policy globali richiedono maggioranza semplice, mentre le altre azioni richiedono supermajority.
Operazione	<i>getProposal</i>	getProposal(uint256 proposalId)	Restituisce i dati completi relativi a una proposta esistente.
Operazione	<i>getApprovers</i>	getApprovers(uint256 proposalId)	Restituisce la lista degli indirizzi che hanno votato a favore di una proposta.
Operazione	<i>getRejecters</i>	getRejecters(uint256 proposalId)	Restituisce la lista degli indirizzi che hanno votato contro una proposta.
Funzione interna	<i>_createBaseProposal</i>	_createBaseProposal(DataTypes.GovernanceActionType actionType, bytes32 policyCID)	Crea la struttura base di una proposta, assegna l'identificativo, imposta lo stato iniziale e registra l'evento di audit.
Funzione interna	<i>_executeProposal</i>	_executeProposal(PolicyProposal memory proposal)	Esegue l'azione associata a una proposta approvata, chiamando il registro competente in base al tipo di proposta.
Evento	<i>PolicyProposalCreated</i>	event	Evento emesso quando viene creata una nuova proposta di governance.
Evento	<i>PolicyVoteCast</i>	event	Evento emesso quando un'organizzazione vota una proposta.
Evento	<i>PolicyProposalFinalized</i>	event	Evento emesso quando una proposta viene approvata ed eseguita oppure respinta.

Tabella 4 Riferimenti PolicyGovernance

PolicyRegistry.sol

Il contratto *PolicyRegistry* implementa il registro delle policy globali e dei ruoli applicativi del sistema. Per rappresentare la logiche delle policy di accesso globale, che stabiliscono quali ruoli possono eseguire determinate azioni su specifiche tipologie documentali, viene utilizzato un mapping, *permissions*, che **associa ruolo, tipo di documento e azione** e che restituisce un valore booleano per indicare se **l'operazione è consentita** o meno.

La **modifica dei permessi** definiti non può essere effettuata liberamente da un amministratore singolo, ma avviene esclusivamente tramite il contratto *PolicyGovernance*, dopo il **raggiungimento del quorum** previsto dal processo di governance multi-organizzazione, che nel caso in esame delle policy di accesso globale è la maggioranza semplice.

Il contratto non associa direttamente un address a un ruolo applicativo, ma utilizza il mapping *credentialTypeRoles* per **collegare un tipo di credenziale a uno specifico ruolo** del sistema; in questo modo, il ruolo operativo di un soggetto viene ricavato dalla credenziale presentata.

On-chain, il *PolicyRegistry* applica i controlli necessari per **collegare la credenziale alla logica di autorizzazione**. In particolare, tramite la funzione *canPerformWithCredential* riceve il DID del richiedente, la credenziale presentata, il tipo documentale e l'azione richiesta; a questo punto il contratto verifica prima che la **credenziale** presentata sia **valida** per il soggetto indicato tramite *CredentialStatusRegistry*; successivamente ricava il **tipo di credenziale**, lo associa al **ruolo applicativo** corrispondente e consulta il mapping *permissions* per determinare se **l'operazione è consentita dalla policy globale**. L'associazione tra tipo di credenziale e ruolo applicativo viene configurata tramite la funzione *setCredentialTypeRole*, che può essere eseguita solo dal *bootstrapAdmin* e solo prima del blocco della fase di bootstrap.

Inoltre, è presente anche la funzione *canRolePerform*, che consente di **verificare** direttamente **se un determinato ruolo può eseguire una certa azione** su uno specifico tipo documentale.

Un altro parametro rilevante è *maxDelegationDepth*, che definisce la **profondità massima ammessa** per le catene di delega. Il valore viene inizializzato a 1 e può essere aggiornato solo attraverso il contratto *PolicyGovernance*, tramite l'apposita funzione. In questo modo, il modello di delega non è lasciato a decisioni locali, ma rimane soggetto a regole comuni definite dal sistema. Inoltre, il contratto prevede un **limite massimo assoluto**, *ABSOLUTE_MAX_DELEGATION_DEPTH*, pari a 10, che impedisce alla governance di impostare una profondità eccessivamente elevata.

Così come in altri contratti, si distingue tra **configurazioni iniziali di bootstrap** e **modifiche ordinarie** soggette a governance. Il *bootstrapAdmin* può impostare la configurazione iniziale, ma una volta bloccato il bootstrap, non può più modificare tali impostazioni, bensì tutto passerà dal contratto *PolicyGovernance*.

Il contratto emette inoltre **eventi Solidity specifici**, come *MaxDelegationDepthUpdated* e *BootstrapLocked*, e interagisce con *AuditRegistry* per registrare le modifiche rilevanti tramite **eventi di audit**.

La seguente tabella riassume le principali componenti del contratto:

Categoria	Elemento	Tipo / Firma	Descrizione
Interfaccia	<i>IOrganizationRegistryForPolicies</i>	interface	Interfaccia verso OrganizationRegistry.sol, utilizzata per verificare se un address corrisponde a un'organizzazione attiva.
Interfaccia	<i>ICredentialStatusRegistryForPolicies</i>	interface	Interfaccia verso il registro delle credenziali, usata per verificare se una credenziale è valida per un subject DID e per ottenere il tipo della credenziale.
Interfaccia	<i>IAuditRegistryForPolicies</i>	interface	Interfaccia verso AuditRegistry.sol, utilizzata per registrare eventi di audit relativi a ruoli e policy.
Dati	<i>bootstrapAdmin</i>	address public immutable	Indirizzo dell'account che effettua il deploy del contratto. Ha il compito di configurare il sistema nella fase iniziale di bootstrap.
Dati	<i>governanceContract</i>	address public	Indirizzo del contratto PolicyGovernance.sol, l'unico autorizzato a modificare i permessi globali e la profondità massima delle deleghe.
Dati	<i>bootstrapLocked</i>	bool public	Indica se la fase di bootstrap è stata bloccata. Dopo il blocco non è più possibile modificare il contratto di governance.
Dati	<i>maxDelegationDepth</i>	uint256 public	Profondità massima delle deleghe consentita dal sistema. Il valore iniziale è pari a 1.
Dati	<i>ABSOLUTE_MAX_DELEGATION_DEPTH</i>	uint256 public constant	Limite massimo assoluto della profondità di delega, impostato a 10.
Dati	<i>organizationRegistry</i>	IOrganizationRegistryForPolicies public	Riferimento al registro delle organizzazioni, usato per controllare che chi assegna o revoca ruoli sia un'organizzazione attiva.
Dati	<i>credentialRegistry</i>	ICredentialStatusRegistryForPolicies public	Riferimento al registro delle credenziali, usato per validare una credenziale e recuperarne il tipo.

Categoria	Elemento	Tipo / Firma	Descrizione
<i>Dati</i>	<i>auditRegistry</i>	IAuditRegistryForPolicies public	Riferimento al registro di audit, usato per tracciare le operazioni critiche.
<i>Dati</i>	<i>credentialTypeRoles</i>	mapping(bytes32 => DataTypes.Role) private	Mapping che associa un tipo di credenziale VC a un ruolo applicativo.
<i>Dati</i>	<i>permissions</i>	mapping(Role => mapping(bytes32 => mapping(bytes32 => bool)))	Mapping che definisce se un determinato ruolo può eseguire una certa azione su uno specifico tipo di documento.
<i>Modifier</i>	<i>onlyBootstrapAdmin</i>	modifier	Consente l'esecuzione della funzione solo al bootstrap admin.
<i>Modifier</i>	<i>onlyGovernance</i>	modifier	Consente l'esecuzione della funzione solo al contratto di governance.
<i>Operazione</i>	<i>constructor</i>	constructor(address organizationRegistryAddress, address auditRegistryAddress)	Inizializza il contratto impostando il bootstrap admin, la profondità iniziale di delega e i riferimenti ai registri delle organizzazioni e di audit.
<i>Operazione</i>	<i>setGovernanceContract</i>	setGovernanceContract(address governanceContractAddress)	Imposta l'indirizzo del contratto di governance durante la fase di bootstrap.
<i>Operazione</i>	<i>lockBootstrap</i>	lockBootstrap()	Blocca definitivamente la fase di bootstrap, dopo che il contratto di governance è stato configurato.
<i>Operazione</i>	<i>setCredentialTypeRole</i>	setCredentialTypeRole(bytes32 credentialType, DataTypes.Role role) external	Configura, durante il bootstrap, l'associazione tra un tipo di credenziale e un ruolo applicativo.
<i>Operazione</i>	<i>setPermissionFromGovernance</i>	setPermissionFromGovernance(DataTypes.Role role, bytes32 documentType, bytes32 action, bool allowed, bytes32 policyCID)	Permette solo alla governance di modificare una policy globale di accesso.
<i>Operazione</i>	<i>setMaxDelegationDepthFromGovernance</i>	setMaxDelegationDepthFromGovernance(uint256 newMaxDelegationDepth, bytes32 policyCID)	Permette solo alla governance di aggiornare la profondità massima delle deleghe, rispettando il limite assoluto previsto.
<i>Operazione</i>	<i>getCredentialTypeRole</i>	getCredentialTypeRole(bytes32 credentialType) external view returns (DataTypes.Role)	Restituisce il ruolo applicativo associato a uno specifico tipo di credenziale.
<i>Operazione</i>	<i>canRolePerform</i>	canRolePerform(DataTypes.Role role, bytes32 documentType, bytes32 action) external view returns (bool)	Verifica direttamente se un ruolo può eseguire una certa azione su un determinato tipo di documento.
<i>Operazione</i>	<i>canPerformWithCredential</i>	canPerformWithCredential(bytes32 requesterDID, DataTypes.PresentedCredential calldata presentedCredential, bytes32 documentType, bytes32 action) public view returns (bool)	Verifica se un soggetto può eseguire una certa azione usando una credenziale valida associata a un ruolo autorizzato.
<i>Evento</i>	<i>CredentialTypeRoleSet</i>	event	Evento emesso quando viene configurata l'associazione tra tipo di credenziale e ruolo applicativo.
<i>Evento</i>	<i>PermissionSet</i>	event	Evento emesso quando la governance modifica un permesso globale.

Categoria	Elemento	Tipo / Firma	Descrizione
Evento	<i>GovernanceContractSet</i>	event	Evento emesso quando viene impostato il contratto di governance.
Evento	<i>BootstrapLocked</i>	event	Evento emesso quando la fase di bootstrap viene bloccata.
Evento	<i>MaxDelegationDepthUpdated</i>	event	Evento emesso quando viene aggiornata la profondità massima delle deleghe.

Tabella 5 Riferimenti PolicyRegistry

DocumentLifecycleRegistry.sol

Il contratto *DocumentLifecycleRegistry.sol* implementa il registro del ciclo di vita documentale. Nel modello progettuale, questo registro ha il compito di mantenere lo stato dei documenti clinici, le versioni, i riferimenti allo storage off-chain e le informazioni necessarie per una successiva verifica. Nell'implementazione del prototipo viene mantenuta la stessa impostazione logica, semplificando però la gestione del contenuto documentale.

Un aspetto importante del contratto è l'integrazione con l'IdentityRegistry. Prima di creare, aggiornare o revocare un documento, il contratto verifica che il DID del soggetto coinvolto sia attivo e che l'address che invoca la funzione sia effettivamente il controller del DID dichiarato, per mantenere un collegamento tra l'identità decentralizzata rappresentata dal DID e l'address *Ethereum* che esegue l'operazione.

Per quanto concerne la rappresentazione dei documenti, ogni documento è rappresentato tramite la struttura *DocumentRecord*, che contiene le informazioni necessarie a identificare il documento, associarlo a un paziente, indicarne il creatore, definirne il tipo documentale, registrarne lo stato e gestirne il versionamento. Il campo CID mantiene il significato previsto nel modello ed è utilizzato come riferimento IPFS al contenuto documentale memorizzato off-chain.

Le operazioni principali sul documento sono soggette al controllo delle policy globali definite in *PolicyRegistry*. Nel processo di creazione di un documento viene verificato che il *patientDID* e il *creatorDID* siano attivi, mentre sul *creatorDID* si verifica che *msg.sender* sia il controller del *creatorDID*, cioè sul soggetto che sta creando il documento. Inoltre, è richiesto che il *creatorDID*, tramite la credenziale presentata, risulti autorizzato dalla policy globale all'azione CREATE per il tipo documentale indicato.

Quando un documento viene creato, la prima versione viene registrata nello stato *Certified*, che rappresenta la versione corrente e attiva del documento. Nel caso di aggiornamento del documento, viene richiesto il permesso di *UPDATE*.

La revoca del documento corrente può essere effettuata da un soggetto autorizzato dalla policy globale all'azione *REVOKE*; Il DID del richiedente deve essere attivo e controllato dall'address che invoca la funzione. L'autorizzazione viene verificata tramite la credenziale presentata e la funzione *canPerformWithCredential* di *PolicyRegistry*. In caso di revoca, la versione corrente passa allo stato *Revoked* e non viene più considerata attiva.

Il contratto fornisce inoltre funzioni di consultazione e verifica, utili agli altri componenti del sistema per controllare lo stato dei documenti prima di autorizzare operazioni successive. In particolare, funzioni come *verifyCID*, *isDocumentActive*, *getPatientDID*, *getDocumentType* e *getCurrentCID* permettono di verificare l'integrità del riferimento documentale, lo stato corrente del documento e le informazioni necessarie per l'applicazione delle policy.

Tutte le operazioni critiche, come creazione, aggiornamento e revoca, vengono infine registrate tramite *AuditRegistry.sol*, garantendo la tracciabilità del ciclo di vita documentale.

La seguente tabella riassume le principali componenti del contratto:

Categoria	Elemento	Tipo / Firma	Descrizione
Interfaccia	<i>IIidentityRegistryForDocuments</i>	interface	Interfaccia verso IdentityRegistry.sol, usata per verificare se un DID è attivo e per ottenere l'address controller associato a un DID.
Interfaccia	<i>IPolicyRegistryForDocuments</i>	interface	Interfaccia verso PolicyRegistry.sol, utilizzata per verificare se un utente può eseguire una determinata azione su uno specifico tipo documentale.
Interfaccia	<i>IAuditRegistryForDocuments</i>	interface	Interfaccia verso AuditRegistry.sol, utilizzata per registrare eventi di audit relativi alle operazioni documentali.
Struct	<i>DocumentRecord</i>	struct	Struttura che rappresenta una versione di un documento. Contiene identificativo, paziente, creatore, tipo documentale, CID, stato, numero di versione, riferimento alla versione precedente e timestamp.
Dati	<i>identityRegistry</i>	IIidentityRegistryForDocuments public	Riferimento al registro delle identità, usato per validare DID attivi e controllare il controller del DID.
Dati	<i>policyRegistry</i>	IPolicyRegistryForDocuments public	Riferimento al registro delle policy, usato per verificare i permessi sulle operazioni documentali.
Dati	<i>auditRegistry</i>	IAuditRegistryForDocuments public	Riferimento al registro di audit, usato per tracciare le operazioni critiche.
Dati	<i>documentExists</i>	mapping(bytes32 => bool)	Mapping che indica se un documento con un determinato documentId è già stato registrato.
Dati	<i>currentVersion</i>	mapping(bytes32 => uint256)	Mapping che associa a ogni documento il numero della versione corrente.
Dati	<i>documentVersions</i>	mapping(bytes32 => mapping(uint256 => DocumentRecord))	Mapping che associa a ogni documento e a ogni numero di versione il relativo record documentale.
Operazione	<i>constructor</i>	constructor(address policyRegistryAddress, address auditRegistryAddress)	Inizializza il contratto impostando i riferimenti al registro delle policy e al registro di audit.
Operazione	<i>createDocument</i>	createDocument(bytes32 documentId, address patientDID, bytes32 creatorDID, DataTypes.PresentedCredential calldata presentedCredential, bytes32 documentType, string calldata CID)	Registra un nuovo documento nella sua prima versione. L'operazione è consentita solo se il chiamante è autorizzato dalla policy a eseguire l'azione di creazione sul tipo documentale indicato.
Operazione	<i>createNewVersion</i>	createNewVersion(bytes32 documentId, bytes32 newCID, DataTypes.PresentedCredential calldata presentedCredential, string calldata newCID)	Crea una nuova versione di un documento esistente. La versione corrente viene archiviata e la nuova versione diventa quella certificata.
Operazione	<i>revokeDocument</i>	revokeDocument(bytes32 documentId, bytes32 requesterDID, DataTypes.PresentedCredential calldata presentedCredential)	Revoca la versione corrente del documento. L'operazione può essere eseguita dal paziente associato al

Categoria	Elemento	Tipo / Firma	Descrizione
			documento oppure da un soggetto autorizzato dalla policy.
Operazione	<i>getCurrentDocument</i>	getCurrentDocument(bytes32 documentId)	Restituisce il record relativo alla versione corrente di un documento.
Operazione	<i>getDocumentVersion</i>	getDocumentVersion(bytes32 documentId, uint256 version)	Restituisce il record relativo a una specifica versione di un documento.
Operazione	<i>verifyCID</i>	verifyCID(bytes32 documentId, uint256 version, bytes32 CID)	Verifica se il CID fornito corrisponde al CID registrato per una determinata versione del documento.
Operazione	<i>isDocumentActive</i>	isDocumentActive(bytes32 documentId)	Restituisce true se la versione corrente del documento è nello stato Certified.
Operazione	<i>getPatientDID</i>	getPatientDID(bytes32 documentId)	Restituisce l'indirizzo del paziente associato alla versione corrente del documento.
Operazione	<i>getDocumentType</i>	getDocumentType(bytes32 documentId)	Restituisce il tipo documentale associato alla versione corrente del documento.
Operazione	<i>getCurrentCID</i>	getCurrentCID(bytes32 documentId)	Restituisce il CID associato alla versione corrente del documento.
Evento	<i>DocumentCreated</i>	event	Evento emesso quando viene registrato un nuovo documento.
Evento	<i>DocumentVersionCreated</i>	event	Evento emesso quando viene creata una nuova versione di un documento.
Evento	<i>DocumentRevoked</i>	event	Evento emesso quando la versione corrente di un documento viene revocata.

Tabella 6 Riferimenti DocumentLifecycleRegistry

PatientDrivenPolicyRegistry.sol

Il contratto *PatientDrivenPolicyRegistry.sol* implementa il registro delle policy definite dal paziente. In generale, come definito nel WP2, le policy patient-driven non possano ampliare i permessi globali stabiliti dal sistema, ma solo restringerli.

Una restrizione può limitare l'accesso a un documento in base al soggetto richiedente, all'azione richiesta, alla finalità dichiarata o a una combinazione di questi elementi. Ogni restrizione è rappresentata dalla struttura *PatientPolicy*, che contiene le informazioni necessarie per identificare la policy, associarla al paziente e al documento target, definirne l'ambito di applicazione, stabilirne il periodo di validità e registrarne l'eventuale revoca.

La funzione *registerRestriction* può essere chiamata solo dall'address che risulta controller del *patientDID* associato al documento. Prima di registrare la restrizione, il contratto verifica che il documento esista, che sia attivo e che il chiamante coincida con il *patientDID* salvato nel *DocumentLifecycleRegistry.sol*. Questo controllo impedisce a un soggetto di definire restrizioni su documenti appartenenti ad altri pazienti.

Le funzioni più importanti per l'integrazione con il resto del sistema sono *isRestricted* e *isAllowed*. La prima consente all'*AccessController* di verificare la presenza di restrizioni patient-driven applicabili, mentre la seconda permette di verificare eventuali autorizzazioni positive espresse dal paziente. Entrambe vengono chiamate dall'*AccessController* durante la valutazione di una richiesta di accesso per controllare se, per il documento indicato, esista una restrizione patient-driven non revocata, temporalmente valida e compatibile con requester, azione e finalità della richiesta. Se esiste una

restrizione valida, non revocata e temporalmente attiva, l'accesso viene negato. In questo modo, le policy patient-driven agiscono come un ulteriore livello di controllo rispetto alle policy globali.

Il contratto supporta anche una logica di wildcard. Il contratto supporta anche una logica di wildcard. Per le restrizioni, se *restrictedUserDID* è pari a *bytes32(0)*, la restrizione vale per tutti gli utenti; se *restrictedAction* o *purpose* sono pari a *bytes32(0)*, la restrizione vale rispettivamente per qualsiasi azione o qualsiasi finalità. In modo analogo, per le autorizzazioni positive, *allowedUserDID* e *allowedAction* possono essere usati per definire allowance puntuali o più generali.

Questa scelta consente di rappresentare restrizioni sia puntuali sia più generali, mantenendo comunque la logica del prototipo semplice.

Le operazioni di registrazione e revoca delle restrizioni vengono infine tracciate tramite *AuditRegistry.sol*, garantendo osservabilità e verificabilità delle decisioni espresse dal paziente.

La seguente tabella riassume le principali componenti del contratto:

Categoria	Elemento	Tipo / Firma	Descrizione
Interfaccia	<code>IIdentityRegistryForPatientPolicies</code>	interface	Interfaccia verso <i>IdentityRegistry.sol</i> , usata per verificare se un DID è attivo e per recuperare il controller associato al DID.
Interfaccia	<code>IDocumentLifecycleRegistryForPatientPolicies</code>	interface	Interfaccia verso <i>DocumentLifecycleRegistry.sol</i> , utilizzata per verificare l'esistenza e lo stato del documento e per recuperare il paziente associato.
Interfaccia	<code>IAuditRegistryForPatientPolicies</code>	interface	Interfaccia verso <i>AuditRegistry.sol</i> , utilizzata per registrare eventi di audit relativi alle policy patient-driven.
Struct	<code>PatientPolicy</code>	struct	Struttura che rappresenta una policy definita dal paziente. Contiene identificativo, paziente, documento target, utente ristretto, azione, finalità, periodo di validità, CID della policy e informazioni di revoca.
Dati	<code>identityRegistry</code>	<code>IIdentityRegistryForPatientPolicies public</code>	Riferimento al registro delle identità, usato per verificare DID attivi e controller dei DID.
Dati	<code>documentRegistry</code>	<code>IDocumentLifecycleRegistryForPatientPolicies public</code>	Riferimento al registro documentale, usato per controllare che il documento esista, sia attivo e appartenga al paziente chiamante.
Dati	<code>auditRegistry</code>	<code>IAuditRegistryForPatientPolicies public</code>	Riferimento al registro di audit, usato per tracciare registrazione e revoca delle policy.
Dati	<code>nextPolicyId</code>	<code>uint256 public</code>	Contatore progressivo usato per assegnare un identificativo univoco alle nuove policy.
Dati	<code>policies</code>	<code>mapping(uint256 => PatientPolicy)</code>	Mapping che associa l'identificativo di una policy al relativo record.
Dati	<code>policyExists</code>	<code>mapping(uint256 => bool)</code>	Mapping che indica se una determinata policy è stata registrata.
Dati	<code>policiesByDocument</code>	<code>mapping(bytes32 => uint256[])</code>	Mapping che associa a ogni documento la lista degli identificativi delle policy patient-driven collegate.
Operazione	<code>constructor</code>	<code>constructor(address documentRegistryAddress, address auditRegistryAddress, address identityRegistryAddress)</code>	Inizializza il contratto impostando i riferimenti al registro documentale e al registro di audit.

Operazione	registerRestriction	registerRestriction(bytes32 targetDocumentId, address restrictedUserDID, bytes32 restrictedAction, bytes32 purpose, uint256 validFrom, uint256 validUntil, bytes32 policyCID)	Registra una nuova restrizione patient-driven su uno specifico documento. Può essere eseguita solo dal paziente associato al documento.
Operazione	revokePatientPolicy	revokePatientPolicy(uint256 policyId)	Revoca una policy patient-driven esistente. Può essere eseguita solo dal paziente che ha creato la policy.
Operazione	isRestricted	isRestricted(address requesterDID, bytes32 documentId, bytes32 action, bytes32 purpose)	Verifica se una richiesta è soggetta a una restrizione patient-driven valida e non revocata.
Operazione	getPatientPolicy	getPatientPolicy(uint256 policyId)	Restituisce il record completo relativo a una specifica policy.
Operazione	getPoliciesByDocument	getPoliciesByDocument(bytes32 documentId)	Restituisce gli identificativi delle policy associate a un determinato documento.
Funzione interna	_matches	_matches(PatientPolicy memory policy, address requesterDID, bytes32 action, bytes32 purpose)	Verifica se una policy è valida, non revocata e applicabile al richiedente, all'azione e alla finalità indicati.
Evento	PatientPolicyRegistered	event	Evento emesso quando viene registrata una nuova restrizione patient-driven.
Evento	PatientPolicyRevoked	event	Evento emesso quando una policy patient-driven viene revocata.

Tabella 7 Riferimenti PatientDrivenPolicyRegistry

DelegationRegistry.sol

Il contratto *DelegationRegistry.sol* implementa il registro delle deleghe. Nel modello progettuale, la delega rappresenta un diritto derivato, temporaneo, limitato, revocabile e verificabile, attraverso cui un soggetto può trasferire a un altro soggetto la possibilità di eseguire una specifica azione su un determinato documento, per una finalità dichiarata.

Nel prototipo, questa logica viene rappresentata tramite due strutture principali: *DelegationProposal* e *Delegation*. La separazione tra proposta e delega attiva è fondamentale: un utente autorizzato può proporre una delega tramite *proposeDelegation*, ma solo se il suo *delegatorDID* è attivo, se l'address chiamante ne è il controller e se, tramite la credenziale presentata, risulta autorizzato dalla policy globale all'azione di delega per il tipo documentale del documento target. Tale delega non diventa immediatamente valida, bensì è effettivamente utilizzabile solo dopo l'approvazione esplicita del paziente tramite *approveDelegationProposal*.

Questa scelta è coerente con il modello descritto, poiché mantiene il paziente come soggetto di controllo finale sul conferimento di diritti derivati relativi ai propri documenti. Il delegante può quindi proporre una delega, ma non può renderla autonomamente efficace senza il consenso del paziente associato al documento.

Ogni proposta di delega contiene le informazioni necessarie a identificare il delegante, il delegato, il documento target, l'azione delegata, la finalità, l'intervallo temporale di validità, l'eventuale delega padre, la profondità della delega e lo stato della proposta. Quando il paziente approva la proposta, il

contratto crea un record di delega attiva, che viene indicizzato tramite una chiave calcolata su delegato, documento e azione. Tale indicizzazione consente di recuperare le deleghe potenzialmente applicabili durante la valutazione di una richiesta di accesso.

Il contratto consente inoltre al paziente di rifiutare una proposta tramite *rejectDelegationProposal* e prevede la possibilità di revocare una delega tramite *revokeDelegation*. La revoca può essere effettuata dal controller del DID del delegante, dal controller del DID del delegato oppure dal controller del DID del paziente associato al documento.

La funzione *isDelegationValid* rappresenta il principale punto di verifica del registro. Essa controlla se esiste una delega valida per un determinato delegato, documento, azione e finalità, tenendo conto dell'intervallo temporale di validità, dello stato di revoca, dello stato attivo del documento e della profondità massima consentita dal *PolicyRegistry* in quel momento.

Infine, il contratto implementa anche la possibilità di creare deleghe derivate tramite *proposeDerivedDelegation*. Una delega derivata può essere proposta solo dal delegato della delega padre, deve rispettare l'intervallo temporale della delega originaria e non può superare la profondità massima stabilita nel registro delle policy. La validità della catena di deleghe viene verificata ricorsivamente, in modo che una delega derivata non sia considerata valida se una delega precedente nella catena non è più valida.

Le operazioni principali, come proposta, approvazione, rifiuto, creazione e revoca delle deleghe, vengono tracciate tramite *AuditRegistry.sol*, garantendo osservabilità e verificabilità del processo.

La seguente tabella riassume le principali componenti del contratto:

Categoria	Elemento	Tipo / Firma	Descrizione
Interfaccia	<i>IdentityRegistryForDelegations</i>	interface	Interfaccia verso <i>IdentityRegistry.sol</i> , usata per verificare se un DID è attivo e per recuperare il controller associato al DID.
Interfaccia	<i>DocumentLifecycleRegistryForDelegations</i>	interface	Interfaccia verso <i>DocumentLifecycleRegistry.sol</i> , usata per verificare esistenza, stato, paziente e tipo documentale del documento.
Interfaccia	<i>PolicyRegistryForDelegations</i>	interface	Interfaccia verso <i>PolicyRegistry.sol</i> , usata per verificare i permessi tramite credenziale presentata e recuperare la profondità massima di delega.
Interfaccia	<i>AuditRegistryForDelegations</i>	interface	Interfaccia verso <i>AuditRegistry.sol</i> , usata per registrare eventi di audit relativi alle deleghe.
Struct	<i>DelegationProposal</i>	struct	Rappresenta una proposta di delega non ancora approvata o rifiutata dal paziente. Contiene identificativo proposta, delegante, delegato, documento, azione, finalità, eventuale delega padre, profondità, profondità massima al momento della creazione, validità temporale, stato, timestamp di creazione e riferimento alla delega eventualmente creata.
Struct	<i>Delegation</i>	struct	Rappresenta una delega approvata e quindi potenzialmente utilizzabile. Contiene identificativo delega,

Categoria	Elemento	Tipo / Firma	Descrizione
			proposta originaria, delegante, delegato, documento, azione, finalità, eventuale delega padre, profondità, profondità massima al momento della creazione, validità temporale, stato di revoca e timestamp di revoca.
<i>Dati</i>	identityRegistry	IdentityRegistryForDelegations public	Riferimento al registro delle identità, usato per validare DID attivi e controllare il controller dei DID.
<i>Dati</i>	documentRegistry	IDocumentLifecycleRegistryForDelegations public	Riferimento al registro documentale, usato per controllare che il documento esista, sia attivo e per recuperare paziente e tipo documentale.
<i>Dati</i>	policyRegistry	IPolicyRegistryForDelegations public	Riferimento al registro delle policy, usato per verificare se un soggetto può proporre deleghe e per leggere la profondità massima consentita.
<i>Dati</i>	auditRegistry	IAuditRegistryForDelegations public	Riferimento al registro di audit, usato per tracciare le operazioni critiche sulle deleghe.
<i>Dati</i>	nextProposalId	uint256 public	Contatore progressivo usato per assegnare un identificativo univoco alle nuove proposte di delega.
<i>Dati</i>	nextDelegationId	uint256 public	Contatore progressivo usato per assegnare un identificativo univoco alle deleghe approvate.
<i>Dati</i>	proposals	mapping(uint256 => DelegationProposal) private	Mapping che associa l'identificativo di una proposta al relativo record.
<i>Dati</i>	proposalExists	mapping(uint256 => bool) public	Mapping che indica se una proposta di delega esiste.
<i>Dati</i>	delegations	mapping(uint256 => Delegation) private	Mapping che associa l'identificativo di una delega al relativo record.
<i>Dati</i>	delegationExists	mapping(uint256 => bool) public	Mapping che indica se una delega esiste.
<i>Dati</i>	delegationsByKey	mapping(bytes32 => uint256[]) private	Mapping che associa una chiave, calcolata su delegato, documento e azione, alla lista delle deleghe corrispondenti.
<i>Operazione</i>	constructor	constructor(address identityRegistryAddress, address documentRegistryAddress, address policyRegistryAddress, address auditRegistryAddress)	Inizializza il contratto impostando i riferimenti al registro delle identità, al registro documentale, al registro delle policy e al registro di audit.
<i>Operazione</i>	proposeDelegation	proposeDelegation(bytes32 delegatorDID, bytes32 delegateDID, DataTypes.PresentedCredential calldata presentedCredential, bytes32 targetDocumentId, bytes32 delegatedAction, bytes32 purposeOfDelegation, uint256 validFrom, uint256 validUntil) external returns (uint256)	Crea una proposta di delega diretta. Può essere eseguita solo dal controller del DID delegante, che deve presentare una credenziale valida per l'azione DELEGATE sul tipo documentale del documento.
<i>Operazione</i>	proposeDerivedDelegation	proposeDerivedDelegation(uint256 parentDelegationId, bytes32 newDelegateDID,	Crea una proposta di delega derivata a partire da una delega padre valida. Può essere eseguita solo dal controller

Categoria	Elemento	Tipo / Firma	Descrizione
		uint256 validFrom, uint256 validUntil) external returns (uint256)	del delegato della delega padre e deve rispettare validità temporale e profondità massima.
Operazione	approveDelegationProposal	approveDelegationProposal(uint256 proposalId) external returns (uint256)	Permette al paziente associato al documento di approvare una proposta di delega. L'approvazione genera una delega effettiva e restituisce il relativo delegationId.
Operazione	rejectDelegationProposal	rejectDelegationProposal(uint256 proposalId) external	Permette al paziente associato al documento di rifiutare una proposta di delega ancora pendente.
Operazione	revokeDelegation	revokeDelegation(uint256 delegationId, bytes32 revokedByDID) external	Revoca una delega esistente. Può essere chiamata dal controller del DID indicato in revokedByDID, se tale DID corrisponde al delegante, al delegato o al paziente associato al documento.
Operazione	isDelegationValid	isDelegationValid(bytes32 delegateDID, bytes32 documentId, bytes32 action, bytes32 purpose) external view returns (bool)	Verifica se esiste una delega valida per il delegato, il documento, l'azione e la finalità indicati.
Operazione	getDelegationProposal	getDelegationProposal(uint256 proposalId) external view returns (DelegationProposal memory)	Restituisce il record completo di una proposta di delega esistente.
Operazione	getDelegation	getDelegation(uint256 delegationId) external view returns (Delegation memory)	Restituisce il record completo di una delega approvata esistente.
Operazione	getDelegationsByKey	getDelegationsByKey(bytes32 delegateDID, bytes32 documentId, bytes32 action) external view returns (uint256[] memory)	Restituisce gli identificativi delle deleghe associate a un determinato delegato, documento e azione.
Funzione interna	_delegationKey	_delegationKey(bytes32 delegateDID, bytes32 documentId, bytes32 action) internal pure returns (bytes32)	Calcola la chiave utilizzata per indicizzare le deleghe in base a delegato, documento e azione.
Funzione interna	_isDelegationCurrentlyValid	_isDelegationCurrentlyValid(Delegation memory delegationRecord, bytes32 purpose) internal view returns (bool)	Verifica se una singola delega è valida rispetto a revoca, intervallo temporale, finalità, profondità massima, stato del documento e stato attivo del delegate DID.
Funzione interna	_isDelegationChainValid	_isDelegationChainValid(Delegation memory delegationRecord, bytes32 purpose) internal view returns (bool)	Verifica ricorsivamente la validità della catena di deleghe, controllando anche le eventuali deleghe padre.
Evento	DelegationProposed	event DelegationProposed(uint256 indexed proposalId, bytes32 indexed delegatorDID, bytes32 indexed delegateDID, bytes32 targetDocumentId, bytes32 delegatedAction, bytes32 purposeOfDelegation, uint256 validFrom, uint256 validUntil)	Evento emesso quando viene proposta una nuova delega.
Evento	DelegationProposalApproved	event DelegationProposalApproved(uint256 indexed proposalId, uint256 indexed delegationId, bytes32 indexed patientDID)	Evento emesso quando il paziente approva una proposta di delega e viene creata la delega effettiva.
Evento	DelegationProposalRejected	event DelegationProposalRejected(uint256 indexed proposalId, bytes32 indexed patientDID)	Evento emesso quando il paziente rifiuta una proposta di delega.
Evento	DelegationRevoked	event DelegationRevoked(uint256 indexed delegationId, bytes32 indexed revokedByDID, address indexed revokedByController, uint256 revokedAt)	Evento emesso quando una delega viene revocata. Include il DID che revoca, il relativo controller e il timestamp di revoca.

AccessController.sol

Il contratto *AccessController.sol* rappresenta il Policy Engine del prototipo, seppur in forma semplificata. Il suo ruolo è valutare le richieste di accesso ai documenti combinando le informazioni provenienti dai principali registri del sistema: *DocumentLifecycleRegistry.sol*, *PolicyRegistry.sol*, *DelegationRegistry.sol*, *PatientDrivenPolicyRegistry.sol* e *IdentityRegistry.sol*. A differenza degli altri contratti, non gestisce direttamente documenti, ruoli, deleghe o restrizioni, ma coordina tali componenti per produrre una decisione finale di accesso.

La funzione centrale del contratto è *_canAccessWithReason*, che applica un ordine di valutazione preciso e restituisce sia l'esito booleano della richiesta sia il codice motivazionale associato. In primo luogo, viene verificata la validità del richiedente e l'esistenza del documento. Se il documento non esiste, l'accesso viene negato con il relativo reason code. Successivamente viene controllato che il documento sia attivo; in caso contrario, la richiesta viene respinta perché il documento non si trova in uno stato valido per l'accesso.

Dopo le verifiche iniziali sul documento, il contratto controlla la presenza di eventuali restrizioni patient-driven. Se esiste una restrizione valida e applicabile al richiedente, al documento, all'azione o alla finalità indicata, l'accesso viene negato. In questo modo, le policy definite dal paziente agiscono come livello restrittivo rispetto alle altre forme di autorizzazione.

In assenza di restrizioni applicabili, il contratto verifica se il richiedente coincide con il paziente associato al documento e se l'azione richiesta è *READ*. In tal caso, l'accesso viene consentito in quanto il paziente è il soggetto direttamente collegato al documento. Se invece il richiedente non è il paziente, viene verificata l'eventuale presenza di una delega valida tramite *DelegationRegistry.sol*. Qualora esista una delega compatibile con documento, azione e finalità, l'accesso viene autorizzato.

Solo dopo questi controlli viene consultato *PolicyRegistry*, che verifica se il *requesterDID*, tramite la credenziale presentata, risulta autorizzato dalla policy globale a eseguire l'azione richiesta sul tipo documentale considerato. Se anche questa condizione non è soddisfatta, l'accesso viene negato perché nessuna delle regole autorizzative previste dal sistema risulta applicabile.

L'ordine di valutazione implementato può quindi essere sintetizzato come segue:

1. verifica della validità del richiedente, dell'esistenza e dello stato del documento;
2. controllo delle restrizioni patient-driven;
3. accesso del paziente associato al documento;
4. verifica delle deleghe valide;
5. verifica delle policy globali.

Il contratto espone due modalità di valutazione. La funzione *canAccess* consente una verifica in sola lettura, senza produrre eventi o modificare lo stato. La funzione *requestAccess*, invece, rappresenta la richiesta di accesso tracciata: prima della valutazione verifica che *msg.sender* sia il controller del *requesterDID* e poi esegue la stessa logica autorizzativa. L'esito viene registrato tramite l'evento *AccessRequested* e tramite *AuditRegistry.sol*, rendendo tracciabili sia gli accessi concessi sia quelli negati.

È importante sottolineare che *AccessController.sol* non recupera né decifra il documento. Il suo compito è esclusivamente stabilire se l'accesso sia autorizzato. Questa scelta è coerente con il modello progettuale, in cui la decisione autorizzativa, il recupero del payload off-chain e la gestione delle chiavi di cifratura rimangono componenti concettualmente distinte.

La seguente tabella riassume le principali componenti del contratto.

Ecco la **tabella aggiornata coerente con il codice**:

Categoria	Elemento	Tipo / Firma	Descrizione
<i>Interfaccia</i>	IdentityRegistryForAccess	interface	Interfaccia verso IdentityRegistry.sol, usata per verificare se un DID è attivo e per recuperare il controller associato al DID.
<i>Interfaccia</i>	IDocumentLifecycleRegistryForAccess	interface	Interfaccia verso DocumentLifecycleRegistry.sol, utilizzata per verificare esistenza, stato, paziente associato e tipo documentale del documento.
<i>Interfaccia</i>	IPolicyRegistryForAccess	interface	Interfaccia verso PolicyRegistry.sol, utilizzata per verificare se un soggetto può eseguire una determinata azione su uno specifico tipo documentale tramite credenziale presentata.
<i>Interfaccia</i>	IDelegationRegistryForAccess	interface	Interfaccia verso DelegationRegistry.sol, utilizzata per verificare l'esistenza di una delega valida per il richiedente, il documento, l'azione e la finalità.
<i>Interfaccia</i>	IPatientDrivenPolicyRegistryForAccess	interface	Interfaccia verso PatientDrivenPolicyRegistry.sol, utilizzata per verificare se esistono restrizioni patient-driven applicabili alla richiesta.
<i>Interfaccia</i>	IAuditRegistryForAccess	interface	Interfaccia verso AuditRegistry.sol, utilizzata per registrare l'esito delle richieste di accesso.
<i>Dati</i>	identityRegistry	IdentityRegistryForAccess public	Riferimento al registro delle identità, usato per validare il DID richiedente e verificarne il controller.
<i>Dati</i>	documentRegistry	IDocumentLifecycleRegistryForAccess public	Riferimento al registro documentale, usato per verificare esistenza, stato, paziente e tipo del documento.
<i>Dati</i>	policyRegistry	IPolicyRegistryForAccess public	Riferimento al registro delle policy globali, usato per verificare se la credenziale presentata abilita l'azione richiesta.
<i>Dati</i>	delegationRegistry	IDelegationRegistryForAccess public	Riferimento al registro delle deleghe, usato per verificare deleghe valide per documento, azione e finalità.
<i>Dati</i>	patientPolicyRegistry	IPatientDrivenPolicyRegistryForAccess public	Riferimento al registro delle policy definite dal paziente, usato per controllare eventuali restrizioni.
<i>Dati</i>	auditRegistry	IAuditRegistryForAccess public	Riferimento al registro di audit, usato per tracciare l'esito delle richieste di accesso.
<i>Operazione</i>	constructor	constructor(address identityRegistryAddress, address documentRegistryAddress, address policyRegistryAddress, address delegationRegistryAddress, address patientPolicyRegistryAddress, address auditRegistryAddress)	Inizializza il contratto impostando i riferimenti ai registri necessari alla valutazione delle richieste di accesso. Non accetta indirizzi nulli.
<i>Operazione</i>	canAccess	canAccess(bytes32 requesterDID, DataTypes.PresentedCredential calldata presentedCredential, bytes32 documentId, bytes32 action, bytes32 purpose) public view returns (bool)	Verifica se un DID può eseguire una determinata azione su un documento, considerando DID richiedente, credenziale presentata, documento, azione, finalità, restrizioni patient-driven, deleghe e policy globali.
<i>Operazione</i>	requestAccess	requestAccess(bytes32 requesterDID, DataTypes.PresentedCredential calldata presentedCredential, bytes32 documentId, bytes32	Esegue una richiesta di accesso con finalità esplicita, verifica che il chiamante sia il controller del DID richiedente, emette l'evento AccessRequested e registra l'esito nell'audit.

Categoria	Elemento	Tipo / Firma	Descrizione
Funzione interna		action, bytes32 purpose) public returns (bool)	
	_credentialIdFromPresentedCredential	_credentialIdFromPresentedCredential(DataTypes.PresentedCredential calldata presentedCredential) internal pure returns (bytes32)	Calcola l'identificativo della credenziale presentata tramite hash dell'id; restituisce bytes32(0) se l'id è vuoto.
Funzione interna	_canAccessWithReason	_canAccessWithReason(bytes32 requesterDID, DataTypes.PresentedCredential calldata presentedCredential, bytes32 documentId, bytes32 action, bytes32 purpose) internal view returns (bool, bytes32)	Contiene la logica principale di valutazione dell'accesso e restituisce sia l'esito booleano sia il codice motivazionale.
Evento	AccessRequested	event AccessRequested(address indexed caller, bytes32 indexed requesterDID, bytes32 indexed documentId, bytes32 action, bytes32 purpose, bytes32 credentialId, bool allowed, bytes32 reasonCode)	Evento emesso quando viene effettuata una richiesta di accesso tramite requestAccess. Registra chiamante, DID richiedente, documento, azione, finalità, credenziale, esito e motivo della decisione.

Tabella 9 Riferimenti AccessController

IdentityRegistry.sol

Il contratto *IdentityRegistry.sol* implementa il **registro delle identità digitali** basate su DID. Coerentemente con il modello descritto nel WP2, il **DID Document completo** non viene salvato direttamente on-chain, ma resta **off-chain**; sul ledger vengono registrati soltanto il DID canonico, il controller Ethereum associato, il CID del DID Document, lo stato e le informazioni di versionamento. Ogni **identità** è rappresentata tramite la **struttura DIDRecord**, che consente di sapere se un DID è attivo, chi lo controlla, quale documento DID lo descrive e quando è stato creato, aggiornato o disattivato.

Il contratto permette al controller di registrare un nuovo DID, aggiornare il riferimento al DID Document e disattivare il DID, con il vincolo che **uno stesso controller non può registrare più DID** e che l'aggiornamento del DID Document e la disattivazione possono essere **eseguiti solo dal controller** del DID e solo se il DID è ancora attivo. Inoltre, espone funzioni di consultazione come *resolveDID* e *isActiveDID*, utili agli altri contratti per verificare identità e controller.

Le operazioni critiche vengono tracciate tramite *AuditRegistry.sol*, garantendo auditabilità delle attività legate alla gestione delle identità.

Categoria	Elemento	Tipo / Firma	Descrizione
Interfaccia	IAuditRegistryForIdentities	interface	Interfaccia verso AuditRegistry.sol, utilizzata per registrare eventi di audit relativi alle operazioni sulle identità.
Struct	DIDRecord	struct	Struttura che rappresenta un DID registrato. Contiene DID, controller, CID e hash del DID Document, stato, versione e timestamp di creazione, aggiornamento e disattivazione.
Dati	auditRegistry	IAuditRegistryForIdentities public	Riferimento al registro di audit, usato per tracciare registrazione, aggiornamento e disattivazione dei DID.

Categoria	Elemento	Tipo / Firma	Descrizione
Dati	records	mapping(bytes32 => DIDRecord) private	Mapping che associa a ogni DID il relativo record completo.
Dati	didExists	mapping(bytes32 => bool) public	Mapping che indica se un DID è già stato registrato nel sistema.
Dati	didByController	mapping(address => bytes32) private	Mapping che associa a ogni controller Ethereum il DID da lui controllato.
Modifier	onlyController	onlyController(bytes32 did)	Consente l'esecuzione della funzione solo al controller del DID indicato.
Operazione	constructor	constructor(address auditRegistryAddress)	Inizializza il contratto impostando il riferimento al registro di audit.
Operazione	registerDID	registerDID(bytes32 did, bytes32 didDocumentCID, bytes32 didDocumentHash) external	Registra un nuovo DID, associandolo al chiamante come controller, salvando CID/hash del DID Document e impostando lo stato iniziale ad Active.
Operazione	updateDIDDocument	updateDIDDocument(bytes32 did, bytes32 newDIDDocumentCID, bytes32 newDIDDocumentHash) external	Permette al controller di aggiornare CID e hash del DID Document, incrementando la versione del record.
Operazione	deactivateDID	deactivateDID(bytes32 did) external	Permette al controller di disattivare un DID attivo, aggiornandone stato e timestamp di disattivazione.
Operazione	resolveDID	resolveDID(bytes32 did) external view returns (DIDRecord memory)	Restituisce il record completo associato a un DID esistente.
Operazione	isActiveDID	isActiveDID(bytes32 did) public view returns (bool)	Restituisce true se il DID esiste ed è nello stato Active.
Operazione	controllerOf	controllerOf(bytes32 did) public view returns (address)	Restituisce l'indirizzo controller associato a un DID, oppure address(0) se il DID non esiste.
Operazione	didOfController	didOfController(address controller) external view returns (bytes32)	Restituisce il DID associato a uno specifico controller Ethereum.
Operazione	isController	isController(bytes32 did, address account) external view returns (bool)	Verifica se un determinato address è il controller del DID indicato.
Evento	DIDRegistered	event	Evento emesso quando viene registrato un nuovo DID.
Evento	DIDDocumentUpdated	event	Evento emesso quando il DID Document associato a un DID viene aggiornato.
Evento	DIDDeactivated	event	Evento emesso quando un DID viene disattivato.

Tabella 10 Riferimenti IdentityRegistry

TrustRegistry.sol

Il contratto implementa il **registro degli issuer fidati** per tipologia di *Verifiable Credential*. Nel modello del sistema, credenziali considerate valide non possono essere emesse da chiunque, bensì un issuer è ritenuto fidato solo se il suo DID è attivo *nell'IdentityRegistry* e se il controller *Ethereum* associato a quel DID corrisponde a un'organizzazione attiva registrata *nell'OrganizationRegistry*. In questo modo, la fiducia non viene attribuita direttamente a un semplice indirizzo, ma deriva dalla combinazione tra identità DID e appartenenza a un'organizzazione riconosciuta.

Il contratto prevede una fase iniziale di bootstrap, durante la quale il *bootstrapAdmin* può configurare gli issuer fidati; dopo il blocco del bootstrap, invece, le operazioni di autorizzazione e revoca possono essere eseguite solo dal contratto di governance configurato.

Le autorizzazioni vengono salvate nel mapping *trustedIssuerForType*, che associa una coppia issuer DID/tipo di credenziale a un valore booleano. Le operazioni di autorizzazione e revoca sono tracciate tramite eventi e tramite *AuditRegistry*, così da mantenere verificabile l'evoluzione del perimetro di fiducia del sistema.

La seguente tabella riassume le principali componenti del contratto:

Categoria	Elemento	Tipo / Firma	Descrizione
Interfaccia	<i>IIdentityRegistryForTrust</i>	interface	Interfaccia verso <i>IdentityRegistry.sol</i> , usata per verificare se un DID issuer è attivo e per recuperare il controller Ethereum associato.
Interfaccia	<i>IOrganizationRegistryForTrust</i>	interface	Interfaccia verso <i>OrganizationRegistry.sol</i> , usata per verificare se il controller Ethereum dell'issuer è un'organizzazione attiva.
Interfaccia	<i>IAuditRegistryForTrust</i>	interface	Interfaccia verso <i>AuditRegistry.sol</i> , usata per registrare eventi di audit relativi alla gestione degli issuer fidati.
Dati	<i>bootstrapAdmin</i>	address public immutable	Indirizzo dell'account che effettua il deploy del contratto. Ha poteri amministrativi durante la fase iniziale di bootstrap.
Dati	<i>governanceContract</i>	address public	Indirizzo del contratto di governance autorizzato a gestire gli issuer fidati dopo la configurazione.
Dati	<i>bootstrapLocked</i>	bool public	Indica se la fase di bootstrap è stata bloccata. Quando vale true, il bootstrap admin non può più operare direttamente tramite le funzioni protette da <i>onlyBootstrapOrGovernance</i> .
Dati	<i>identityRegistry</i>	<i>IIdentityRegistryForTrust</i> public	Riferimento al registro delle identità, usato per validare lo stato del DID issuer e recuperarne il controller.
Dati	<i>organizationRegistry</i>	<i>IOrganizationRegistryForTrust</i> public	Riferimento al registro delle organizzazioni, usato per verificare che il controller dell'issuer sia un'organizzazione attiva.
Dati	<i>auditRegistry</i>	<i>IAuditRegistryForTrust</i> public	Riferimento al registro di audit, usato per tracciare autorizzazioni e revoche degli issuer fidati.
Dati	<i>trustedIssuerForType</i>	mapping(bytes32 => mapping(bytes32 => bool)) private	Mapping che indica se un determinato issuer DID è fidato per uno specifico tipo di credenziale.
Modifier	<i>onlyBootstrapAdmin</i>	modifier	Consente l'esecuzione della funzione solo al bootstrap admin.
Modifier	<i>onlyBootstrapOrGovernance</i>	modifier	Consente l'esecuzione della funzione al bootstrap admin durante la fase di bootstrap oppure al contratto di governance configurato.
Operazione	constructor	constructor(address identityRegistryAddress, address	Inizializza il contratto impostando il bootstrap admin e i riferimenti ai registri delle identità, delle organizzazioni e di audit.

Categoria	Elemento	Tipo / Firma	Descrizione
		organizationRegistryAddress, address auditRegistryAddress)	
Operazione	setGovernanceContract	setGovernanceContract(address governanceContractAddress) external	Imposta l'indirizzo del contratto di governance durante la fase di bootstrap. Non accetta address(0) e non può essere eseguita dopo il lock del bootstrap.
Operazione	lockBootstrap	lockBootstrap() external	Blocca definitivamente la fase di bootstrap, impedendo ulteriori operazioni dirette del bootstrap admin nelle funzioni riservate a bootstrap o governance.
Operazione	authorizeIssuer	authorizeIssuer(bytes32 issuerDID, bytes32 credentialType) external	Autorizza un DID issuer per uno specifico tipo di credenziale, verificando che l'issuer DID non sia nullo, che il tipo di credenziale non sia nullo, che il DID sia attivo e che il suo controller sia un'organizzazione attiva.
Operazione	revokeIssuerAuthorization	revokeIssuerAuthorization(bytes32 issuerDID, bytes32 credentialType) external	Revoca l'autorizzazione di un issuer fidato per uno specifico tipo di credenziale. Può essere eseguita solo se l'issuer risulta attualmente fidato per quel tipo.
Operazione	isTrustedIssuer	isTrustedIssuer(bytes32 issuerDID, bytes32 credentialType) external view returns (bool)	Restituisce true se il mapping indica che l'issuer DID è fidato per il tipo di credenziale indicato. Non riesegue il controllo dinamico su DID attivo o organizzazione attiva.
Evento	GovernanceContractSet	event GovernanceContractSet(address indexed governanceContract)	Evento emesso quando viene impostato il contratto di governance.
Evento	BootstrapLocked	event BootstrapLocked(uint256 timestamp)	Evento emesso quando la fase di bootstrap viene bloccata, includendo il timestamp.
Evento	TrustedIssuerAuthorized	event TrustedIssuerAuthorized(bytes32 indexed issuerDID, bytes32 indexed credentialType, address indexed controller)	Evento emesso quando un issuer DID viene autorizzato per un tipo di credenziale. Include anche il controller dell'issuer.
Evento	TrustedIssuerRevoked	event TrustedIssuerRevoked(bytes32 indexed issuerDID, bytes32 indexed credentialType, address indexed revokedBy)	Evento emesso quando l'autorizzazione di un issuer DID viene revocata. Include anche l'indirizzo che ha effettuato la revoca.

Tabella 11 Riferimenti TrustRegistry

CredentialStatusRegistry.sol

Il contratto implementa il **registro di stato delle Verifiable Credential**. La credenziale completa resta off-chain, mentre on-chain vengono salvati solo i **metadati minimi necessari** alla gestione dello stato: identificativo della credenziale, issuer DID, tipo di credenziale, stato, intervallo temporale di validità e timestamp relativi a emissione, aggiornamento ed eventuale revoca.

Prima di registrare lo stato di una credenziale, il contratto verifica che l'issuer DID sia attivo, che il chiamante sia il controller dell'issuer e che l'issuer sia fidato per quello specifico tipo di credenziale tramite TrustRegistry. Il subject della credenziale non viene salvato nel record on-chain, ma viene verificato nella fase di presentazione della credenziale, tramite *isPresentedCredentialValidForSubject*, che controlla la **coerenza tra credenziale presentata, subject atteso, issuer, tipo di credenziale e stato registrato**.

Il contratto consente inoltre all'issuer di sospendere, riattivare o revocare una credenziale, mantenendo tracciabilità tramite **eventi Solidity** e *AuditRegistry*.

Categoria	Elemento	Tipo / Firma	Descrizione
<i>Interfaccia</i>	<code>IdentityRegistryForCredentials</code>	interface	Interfaccia verso <code>IdentityRegistry.sol</code> , usata per verificare se un DID è attivo e per recuperare il controller dell'issuer.
<i>Interfaccia</i>	<code>ITrustRegistryForCredentials</code>	interface	Interfaccia verso <code>TrustRegistry.sol</code> , usata per verificare se un issuer DID è fidato per uno specifico tipo di credenziale.
<i>Interfaccia</i>	<code>IAuditRegistryForCredentials</code>	interface	Interfaccia verso <code>AuditRegistry.sol</code> , usata per registrare eventi di audit relativi alle credenziali.
<i>Struct</i>	<code>CredentialStatus</code>	struct	Struttura che rappresenta lo stato di una VC. Contiene <code>credentialId</code> , <code>issuerDID</code> , <code>credentialType</code> , stato, validità temporale, timestamp di emissione, aggiornamento e revoca.
<i>Dati</i>	<code>identityRegistry</code>	<code>IdentityRegistryForCredentials public</code>	Riferimento al registro delle identità, usato per verificare DID attivi e controller dell'issuer.
<i>Dati</i>	<code>trustRegistry</code>	<code>ITrustRegistryForCredentials public</code>	Riferimento al registro di fiducia, usato per verificare che l'issuer sia autorizzato per il tipo di credenziale.
<i>Dati</i>	<code>auditRegistry</code>	<code>IAuditRegistryForCredentials public</code>	Riferimento al registro di audit, usato per tracciare emissione, sospensione, riattivazione e revoca delle credenziali.
<i>Dati</i>	<code>credentials</code>	<code>mapping(bytes32 => CredentialStatus) private</code>	Mapping che associa a ogni <code>credentialId</code> il relativo record di stato.
<i>Dati</i>	<code>credentialExists</code>	<code>mapping(bytes32 => bool) public</code>	Mapping che indica se una credenziale è già stata registrata.
<i>Dati</i>	<code>credentialsByIssuer</code>	<code>mapping(bytes32 => bytes32[]) private</code>	Mapping che associa a ogni issuer DID la lista delle credenziali emesse.
<i>Modifier</i>	<code>onlyIssuerController</code>	<code>onlyIssuerController(bytes32 credentialId)</code>	Consente l'esecuzione della funzione solo al controller del DID issuer della credenziale.
<i>Operazione</i>	<code>constructor</code>	<code>constructor(address identityRegistryAddress, address trustRegistryAddress, address auditRegistryAddress)</code>	Inizializza il contratto impostando i riferimenti ai registri delle identità, della fiducia e di audit.
<i>Operazione</i>	<code>issueCredentialStatus</code>	<code>issueCredentialStatus(bytes32 credentialId, bytes32 issuerDID, bytes32 credentialType, uint256 validFrom, uint256 validUntil) external</code>	Registra lo stato di una nuova credenziale, verificando identificativo valido, issuer DID attivo, controller dell'issuer, issuer fidato, intervallo temporale corretto e assenza di duplicati.
<i>Operazione</i>	<code>issueCredential</code>	<code>issueCredential(bytes32 credentialId, bytes32 issuerDID, bytes32 credentialType, uint256 validFrom, uint256 validUntil) external</code>	Alias di <code>issueCredentialStatus</code> , mantenuto per semplificare la migrazione di script esistenti.
<i>Operazione interna</i>	<code>_issueCredentialStatus</code>	<code>_issueCredentialStatus(bytes32 credentialId, bytes32 issuerDID, bytes32 credentialType, uint256 validFrom, uint256 validUntil) internal</code>	Funzione interna che contiene la logica effettiva di registrazione dello stato della credenziale.
<i>Operazione</i>	<code>suspendCredential</code>	<code>suspendCredential(bytes32 credentialId) external</code>	Sospende una credenziale attiva. Può essere eseguita solo dal controller dell'issuer.

Categoria	Elemento	Tipo / Firma	Descrizione
Operazione	reactivateCredential	reactivateCredential(bytes32 credentialId) external	Riattiva una credenziale sospesa, se non è scaduta, se l'issuer DID è ancora attivo e se l'issuer è ancora fidato.
Operazione	revokeCredential	revokeCredential(bytes32 credentialId) external	Revoca una credenziale non ancora revocata, aggiornandone stato, timestamp di aggiornamento e timestamp di revoca.
Operazione	getCredential	getCredential(bytes32 credentialId) external view returns (CredentialStatus memory)	Restituisce il record completo associato a una credenziale esistente.
Operazione	getCredentialType	getCredentialType(bytes32 credentialId) external view returns (bytes32)	Restituisce il tipo della credenziale, oppure bytes32(0) se la credenziale non esiste.
Operazione	getIssuerDID	getIssuerDID(bytes32 credentialId) external view returns (bytes32)	Restituisce l'issuer DID della credenziale, oppure bytes32(0) se non esiste.
Operazione	getCredentialsByIssuer	getCredentialsByIssuer(bytes32 issuerDID) external view returns (bytes32[] memory)	Restituisce la lista degli identificativi delle credenziali emesse da un issuer DID.
Operazione	isCredentialValid	isCredentialValid(bytes32 credentialId) public view returns (bool)	Verifica se una credenziale esiste, è attiva, temporalmente valida, associata a un issuer DID attivo e rilasciata da un issuer ancora fidato.
Operazione	isPresentedCredentialValidForSubject	isPresentedCredentialValidForSubject(DataTypes.PresentedCredential calldata presentedCredential, bytes32 expectedSubjectDID) external view returns (bool)	Verifica se una credenziale presentata in formato vc+sd-jwt è valida per il subject DID atteso, controllando formato, campi obbligatori, credentialId derivato, issuer, tipo di credenziale, stato della credenziale e DID attivi.
Operazione	presentedCredentialDigest	presentedCredentialDigest(DataTypes.PresentedCredential calldata presentedCredential) public pure returns (bytes32)	Calcola il digest della credenziale presentata usando i suoi campi principali.
Operazione	deriveCredentialId	deriveCredentialId(DataTypes.PresentedCredential calldata presentedCredential) external pure returns (bytes32)	Deriva il credentialId a partire dall'id testuale della credenziale presentata.
Operazione	deriveIssuerDID	deriveIssuerDID(DataTypes.PresentedCredential calldata presentedCredential) external pure returns (bytes32)	Deriva l'issuer DID a partire dal campo issuer della credenziale presentata.
Operazione	deriveSubjectDID	deriveSubjectDID(DataTypes.PresentedCredential calldata presentedCredential) external pure returns (bytes32)	Deriva il subject DID a partire dal campo subject della credenziale presentata.
Operazione	deriveCredentialType	deriveCredentialType(DataTypes.PresentedCredential calldata presentedCredential) external pure returns (bytes32)	Deriva il tipo di credenziale a partire dal campo credentialType della credenziale presentata.
Funzione interna	_hashString	_hashString(string memory value) internal pure returns (bytes32)	Calcola l'hash keccak256 di una stringa.

Categoria	Elemento	Tipo / Firma	Descrizione
Funzione interna	_isVcSdJwt	_isVcSdJwt(string memory value) internal pure returns (bool)	Verifica se il formato della credenziale è vc+sd-jwt.
Funzione interna	_recoverEthSignedMessage	_recoverEthSignedMessage(bytes32 digest, bytes memory signature) internal pure returns (address)	Recupera l'indirizzo Ethereum del firmatario a partire da digest e firma. Nel codice attuale non viene richiamata da altre funzioni.
Evento	CredentialIssued	event	Evento emesso quando viene registrato lo stato di una nuova credenziale.
Evento	CredentialSuspended	event	Evento emesso quando una credenziale viene sospesa.
Evento	CredentialReactivated	event	Evento emesso quando una credenziale sospesa viene riattivata.
Evento	CredentialRevoked	event	Evento emesso quando una credenziale viene revocata.

Tabella 12 Riferimenti CredentialStatusRegistry

Script Deploy.ts

Lo script di deploy ha il compito di inizializzare l'intero prototipo, distribuire gli smart contract principali, configurare le relazioni tra i registri e dimostrare il funzionamento dei flussi più rilevanti del sistema. Realizza, quindi, una sequenza completa di bootstrap, configurazione delle identità, gestione delle credenziali, approvazione delle policy, creazione documentale, valutazione degli accessi, deleghe, revoche e audit.

Inizializzazione dell'ambiente

La prima fase riguarda l'**inizializzazione** dell'**ambiente off-chain** e degli **account di test**. Lo script configura il supporto a *IPFS* ed *Helia* per il salvataggio dei contenuti esterni e inizializza la logica *SD-JWT* utilizzata.

In questa fase vengono inoltre recuperati gli account *Hardhat* che rappresentano i diversi soggetti del sistema; vengono poi definiti ruoli, azioni, tipi documentali, finalità, DID e identificativi delle credenziali utilizzati nei passaggi successivi.

Deploy dei registri di base

La seconda fase consiste nel **deploy dei registri di base**. Vengono distribuiti ***AuditRegistry.sol***, ***OrganizationRegistry.sol*** e ***IdentityRegistry.sol***. *AuditRegistry* viene deployato per primo perché costituisce il componente trasversale utilizzato dagli altri contratti per registrare eventi rilevanti. *OrganizationRegistry* e *IdentityRegistry* ricevono infatti l'indirizzo di *AuditRegistry* nel costruttore, così da poter produrre eventi di audit durante le operazioni successive.

Dopo il deploy dei registri iniziali, lo script autorizza i primi **audit emitter**. In particolare, *OrganizationRegistry* e *IdentityRegistry* vengono abilitati a invocare *AuditRegistry.logEvent*.

Bootstrap delle organizzazioni

La terza fase è il **bootstrap delle organizzazioni**. Durante questa fase, il **deployer**, che agisce come bootstrap admin, registra le **organizzazioni iniziali** nel sistema. Le tre organizzazioni vengono inserite come **autorità attive** all'interno di *OrganizationRegistry* e contribuiscono alla composizione iniziale del network permissioned. Il numero di organizzazioni attive viene poi utilizzato nelle fasi successive per il calcolo dei quorum di governance.

Registrazione DID e DID Document

La quarta fase riguarda la **registrazione delle identità DID e dei DID Document**. Per ciascun soggetto del sistema viene creato un DID Document off-chain, contenente *controller*, *verification method*, *relationship* di autenticazione e *service endpoint*. Il DID Document viene caricato su IPFS, mentre on-chain vengono registrati il DID e il riferimento CID. In questo modo il DID Document completo rimane off-chain, ma la sua integrità è verificabile attraverso i riferimenti salvati in *IdentityRegistry*.

Trust Registry e Credential Status Registry

La quinta fase introduce *TrustRegistry* e *CredentialStatusRegistry*. *TrustRegistry* viene deployato per gestire gli issuer fidati per specifiche tipologie di credenziali. In particolare, lo script autorizza il DID della prima organizzazione come issuer fidato per le credenziali del dominio sanitario.

Successivamente viene deployato *CredentialStatusRegistry*, che conserva on-chain solo lo stato minimo delle credenziali, mentre le VC complete restano off-chain. In questa fase vengono create credenziali per medico, paziente e auditor, salvando su IPFS i contenuti off-chain e registrando on-chain gli stati necessari per validità, sospensione e revoca.

Policy Registry e Policy Governance

La sesta fase riguarda il deploy di *PolicyRegistry* e *PolicyGovernance*. *PolicyRegistry* viene inizializzato con i riferimenti a *OrganizationRegistry*, *CredentialStatusRegistry* e *AuditRegistry*. Inoltre, in questa fase viene configurato anche il mapping tra tipo di credenziale e ruolo applicativo.

Successivamente viene deployato *PolicyGovernance*, che rappresenta il modulo attraverso cui le policy globali e le decisioni di governance vengono approvate tramite proposta, voto, quorum ed esecuzione.

Inoltre, *PolicyGovernance* viene registrato come *governanceContract* in *OrganizationRegistry*, *PolicyRegistry* e *TrustRegistry*. In questo modo, dopo il blocco del bootstrap, le modifiche critiche non possono più essere effettuate direttamente da un amministratore iniziale, ma devono passare dal processo di governance multi-organizzazione.

Contratti operativi

La settima fase riguarda il deploy dei contratti operativi, in particolare vengono distribuiti *DocumentLifecycleRegistry*, *PatientDrivenPolicyRegistry*, *DelegationRegistry* e *AccessController*. Questi contratti rappresentano il livello applicativo del prototipo: il primo gestisce ciclo di vita e versionamento dei documenti, il secondo le restrizioni definite dal paziente, il terzo le deleghe dirette e derivate, mentre *AccessController* coordina i registri per produrre la decisione finale di accesso.

Inoltre, i contratti deployati vengono autorizzati come audit emitter per consentire la scrittura di eventi di audit in *AuditRegistry*. Questo consente di tracciare creazione e aggiornamento dei documenti, registrazione di restrizioni patient-driven, proposta e revoca delle deleghe e richieste di accesso consentite o negate.

Una volta fatto ciò, lo script blocca il bootstrap di *OrganizationRegistry*, *PolicyRegistry*, *TrustRegistry* e *AuditRegistry*. Da questo momento, la configurazione iniziale viene considerata stabilizzata: le modifiche alle organizzazioni, alle policy globali e agli issuer fidati devono avvenire tramite governance, mentre la produzione di audit rimane limitata agli emitter autorizzati.

Implementazione dei flussi operativi

Definizione delle policy di governance

Le **policy globali** non vengono impostate direttamente da un amministratore, ma sono create come contenuti off-chain salvati su IPFS e poi sottoposte a *PolicyGovernance*. Le organizzazioni votano le proposte e, al raggiungimento del quorum, le policy vengono eseguite su *PolicyRegistry*. Lo script

approva, ad esempio, le autorizzazioni del ruolo *Doctor* per creare, leggere, aggiornare, revocare e delegare documenti di tipo *REFERTO*, e l'autorizzazione del ruolo *Auditor* alla lettura.

Segue la governance della **profondità massima delle deleghe**, per cui le organizzazioni votano e, dopo la finalizzazione, il valore viene aggiornato in *PolicyRegistry*. Nel caso mostrato, la profondità massima viene impostata a 2, consentendo una delega diretta e una delega derivata, ma impedendo ulteriori livelli.

Gestione ciclo di vita dei documenti

Successivamente lo script dimostra il **ciclo di vita documentale**. Un documento clinico viene caricato su IPFS e il relativo CID viene registrato in *DocumentLifecycleRegistry* al momento della creazione del documento. Il documento viene associato al DID del paziente e creato dal medico tramite DID e credenziale valida. Viene poi caricata una seconda versione su IPFS e registrata una **nuova versione documentale**, per cui la versione precedente viene archiviata, mentre la nuova diventa la versione corrente certificata. Lo script verifica anche che il contenuto recuperato da IPFS corrisponda al documento originale.

Selective disclosure delle credenziali

Un'ulteriore fase dimostra la **presentazione selettiva SD-JWT** a livello off-chain. Prima della richiesta di accesso, il wallet del medico costruisce una presentazione selettiva rivelando solo alcuni claim richiesti dal verifier, come ruolo e reparto, mantenendo nascosti altri attributi. Il verifier controlla firma, issuer, subject, tipo credenziale, claim richiesti, holder binding e freschezza del nonce. Solo dopo questa verifica off-chain viene invocata la logica on-chain di *AccessController*.

Valutazione degli accessi

Lo script mostra poi la **valutazione degli accessi**. Il medico richiede accesso in lettura al documento e viene autorizzato perché possiede un DID attivo, presenta una credenziale valida e soddisfa la policy globale. L'accesso viene inoltre registrato tramite *AuditRegistry*. Successivamente viene mostrato anche il caso dell'auditor, inizialmente autorizzato dalla policy globale e poi bloccato da una restrizione patient-driven.

La fase delle **policy patient-driven** dimostra che il paziente può registrare restrizioni specifiche su un documento. La policy completa viene salvata off-chain su IPFS, mentre on-chain viene registrato il riferimento e l'ambito della restrizione. Quando l'auditor tenta nuovamente di accedere al documento, *AccessController* rileva la restrizione e nega l'accesso, dimostrando che le policy del paziente possono restringere ma non ampliare i permessi globali.

Gestione delle deleghe

La fase delle **deleghe** mostra prima una **delega diretta** dal medico al secondo medico. Il medico propone la delega, ma questa diventa valida solo dopo l'approvazione del paziente. Una volta approvata, il medico delegato può accedere al documento anche senza possedere una credenziale professionale globale, perché l'autorizzazione deriva dalla delega attiva. Successivamente viene mostrata una **delega derivata** dal medico delegato a un terzo medico, ammessa perché rientra nella profondità massima definita dalla governance.

Lo script dimostra anche i casi di controllo negativo; ad esempio, il tentativo di creare una delega oltre la profondità massima viene bloccato. Inoltre, quando il paziente revoca la delega padre, la delega derivata non viene cancellata materialmente dal registro, ma diventa inutilizzabile perché la catena di delega non è più valida. Questo dimostra la propagazione logica degli effetti della revoca lungo la catena.

Revoca di accesso e documento

Un'altra fase **distingue la revoca dell'accesso dalla revoca del documento**. Il paziente registra una restrizione patient-driven nei confronti del medico, impedendogli di leggere il referto, ma il documento rimane nello stato *Certified* nel *DocumentLifecycleRegistry*. In questo modo viene dimostrato che revocare un consenso o un accesso non equivale a revocare il documento clinico.

Successivamente lo script revoca la Verifiable Credential del medico. L'organizzazione issuer revoca la *MedicalProfessionalVC* tramite *CredentialStatusRegistry*; dopo la revoca, la credenziale non risulta più valida e il medico non può più superare il controllo della policy globale basato su VC.

Audit degli eventi

La fase finale riguarda l'audit e l'esportazione degli asset. Lo script crea un audit batch dimostrativo off-chain, contenente riferimenti alle principali operazioni eseguite e agli asset salvati su IPFS. Tutti gli asset off-chain vengono infine recuperati da IPFS e salvati localmente nella directory *ipfs-output*, insieme a un indice riepilogativo.

Si sottolinea che da soluzione il batch deve contenere gli eventi relativi ad una macro-operazione, mentre in questo caso per ragioni di semplificazione contiene gli eventi di tutte le operazioni descritte in precedenza.

Analisi delle performance

Di seguito viene riportata un'analisi delle performance del sistema proposto, considerando sia i risultati ottenuti dall'esecuzione della proof of concept sia i trade-off e le semplificazioni introdotte durante la fase implementativa. L'analisi prende in esame, in primo luogo, la latenza delle principali operazioni e il throughput osservabile nell'ambiente di test locale; successivamente, viene valutato il consumo di gas delle transazioni on-chain e, infine, viene analizzato lo storage overhead, distinguendo tra dati mantenuti sul ledger e contenuti memorizzati off-chain tramite IPFS.

Metodologia delle misurazioni

La misurazione è stata effettuata andando ad eseguire il test funzionale della Proof of Concept prodotta. Per ciascuna operazione sono stati rilevati il gruppo funzionale di appartenenza, il tipo di chiamata, il tempo di esecuzione in millisecondi, l'eventuale gas consumato e l'esito dell'operazione. Inoltre, le operazioni sono state distinte tra transazioni, chiamate di sola lettura e casi di fallimento atteso. Questa distinzione è rilevante perché le chiamate *view* non modificano lo stato del sistema e non comportano consumo di gas quando eseguite off-chain, mentre le transazioni rappresentano operazioni effettivamente registrate sul ledger.

I tempi rilevati devono essere interpretati considerando che l'esecuzione del prototipo è stata effettuata in locale; infatti, in un ambiente reale, inciderebbero anche tempi di propagazione, consenso, finalizzazione dei blocchi e carico dei nodi. Tuttavia, tali misurazioni sono utili per confrontare il peso relativo delle diverse operazioni e individuare le componenti più onerose del sistema.

Analisi della latenza

Dall'analisi dei tempi emerge che le chiamate di sola lettura, come le verifiche di accesso tramite *canAccess*, le verifiche di validità delle credenziali o il controllo delle restrizioni patient-driven, hanno un costo temporale ridotto nell'ambiente locale.

Le transazioni che modificano lo stato, invece, presentano tempi maggiori, soprattutto quando comportano la creazione di nuovi record persistenti, l'aggiornamento di strutture indicizzate o l'emissione di eventi di audit.

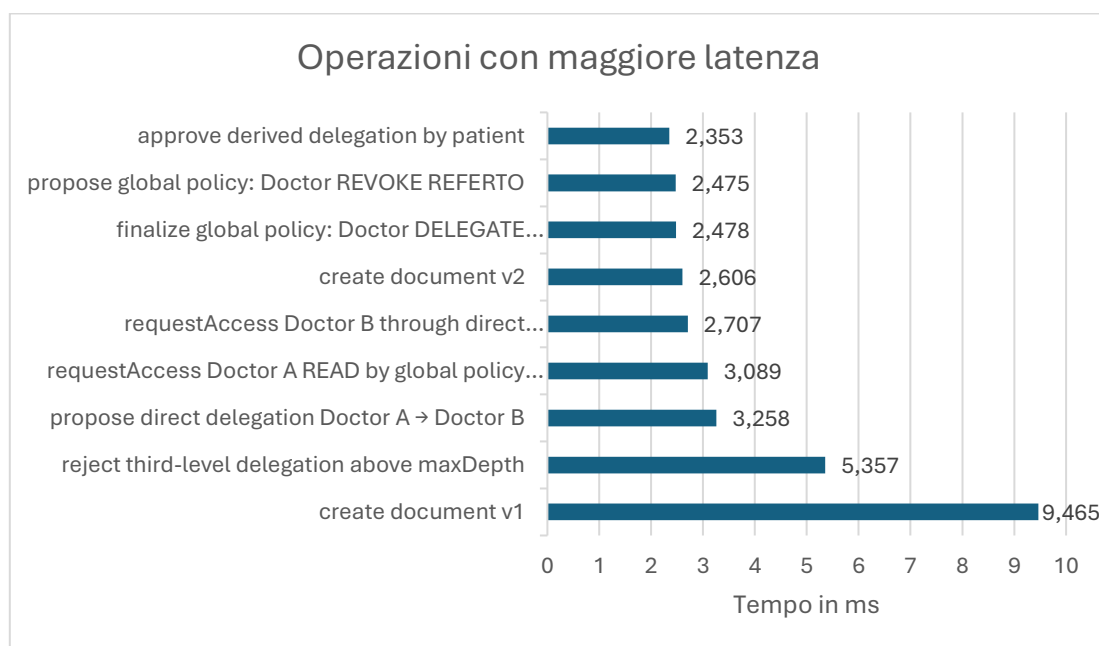


Figura 13 Latenza operazioni

Il grafico nella figura sopra riportata è utile perché fa emergere tre aspetti:

1. **Le operazioni più lente sono quelle che modificano stato**, come creazione documentale, deleghe, richieste di accesso tracciate e restrizioni patient-driven.
2. **Le chiamate di sola lettura non dominano la latenza**, bensì risultano leggere nel contesto locale.
3. **La latenza non coincide sempre con il gas**, perché il tempo locale *Hardhat* dipende anche dall'esecuzione dello script, dalle verifiche, dal revert atteso e dall'ambiente di test (come mostrato successivamente nella fase di analisi del gas consumato).

Latenza cumulativa delle operazioni

Per valutare l'impatto dell'aumento del numero di operazioni, è stata effettuata un'analisi di sensitività sui tempi rilevati nella proof of concept. A partire dai tempi medi osservati per ciascuna classe di operazioni, è stato stimato il tempo cumulativo necessario al crescere del numero di esecuzioni. Il grafico mostra che le operazioni di sola verifica hanno un andamento più contenuto, mentre le operazioni che modificano lo stato del sistema presentano una crescita più marcata. In particolare, le deleghe e le policy approvate tramite governance risultano più onerose perché non corrispondono a una singola verifica, ma a flussi composti da più passaggi logici e transazionali.

Nella tabella sotto riportata vi sono i tempi medi stimati per le singole macro-operazioni, descritte anche nei flussi operativi nel WP2.

Operazione ripetuta	Tempo unitario medio stimato
Registrazione DID	1,361 ms
Emissione stato VC	1,649 ms
Verifica canAccess	1,044 ms
requestAccess tracciata	2,898 ms
Nuova versione documentale	2,606 ms
Restrizione patient-driven	2,194 ms
Delega diretta completa	5,107 ms
Policy globale approvata con 2 voti	6,888 ms

Tabella 13 Latenza cumulativa

Il tempo cumulativo stimato è dato dal prodotto tra il numero di operazioni e il tempo medio unitario.

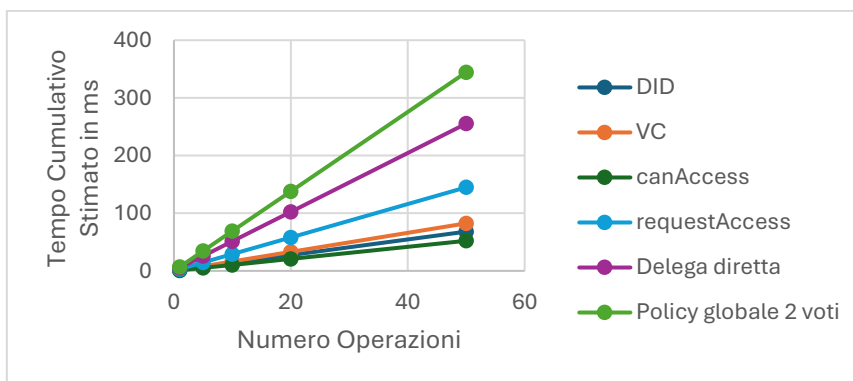


Figura 14 Tempo Cumulativo in relazione al numero di operazioni

Numero operazioni	DID	VC	canAccess	requestAccess	Delega diretta	Policy globale 2 voti
1	1,36	1,65	1,04	2,90	5,11	6,89
5	6,81	8,25	5,22	14,49	25,54	34,44
10	13,61	16,49	10,44	28,98	51,07	68,88
20	27,22	32,98	20,88	57,96	102,14	137,76
50	68,05	82,45	52,20	144,90	255,35	344,40

La governance presenta il tempo cumulativo più elevato tra le operazioni considerate, poiché ogni policy globale richiede proposta, voto e finalizzazione. Ad esempio, nel caso sperimentale, con tre organizzazioni attive il quorum viene raggiunto con due voti; pertanto, una policy globale completa richiede una proposta, due voti e una finalizzazione. Se il numero di voti richiesti aumentasse, il tempo complessivo crescerebbe in modo quasi lineare, poiché ogni voto aggiuntivo introdurrebbe una nuova transazione.

Poiché lo script di test esegue le operazioni in modo sequenziale, i tempi misurati rappresentano il tempo cumulativo della proof of concept in uno scenario controllato. Questo approccio consente di misurare con chiarezza il costo relativo dei singoli passaggi, ma non rappresenta necessariamente il tempo minimo ottenibile in un sistema reale, dove alcune operazioni indipendenti potrebbero essere eseguite in parallelo.

Per questo motivo, l'analisi delle performance può essere letta secondo due prospettive. La prima è quella sequenziale, coerente con l'esecuzione effettiva della proof of concept, nella quale i tempi si sommano. La seconda è quella basata sul percorso critico, nella quale le operazioni indipendenti vengono considerate parallelizzabili e il tempo complessivo dipende dalla catena di dipendenze più lunga. Questa seconda prospettiva è utile per stimare il comportamento del sistema in uno scenario più vicino a un'applicazione reale, in cui più richieste indipendenti possono essere inviate contemporaneamente.

Nel grafico di seguito vengono evidenziate le differenze nelle tempistiche di esecuzione considerando l'esecuzione in parallelo di alcune operazioni che in uno scenario reale possono essere così eseguite.

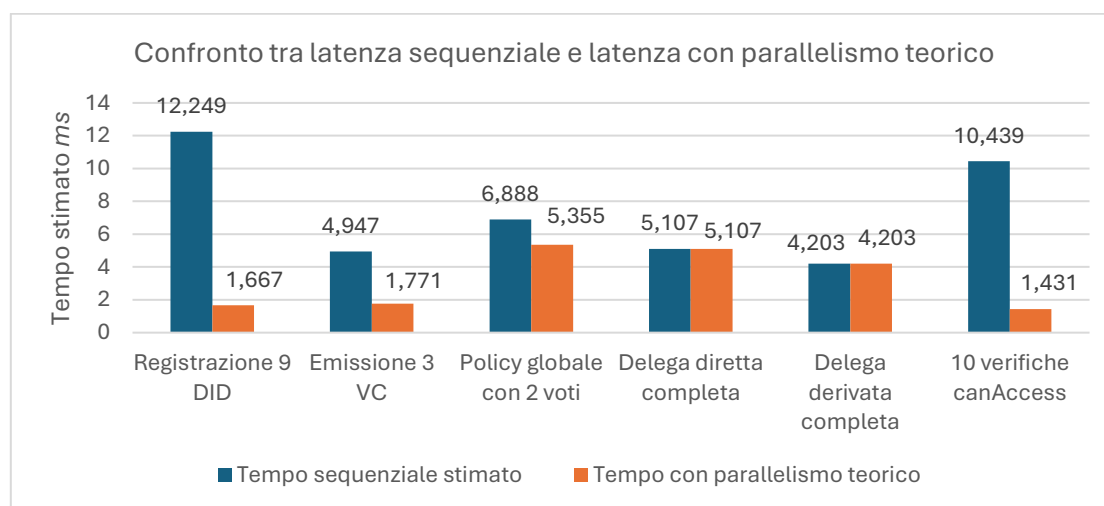


Figura 15 Confronto tra latenza sequenziale e latenza

Alcune attività, come la registrazione di DID relativi a soggetti diversi, l'emissione di credenziali per holder differenti, la verifica di accessi indipendenti o la creazione di proposte di policy distinte, possono essere parallelizzate a livello applicativo. In questi casi, il tempo minimo teorico non coincide con la somma dei tempi delle singole operazioni, ma tende ad avvicinarsi al tempo dell'operazione più lenta del batch. Al contrario, alcuni flussi mantengono una componente sequenziale non eliminabile.

Dunque, è possibile concludere che le operazioni indipendenti beneficiano molto del parallelismo, mentre i flussi con dipendenze logiche mantengono una componente sequenziale non eliminabile.

Throughput

Il throughput osservabile dall'esecuzione della proof of concept deve essere **interpretato in modo qualitativo**, poiché lo script esegue le operazioni in sequenza e in ambiente locale; pertanto, i risultati non rappresentano la **capacità massima del sistema** in condizioni di concorrenza o su una rete distribuita. L'analisi consente comunque di individuare i **principali colli di bottiglia**: le semplici verifiche di accesso risultano leggere, mentre i flussi che richiedono più transazioni, come la governance multi-organizzazione o la creazione di deleghe approvate dal paziente, riducono il throughput complessivo perché **richiedono più passaggi sequenziali**.

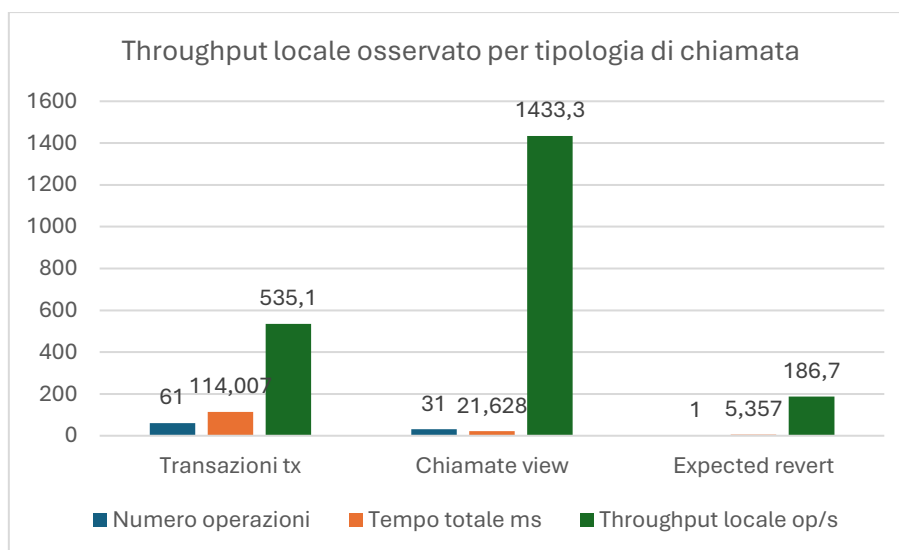


Figura 16 Throughput locale osservato per tipologia di chiamata

Storage Overhead

L'analisi dello storage overhead evidenzia la **separazione tra dati on-chain e dati off-chain**. Il ledger non conserva il contenuto completo dei documenti clinici, delle credenziali, delle policy o dei DID Document, ma soltanto riferimenti, CID, stati e metadati essenziali. I contenuti completi vengono, invece, salvati off-chain tramite IPFS/Helia.

Nel prototipo, gli asset off-chain generati sono di **dimensioni contenute** perché i **payload** utilizzati nella **demo sono sintetici**. In uno scenario reale, la dimensione dei documenti clinici, degli audit batch o delle policy potrebbe **crescere sensibilmente**.

audit-batch_Audit_batch_demo_001_bafkreidhskmnjh.json	29/05/2026 19:22	File di origine JSON	11 KB
deployment-summary.json	29/05/2026 19:22	File di origine JSON	17 KB
deployment-summary_Deployment_summary_completo_bafkreip2ghbmtl.json	29/05/2026 19:22	File di origine JSON	17 KB
did-document_DID_Document_Auditor_bafkreibwrbou7.json	29/05/2026 19:22	File di origine JSON	1 KB
did-document_DID_Document_Doctor_A_bafkreidkflhkk.json	29/05/2026 19:22	File di origine JSON	1 KB
did-document_DID_Document_Doctor_B_bafkreitmx135s.json	29/05/2026 19:22	File di origine JSON	1 KB
did-document_DID_Document_Doctor_C_bafkreigawoemmn4.json	29/05/2026 19:22	File di origine JSON	1 KB
did-document_DID_Document_Organization_1_bafkreicw6l2q2n.json	29/05/2026 19:22	File di origine JSON	1 KB
did-document_DID_Document_Organization_2_bafkreidm3zscnc.json	29/05/2026 19:22	File di origine JSON	1 KB
did-document_DID_Document_Organization_3_bafkrei5kfaejbp.json	29/05/2026 19:22	File di origine JSON	1 KB
did-document_DID_Document_Patient_bafkreihammogyp.json	29/05/2026 19:22	File di origine JSON	1 KB
did-document_DID_Document_Unauthorized_user_Doctor_D_bafkreiblupg43.json	29/05/2026 19:22	File di origine JSON	1 KB
documento_v1_recuperato.txt	29/05/2026 19:22	Documento di testo	1 KB
documento_v2_recuperato.txt	29/05/2026 19:22	Documento di testo	1 KB
global-policy_Global_Policy_Auditor_can_READ_REFERTO_bafkreiqwilkada.json	29/05/2026 19:22	File di origine JSON	1 KB
global-policy_Global_Policy_Doctor_can_CREATE_REFERTO_bafkreiggyu53en.json	29/05/2026 19:22	File di origine JSON	1 KB
global-policy_Global_Policy_Doctor_can_DELEGATE_REFERTO_bafkreignlstrqij.json	29/05/2026 19:22	File di origine JSON	1 KB
global-policy_Global_Policy_Doctor_can_READ_REFERTO_bafkrebyxkl5ymhj.json	29/05/2026 19:22	File di origine JSON	1 KB
global-policy_Global_Policy_Doctor_can_REVOKE_REFERTO_bafkreibfcbexcnj.json	29/05/2026 19:22	File di origine JSON	1 KB
global-policy_Global_Policy_Doctor_can_UPDATE_REFERTO_bafkreih6weeemc.json	29/05/2026 19:22	File di origine JSON	1 KB
governance-policy_Governance_Policy_MaxDelegationDepth_is_2_bafkreipc2vw7p...	29/05/2026 19:22	File di origine JSON	1 KB
health-document_Documento_sanitario_DOCUMENT-001_versione_1_bafkreianny...	29/05/2026 19:22	Documento di testo	1 KB
health-document_Documento_sanitario_DOCUMENT-001_versione_2_bafkreiahoc2i...	29/05/2026 19:22	Documento di testo	1 KB
offchain-assets-index.json	29/05/2026 19:22	File di origine JSON	9 KB
patient-driven-policy_Patient-driven_Policy_restriction_auditor_READ_DOCUMENT-0...	29/05/2026 19:22	File di origine JSON	2 KB
patient-driven-policy_Patient-driven_Policy_restriction_Doctor_A_READ_DOCUMENT...	29/05/2026 19:22	File di origine JSON	2 KB
selective-disclosure-presentation_Selective_disclosure_presentation_Doctor_A_READ...	29/05/2026 19:22	File di origine JSON	3 KB
verifiable-credential_VC_vc_auditor_ana_auditor-professional_v1_bafkreiglvdthrjs...	29/05/2026 19:22	File di origine JSON	1 KB
verifiable-credential_VC_vc_doctor_alice_medical-professional_v1_bafkreicnkgpg5v...	29/05/2026 19:22	File di origine JSON	1 KB
verifiable-credential_VC_vc_patient_bob_identity_v1_bafkreic7vhzjurm.json	29/05/2026 19:22	File di origine JSON	1 KB

Figura 17 Storage overhead

Gas consumption

Il consumo di gas mostra che il costo on-chain non dipende tanto dalla complessità concettuale dell'operazione, quanto dal **numero di scritture persistenti effettuate sui contratti**. Le operazioni più onerose sono quelle che creano o aggiornano record strutturati, come registrazioni DID, emissione dello stato delle credenziali, creazione di documenti, registrazione di restrizioni patient-driven e gestione delle deleghe. Al contrario, le verifiche di accesso in sola lettura hanno costo nullo in termini di gas quando invocate off-chain.

Di seguito viene riportato un grafico a torta che mostra il **consumo di gas in percentuale per macro-area** rispetto al totale, che è stimato a circa **11.410.058 gas**.

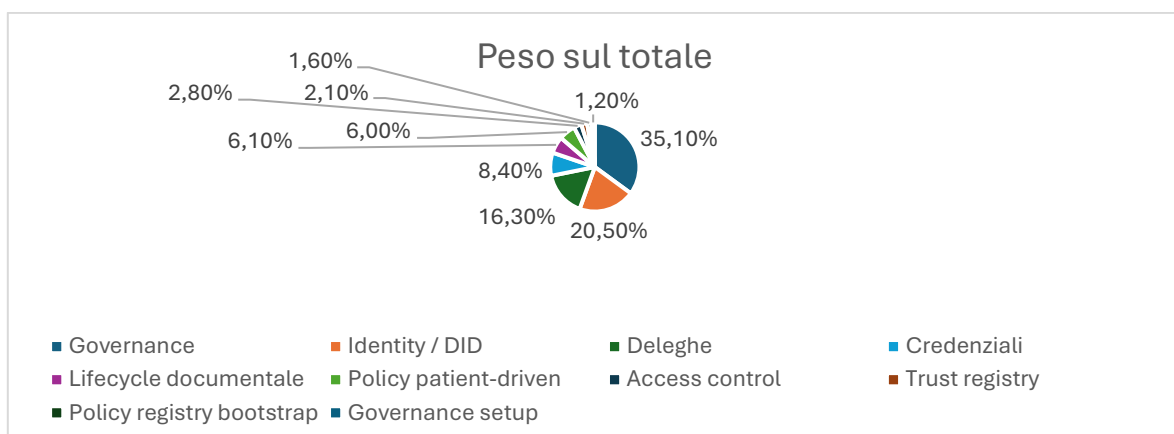


Figura 18 Percentuale consumi di Gas per macro-aree

La componente **più onerosa è la governance**, che rappresenta **circa il 35% del gas totale** consumato. Questo risultato dipende dalla natura multi-step del processo decisionale: ogni modifica alle policy richiede una proposta, uno o più voti e una fase di finalizzazione. Seguono la gestione delle identità DID e il registro delle deleghe, che risultano onerosi perché comportano la creazione di record persistenti e l'aggiornamento di strutture dati on-chain. Al contrario, **l'access control incide in misura minore sul gas** complessivo, poiché molte verifiche vengono eseguite tramite funzioni di sola lettura e non producono transazioni.

Di seguito, sono riportati in forma tabellare i dati sintetizzati nel grafico.

Macro-area	Gas totale	Peso sul totale
Governance	4.001.972	35,1%
Identity / DID	2.341.539	20,5%
Deleghe	1.860.123	16,3%
Credenziali	957.315	8,4%
Lifecycle documentale	695.059	6,1%
Policy patient-driven	681.344	6,0%
Access control	319.560	2,8%
Trust registry	241.068	2,1%
Policy registry bootstrap	176.928	1,6%
Governance setup	135.150	1,2%

Tabella 14 Consumi di Gas per macro-aree

Conclusioni

Nel complesso, l'analisi mostra che il prototipo adotta una logica coerente con gli obiettivi progettuali e che il costo maggiore deriva dalle operazioni che aumentano la sicurezza e la verificabilità del sistema. Si tratta, quindi, di un **trade-off tra efficienza e garanzie**: il sistema sacrifica parte della semplicità e immediatezza di un modello centralizzato per ottenere maggiore tracciabilità, controllo distribuito e verificabilità delle decisioni.

Indice delle figure

Figura 1, Schema ad alto livello scenario d'interesse	5
Figura 2, Accesso a documento cifrato da parte di un utente operativo del sistema	51
Figura 3, Accesso ad un documento cifrato o verifica selettiva da parte di un requester esterno	56
Figura 4, Creazione di un nuovo documento e registrazione nel Document Lifecycle Registry	58
Figura 5, Creazione di una nuova versione di un documento esistente (parte 1)	61
Figura 6, Creazione di una nuova versione di un documento esistente (parte 2)	63
Figura 7, Creazione di un record di delega (parte 1)	64
Figura 8, Creazione di un record di delega (parte 2)	67
Figura 9, Registrazione di un DID e del DID Document associato	81
Figura 10, Auditabilità e ricostruzione ex post	85
Figura 11 Frammento di Codice di DataTypes	97
Figura 12 Frammento di Codice di OrganizationRegistry	99
Figura 13 Latenza operazioni	128
Figura 14 Tempo Cumulativo in relazione al numero di operazioni	129
Figura 15 Confronto tra latenza sequenziale e latenza	130
Figura 16 Throughput locale osservato per tipologia di chiamata	131
Figura 17 Storage overhead	132
Figura 18 Percentuale consumi di Gas per macro-aree	132
Indice delle tabelle	

Indice delle tabelle

Tabella 1,Entità sistema	8
Tabella 2 Riferimenti AuditRegistry	99
Tabella 3 Riferimenti OrganizationRegistry	101
Tabella 4 Riferimenti PolicyGovernance	104
Tabella 5 Riferimenti PolicyRegistry	107
Tabella 6 Riferimenti DocumentLifecycleRegistry	109
Tabella 7 Riferimenti PatientDrivenPolicyRegistry	111
Tabella 8 Riferimenti DelegationRegistry	115
Tabella 9 Riferimenti AccessController	118
Tabella 10 Riferimenti IdentityRegistry	119
Tabella 11 Riferimenti TrustRegistry	121
Tabella 12 Riferimenti CredentialStatusRegistry	124
Tabella 13 Latenza cumulativa	129
Tabella 14 Consumi di Gas per macro-aree	133